

Must-Know Patterns for Placements & Competitive Programming — Part 1

A comprehensive guide covering 16 essential algorithms and data structures for SDE placements, online assessments, and competitive programming.

TABLE OF CONTENTS

1. [Prefix Sum](#)
 2. [Hash Map Counting](#)
 3. [Two Pointers](#)
 4. [Sliding Window](#)
 5. [Binary Search](#)
 6. [Binary Search on Answer](#)
 7. [Sorting + Greedy](#)
 8. [Monotonic Stack](#)
 9. [Heap / Top K](#)
 10. [BFS](#)
 11. [DFS](#)
 12. [Backtracking](#)
 13. [DP \(Dynamic Programming\)](#)
 14. [Tree Recursion](#)
 15. [Graph Traversal](#)
 16. [Union-Find \(DSU\)](#)
-

1. PREFIX SUM

1. Overview

Prefix sum is a preprocessing technique where we create an array `prefix[]` such that `prefix[i]` stores the sum of elements from index `0` to `i` of the original array. Once built, it allows us to answer **range sum queries** in **$O(1)$** time.

It is the simplest and most widely used preprocessing technique in coding interviews and CP.

2. Intuition

Core idea: If you know the total up to any point, the sum between two points is just a subtraction.

Analogy: Imagine a receipt tape where each new total is printed after every item. To know how much you spent between item 3 and item 7, you just look at the total at item 7 and subtract the total at item 2.

Step-by-step reasoning:

1. We have an array `arr = [a0, a1, a2, ..., an-1]`.
2. Build `prefix[i] = arr[0] + arr[1] + ... + arr[i]`.
3. Sum of subarray from `L` to `R` = `prefix[R] - prefix[L-1]` (with `prefix[-1] = 0`).

Why it works: Addition is associative and invertible. The prefix stores cumulative sums, and subtraction undoes the part we don't need.

3. When to Use It

- You need to answer **multiple range sum queries** on a static array.
- Problems asking for **subarray sums** repeatedly.
- Problems involving **contiguous subarrays** with certain sum properties.
- Building frequency prefix arrays for characters (prefix count of 'a', 'b', etc.).

Common trigger phrases:

- "sum of subarray from L to R"
- "number of queries"
- "contiguous subarray"
- "range sum"
- "count of elements in range"

4. When Not to Use It

- The array is **updated frequently** between queries (use Fenwick tree / segment tree instead).
- Only **one query** — just iterate, $O(n)$ is fine.
- Memory is extremely tight and n is huge (though prefix sum is just $O(n)$).
- You need **non-associative operations** (min, max, gcd over range — use segment tree or sparse table).

5. Core Concepts

5.1 One-dimensional Prefix Sum

The standard prefix sum over a 1D array.

```
prefix[i] = prefix[i-1] + arr[i]
range_sum(L, R) = prefix[R] - prefix[L-1]
```

5.2 Two-dimensional Prefix Sum

Extends to 2D matrices. `prefix[i][j]` = sum of rectangle from (0,0) to (i,j).

```
prefix[i][j] = prefix[i-1][j] + prefix[i][j-1] - prefix[i-1][j-1] + mat[i][j]
```

Range sum from (r1,c1) to (r2,c2):

```
sum = prefix[r2][c2] - prefix[r1-1][c2] - prefix[r2][c1-1] + prefix[r1-1][c1-1]
```

5.3 Prefix Frequency

Instead of sum, store counts. Useful for counting how many times a value appears in a range.

For example, prefix count of each character in a string for substring queries.

5.4 Prefix XOR

Same idea but with XOR. Useful because XOR is its own inverse.

```
prefixXor[i] = prefixXor[i-1] ^ arr[i]
range_xor(L, R) = prefixXor[R] ^ prefixXor[L-1]
```

5.5 Difference Array

The inverse of prefix sum. Given an array of differences, reconstructing the prefix gives the original array. Used for range updates.

```
diff[i] = arr[i] - arr[i-1]
range_add(L, R, val): diff[L] += val, diff[R+1] -= val
```

6. Step-by-Step Algorithm

Building prefix sum:

1. Create `prefix` array of size `n` (or `n+1` for convenience).
2. Set `prefix[0] = arr[0]`.
3. For `i = 1` to `n-1`: `prefix[i] = prefix[i-1] + arr[i]`.
4. To answer query `sum(L, R)`:
 - If `L == 0`: return `prefix[R]`.
 - Else: return `prefix[R] - prefix[L-1]`.

1-indexed version (preferred for simplicity):

1. Create `pref` of size `n+1` with `pref[0] = 0`.
2. For `i = 1` to `n`: `pref[i] = pref[i-1] + arr[i-1]`.
3. `sum(L, R)` (0-indexed in original): `pref[R+1] - pref[L]`.

7. Dry Run

Array: `[3, 1, 4, 1, 5, 9, 2]`

Building prefix (1-indexed):

i	arr[i-1]	pref[i] = pref[i-1] + arr[i-1]
0	—	0
1	3	0 + 3 = 3
2	1	3 + 1 = 4
3	4	4 + 4 = 8
4	1	8 + 1 = 9
5	5	9 + 5 = 14
6	9	14 + 9 = 23
7	2	23 + 2 = 25

Queries:

- `sum(1, 4)` (0-indexed: indices 1 to 4 = `[1,4,1,5]`): `pref[5] - pref[1] = 14 - 3 = 11. ✓`
- `sum(0, 2)` (indices 0-2 = `[3,1,4]`): `pref[3] - pref[0] = 8 - 0 = 8. ✓`
- `sum(3, 6)` (indices 3-6 = `[1,5,9,2]`): `pref[7] - pref[3] = 25 - 8 = 17. ✓`

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class PrefixSum {
private:
    vector<long long> pref;
    bool built = false;

public:
    PrefixSum() {}

    // Build prefix sum from array
    void build(const vector<int>& arr) {
        int n = arr.size();
        pref.assign(n + 1, 0);
        for (int i = 1; i <= n; i++) {
            pref[i] = pref[i - 1] + arr[i - 1];
        }
        built = true;
    }

    // Range sum query [l, r] inclusive (0-indexed)
    long long query(int l, int r) {
        if (!built || l > r || l < 0 || r >= (int)pref.size() - 1) {
            return 0; // or throw
        }
        return pref[r + 1] - pref[l];
    }
}
```

```

    // Total sum
    long long total() {
        return pref.back();
    }
};

// Example usage
int main() {
    vector<int> arr = {3, 1, 4, 1, 5, 9, 2};
    PrefixSum ps;
    ps.build(arr);

    cout << "Sum [1,4]: " << ps.query(1, 4) << "\n";    // 11
    cout << "Sum [0,2]: " << ps.query(0, 2) << "\n";    // 8
    cout << "Sum [3,6]: " << ps.query(3, 6) << "\n";    // 17
    cout << "Total: " << ps.total() << "\n";            // 25

    return 0;
}

```

9. Python Implementation

```

from typing import List

class PrefixSum:
    def __init__(self):
        self.pref = []
        self.built = False

    def build(self, arr: List[int]) -> None:
        n = len(arr)
        self.pref = [0] * (n + 1)
        for i in range(1, n + 1):
            self.pref[i] = self.pref[i - 1] + arr[i - 1]
        self.built = True

    def query(self, l: int, r: int) -> int:
        if not self.built or l > r or l < 0 or r >= len(self.pref) - 1:
            return 0
        return self.pref[r + 1] - self.pref[l]

    def total(self) -> int:
        return self.pref[-1] if self.built else 0

# Example usage
if __name__ == "__main__":
    arr = [3, 1, 4, 1, 5, 9, 2]
    ps = PrefixSum()
    ps.build(arr)

```

```
print(f"Sum [1,4]: {ps.query(1, 4)}") # 11
print(f"Sum [0,2]: {ps.query(0, 2)}") # 8
print(f"Sum [3,6]: {ps.query(3, 6)}") # 17
print(f"Total: {ps.total()}") # 25
```

10. Code Explanation

Building phase:

- We allocate `pref` of size `n+1` with `pref[0] = 0`. This avoids special-casing `L = 0`.
- Loop `i = 1` to `n`: `pref[i] = pref[i-1] + arr[i-1]`. Note the off-by-one: `arr[i-1]` corresponds to position `i` in prefix.

Query phase:

- `query(l, r)` returns sum of `arr[l]` through `arr[r]` (inclusive, 0-indexed).
- We return `pref[r+1] - pref[l]`. Why `r+1`? Because `pref[i]` stores sum of first `i` elements (indices 0 to `i-1`).

Why 1-indexed prefix is better:

- `pref[0] = 0` handles the `L = 0` case naturally.
- Formula is always `pref[R+1] - pref[L]` with no if-else.

11. Complexity Analysis

Operation	Time	Space
Building	$O(n)$	$O(n)$
Range query	$O(1)$	$O(1)$
Total	$O(n)$ preprocessing + $O(1)$ per query	$O(n)$

- Best case:** Same as above (no variation).
- Worst case:** Same.
- Note:** If we build in-place (modify original array), space is $O(1)$ extra.

12. Common Patterns

Pattern 1: Subarray Sum Equals K

Identify: Count subarrays with sum = `k`. **Approach:** Use prefix sum + hash map. `pref[i] - pref[j] = k` means `pref[j] = pref[i] - k`. Store frequencies of prefix sums seen so far. **Example:** LeetCode 560 — Subarray Sum Equals K.

Pattern 2: Equilibrium Index / Pivot Index

Identify: Find index where left sum = right sum. **Approach:** Left sum = `pref[i]`, right sum = `total - pref[i+1]`. Compare. **Example:** LeetCode 724 — Find Pivot Index.

Pattern 3: 2D Range Sum Queries

Identify: Multiple queries on submatrix sums. **Approach:** Build 2D prefix sum. **Example:** LeetCode 304 — Range Sum Query 2D — Immutable.

Pattern 4: Subarray with Zero Sum

Identify: Check if any subarray sums to 0. **Approach:** If $\text{pref}[i] == \text{pref}[j]$, sum from $i+1$ to j is 0. **Example:** GFG — Subarray with 0 sum.

Pattern 5: Prefix XOR for Subarray XOR

Identify: Count subarrays with XOR = k . **Approach:** Same as sum pattern but with XOR. $\text{prefXor}[i] \oplus \text{prefXor}[j] = k \rightarrow \text{prefXor}[j] = \text{prefXor}[i] \oplus k$. **Example:** LeetCode 1310 — XOR Queries of a Subarray.

13. Common Mistakes

- **Off-by-one on prefix array size:** Always allocate $n+1$ not n .
- **Forgetting $\text{pref}[0] = 0$:** Without it, query for $L=0$ breaks.
- **Integer overflow:** Use `long long` for sums of large arrays.
- **Confusing 0-indexed vs 1-indexed:** Be consistent. When using 1-indexed prefix, $\text{pref}[i]$ covers $\text{arr}[0..i-1]$.
- **Modifying array after building prefix:** Prefix sum is invalid after updates.
- **Empty array edge case:** $n = 0$ means $\text{pref} = \{0\}$. Queries should handle this.

14. Edge Cases

Case	Input	Expected Behavior
Empty array	<code>[]</code>	$\text{pref} = \{0\}$, any query returns 0
Single element	<code>[5]</code>	$\text{pref} = \{0, 5\}$, $\text{query}(0,0) = 5$
All zeros	<code>[0,0,0]</code>	All queries return 0
Negative numbers	<code>[-5, 2, -3]</code>	Prefix works fine, sum may be negative
Large numbers	<code>[1e9, 1e9, ...]</code>	Use <code>long long</code>
Multiple queries	Many range queries	$O(1)$ each, still fine

15. Variations

Difference Array (Range Update, Point Query)

Instead of prefix of values, build prefix of differences. Apply range updates in $O(1)$, then reconstruct in $O(n)$.

Used in: Problems where you apply many range additions and then query final values. **Example:** LeetCode 370 — Range Addition.

Prefix Sum with Hash Map (Two Sum Pattern)

Use prefix sum with a hash map to find subarrays with a given sum in $O(n)$. **Important for placements:** Very common in interviews (LeetCode 560, 974).

2D Prefix Sum (Immutable Matrix)

Standard for matrix range sum queries. **Used in:** Image processing, game of life, matrix queries.

Prefix Product (with Care)

Product of elements in range. But beware of zeros (they break it). Usually handled with segment tree or sparse table for product queries.

16. Related Algorithms/Data Structures

Structure	When to Use Instead
Fenwick Tree (BIT)	Array is updated between queries. $O(\log n)$ update + query.
Segment Tree	Need range min/max/gcd/product, or need updates.
Sparse Table	Static array, need range min/max/gcd. $O(1)$ query, $O(n \log n)$ build.
Difference Array	Many range updates, few point queries at the end.

Decision guide: Static + sum queries = Prefix Sum. Dynamic + sum queries = Fenwick Tree. Dynamic + non-sum queries = Segment Tree.

17. Practice Problems

Easy

Problem	Platform	Pattern	Rating
Range Sum Query — Immutable	LeetCode 303	1D prefix sum	Easy
Find Pivot Index	LeetCode 724	Prefix sum comparison	Easy

Medium

Problem	Platform	Pattern	Rating
Subarray Sum Equals K	LeetCode 560	Prefix + hash map	Medium
Range Sum Query 2D — Immutable	LeetCode 304	2D prefix sum	Medium
Contiguous Array	LeetCode 525	Prefix + hash map (0/1 $\rightarrow$ -1/1)	Medium

Hard

Problem	Platform	Pattern	Rating
Subarrays with K Different Integers	LeetCode 992	Prefix + sliding window	Hard

Problem	Platform	Pattern	Rating
Maximum Sum of 3 Non-Overlapping Subarrays	LeetCode 689	Prefix + DP	Hard
Count Subarrays With Fixed Bounds	LeetCode 2444	Prefix + two pointers	Hard

18. Interview Explanation

"Prefix sum is a preprocessing technique where we create an array that stores cumulative sums from the start. Once built, we can answer any range sum query in $O(1)$ time by simply subtracting two prefix values. For example, sum from index L to R is $\text{pref}[R] - \text{pref}[L-1]$. I use a 1-indexed prefix array with $\text{pref}[0] = 0$ to avoid edge cases. The build takes $O(n)$ time and $O(n)$ space. It's a building block for harder problems like subarray sum equals k , where we combine it with a hash map."

19. Revision Notes

- **Key idea:** $\text{pref}[i] = \text{sum}(\text{arr}[0..i-1])$, $\text{sum}(L,R) = \text{pref}[R+1] - \text{pref}[L]$
- **Formula:** $\text{pref}[i] = \text{pref}[i-1] + \text{arr}[i-1]$
- **1-indexed:** Always use $\text{size} = n+1$, $\text{pref}[0] = 0$
- **Complexity:** Build $O(n)$, Query $O(1)$, Space $O(n)$
- **Common trap:** Forgetting $\text{pref}[0] = 0$, integer overflow
- **Hash map combo:** For "count subarrays with sum k " — store freq of prefix sums
- **2D version:** $\text{sum} = \text{pref}[r2][c2] - \text{pref}[r1-1][c2] - \text{pref}[r2][c1-1] + \text{pref}[r1-1][c1-1]$

20. Final Cheat Sheet

PREFIX SUM – CHEAT SHEET	
WHEN TO USE:	Static array, multiple range sums
MAIN OP:	$\text{sum}(L,R) = \text{pref}[R+1] - \text{pref}[L]$
BUILD:	$O(n)$
QUERY:	$O(1)$
SPACE:	$O(n)$
KEY CODE:	$\text{pref}[i] = \text{pref}[i-1] + \text{arr}[i-1]$
EDGE CASES:	$n=0$, $L=0$, large numbers, negative
VARIATIONS:	2D prefix, prefix XOR, hash + pref
RELATED:	Fenwick (dynamic), SegTree (non-sum)

2. HASH MAP COUNTING

1. Overview

Hash map counting (also called frequency counting or hash table counting) uses a hash map (`unordered_map` in C++, `dict` in Python) to store frequencies of elements, counts of occurrences, or mappings from keys to

values. It is one of the most versatile tools in competitive programming and interviews.

The key idea: **trade space for time** — use $O(n)$ extra memory to reduce $O(n^2)$ brute force to $O(n)$.

2. Intuition

Core idea: If you could remember everything you've seen so far, you could answer questions about the past instantly.

Analogy: You're at a party and someone asks "How many people here are wearing red shirts?" If you've been keeping a tally on a piece of paper every time someone walked in, you can answer instantly. Otherwise, you'd have to look around the whole room.

Step-by-step reasoning:

1. We need to know something about elements we've seen (frequency, existence, pair matching).
2. A hash map stores key-value pairs with $O(1)$ average lookup.
3. As we iterate, we update the map and query it for information about past elements.
4. This reduces problems that would require nested loops to single pass.

Why it works: Hash maps provide amortized $O(1)$ insertion and lookup. For many problems, the information needed about past elements fits in memory.

3. When to Use It

- Counting frequencies of elements.
- Finding duplicates.
- Two-sum style problems (find a pair that sums to target).
- Checking if an element exists in a collection.
- Grouping elements by some property.
- Caching / memoization results.
- Building adjacency lists for graphs.
- Character counting in strings (anagrams, permutations).

Common trigger phrases:

- "count occurrences"
- "frequency"
- "find if exists"
- "pair that sums to"
- "most frequent"
- "anagram"
- "duplicate"
- "unique"
- "group by"

4. When Not to Use It

- **Small fixed alphabet:** Use a fixed-size array instead (e.g., 26 for lowercase letters, 256 for ASCII). Array is faster and uses less memory.

- **Sorted data, range queries:** Use binary search or prefix sums.
- **Ordered data needed:** Use ordered map (map in C++, TreeMap in Java) or maintain a separate structure.
- **Memory is critical:** Hash maps have overhead (buckets, pointers, hash table resizing).
- **Custom objects without hash functions:** You need to provide a hash function.
- **When $O(n^2)$ is fine:** Small n ($n \leq 1000$) — simpler code may be better.

5. Core Concepts

5.1 Frequency Counting

Count how many times each element appears.

```
unordered_map<int, int> freq;
for (int x : arr) freq[x]++;
// freq[x] gives count of x
```

5.2 Presence / Membership

Check if an element has been seen.

```
unordered_set<int> seen;
if (seen.count(target)) { /* found */ }
```

5.3 Pair / Complement Lookup

Store elements and look for their complement.

```
// Two Sum: find pair that sums to target
unordered_map<int, int> mp; // value -> index
for (int i = 0; i < n; i++) {
    int complement = target - arr[i];
    if (mp.count(complement)) return {mp[complement], i};
    mp[arr[i]] = i;
}
```

5.4 Grouping / Categorization

Group elements by some computed key.

```
// Group anagrams
unordered_map<string, vector<string>> groups;
for (string& s : strs) {
    string key = s;
    sort(key.begin(), key.end());
```

```
    groups[key].push_back(s);  
}
```

5.5 Caching / Memoization

Store computed results to avoid recomputation.

```
unordered_map<int, int> memo;  
int fib(int n) {  
    if (n <= 1) return n;  
    if (memo.count(n)) return memo[n];  
    return memo[n] = fib(n-1) + fib(n-2);  
}
```

5.6 Character Counting

Count characters in strings — useful for anagrams, permutations, substring problems.

6. Step-by-Step Algorithm

Frequency counting:

1. Create empty hash map (`unordered_map`).
2. For each element in input:
 - Increment `map[element]` by 1 (default initializes to 0).
3. After loop, `map[element]` gives frequency.

Two Sum (pair with target sum):

1. Create empty hash map (value $\rightarrow$ index).
2. For each element `arr[i]`:
 - Compute `complement = target - arr[i]`.
 - If complement exists in map, return `{map[complement], i}`.
 - Else, store `map[arr[i]] = i`.
3. If loop ends, no pair found.

7. Dry Run

Problem: Two Sum — find indices of two numbers that add to target.

Input: `arr = [2, 7, 11, 15]`, `target = 9`

Step	i	arr[i]	complement	Map Before	Action
1	0	2	7	{}	Store {2: 0}
2	1	7	2	{2: 0}	Found! Return {0, 1}

Result: `{0, 1}` ($2 + 7 = 9$)

Problem: Count frequencies.

Input: `arr = [1, 2, 2, 3, 3, 3, 4]`

After element	Map State
1	{1: 1}
2	{1: 1, 2: 1}
2	{1: 1, 2: 2}
3	{1: 1, 2: 2, 3: 1}
3	{1: 1, 2: 2, 3: 2}
3	{1: 1, 2: 2, 3: 3}
4	{1: 1, 2: 2, 3: 3, 4: 1}

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Frequency counter class
class FrequencyCounter {
private:
    unordered_map<int, int> freq;

public:
    void add(int x) {
        freq[x]++;
    }

    void add(const vector<int>& arr) {
        for (int x : arr) freq[x]++;
    }

    int getCount(int x) {
        return freq.count(x) ? freq[x] : 0;
    }

    int mostFrequent() {
        int maxFreq = 0, ans = -1;
        for (auto& [val, cnt] : freq) {
            if (cnt > maxFreq) {
                maxFreq = cnt;
                ans = val;
            }
        }
        return ans;
    }
}
```

```

    vector<pair<int, int>> getAllFrequencies() {
        vector<pair<int, int>> res;
        for (auto& p : freq) res.push_back(p);
        return res;
    }
};

// Two Sum using hash map
vector<int> twoSum(const vector<int>& nums, int target) {
    unordered_map<int, int> mp; // value -> index
    for (int i = 0; i < (int)nums.size(); i++) {
        int complement = target - nums[i];
        if (mp.count(complement)) {
            return {mp[complement], i};
        }
        mp[nums[i]] = i;
    }
    return {}; // not found
}

// Example usage
int main() {
    // Frequency counter
    FrequencyCounter fc;
    fc.add({1, 2, 2, 3, 3, 3, 4});
    cout << "Count of 3: " << fc.getCount(3) << "\n"; // 3
    cout << "Most frequent: " << fc.mostFrequent() << "\n"; // 3

    // Two Sum
    vector<int> nums = {2, 7, 11, 15};
    auto res = twoSum(nums, 9);
    cout << "Two Sum: " << res[0] << ", " << res[1] << "\n"; // 0, 1

    return 0;
}

```

9. Python Implementation

```

from collections import defaultdict, Counter
from typing import List, Dict, Tuple, Optional

class FrequencyCounter:
    def __init__(self):
        self.freq: Dict[int, int] = defaultdict(int)

    def add(self, x):
        if isinstance(x, list):
            for val in x:
                self.freq[val] += 1
        else:

```

```

        self.freq[x] += 1

def get_count(self, x: int) -> int:
    return self.freq.get(x, 0)

def most_frequent(self) -> Optional[int]:
    if not self.freq:
        return None
    return max(self.freq.items(), key=lambda p: p[1])[0]

def get_all_frequencies(self) -> List[Tuple[int, int]]:
    return list(self.freq.items())

def two_sum(nums: List[int], target: int) -> List[int]:
    mp = {} # value -> index
    for i, num in enumerate(nums):
        complement = target - num
        if complement in mp:
            return [mp[complement], i]
        mp[num] = i
    return []

# Example usage
if __name__ == "__main__":
    fc = FrequencyCounter()
    fc.add([1, 2, 2, 3, 3, 3, 4])
    print(f"Count of 3: {fc.get_count(3)}") # 3
    print(f"Most frequent: {fc.most_frequent()}") # 3

    nums = [2, 7, 11, 15]
    print(f"Two Sum: {two_sum(nums, 9)}") # [0, 1]

```

10. Code Explanation

FrequencyCounter class:

- Uses `unordered_map<int, int>` — each key maps to its count.
- `add(x)`: Increments count for `x`. If `x` doesn't exist, `freq[x]` default-constructs to 0, then increments.
- `getCount(x)`: Checks if key exists first, then returns value. Empty check avoids inserting a new key.
- `mostFrequent()`: Iterates through map to find max frequency. $O(n)$ each call.

Two Sum:

- Iterate through array once.
- For each element, compute `complement = target - nums[i]`.
- If complement in map, we found a pair.
- Otherwise, store current element's index.
- Single pass: $O(n)$ time, $O(n)$ space.

11. Complexity Analysis

Operation	Average Time	Worst Case	Space
Insert (single)	$O(1)$	$O(n)$	$O(1)$
Lookup	$O(1)$	$O(n)$	$O(1)$
Delete	$O(1)$	$O(n)$	$O(1)$
Iterate over all	$O(k)$	$O(k)$	$O(1)$
Two Sum (single pass)	$O(n)$	$O(n)$	$O(n)$

Note: Worst case $O(n)$ for hash map operations happens with poor hash functions or hash collisions. For CP, `unordered_map` is usually fine. Use `map` ($O(\log n)$) if worst-case matters.

12. Common Patterns

Pattern 1: Two Sum / Complement Search

How to identify: "Find pair with sum/difference/product = k" **Approach:** Store elements in map, look for complement. **Examples:** Two Sum (LeetCode 1), Two Sum II (167), 3Sum (15).

Pattern 2: Frequency Counting

How to identify: "Count occurrences", "most frequent", "mode" **Approach:** Simple frequency map. **Examples:** Majority Element (169), Top K Frequent (347).

Pattern 3: Anagram / Character Count

How to identify: "Anagram", "permutation", "rearrangement" **Approach:** Count characters in both strings, compare maps. **Examples:** Valid Anagram (242), Group Anagrams (49).

Pattern 4: Subarray / Substring with Property

How to identify: "Subarray sum = k", "longest substring with k distinct" **Approach:** Combine hash map with prefix sum or sliding window. **Examples:** Subarray Sum Equals K (560), Longest Substring Without Repeating (3).

Pattern 5: Duplicate Detection

How to identify: "Contains duplicate", "first repeating" **Approach:** Use set/map while iterating. **Examples:** Contains Duplicate (217), Contains Duplicate II (219).

Pattern 6: Caching / Memoization

How to identify: Overlapping subproblems in recursion. **Approach:** Store computed results in map. **Examples:** Fibonacci, DP problems.

13. Common Mistakes

- **Using [] operator for lookup that inserts:** `if (mp[key])` creates key if missing. Use `mp.count(key)` or `mp.find(key)`.
- **Forgetting hash function for custom keys:** Need to provide `std::hash` specialization or use `map`.
- **Assuming O(1) always:** Hash collisions can degrade to O(n). In CP contests, some problems are designed to cause collisions on `unordered_map`.
- **Not handling duplicates in Two Sum:** The problem usually asks for distinct indices. Store the first occurrence only.
- **Integer overflow in complement:** `target - nums[i]` can overflow if using `int` with large values. Use `long long`.
- **Modifying map while iterating:** Can invalidate iterators. Be careful.

14. Edge Cases

Case	Input	Expected Behavior
Empty array	<code>[]</code>	Map stays empty, returns default values
Single element	<code>[5]</code>	<code>getCount(5) = 1</code> , Two Sum not found
All same	<code>[3,3,3,3]</code>	<code>getCount(3) = 4</code>
No pair found	Two Sum with no solution	Return empty or <code>{-1, -1}</code>
Duplicate values	Two Sum with duplicates	<code>[3,3]</code> target=6 → <code>{0,1}</code>
Large values	<code>target = 2e9</code>	Use <code>long long</code>
Zero values	<code>[0, 0]</code> target=0	Works fine, <code>{0, 1}</code>

15. Variations

Unordered Map vs Ordered Map

- `unordered_map`: O(1) average, O(n) worst. No ordering.
- `map` (Red-Black Tree): O(log n) guaranteed. Elements sorted by key.
- **Use map when:** Order matters, worst-case O(1) is critical, or custom hash is complex.

Multiset / Bag

- `unordered_multiset`: Allows duplicates, but can't easily get count without iterating.
- Usually better to use `unordered_map<key, int>` for explicit counts.

Set (No Value, Just Key)

- Use `unordered_set` when you only need to know if something exists.
- Lighter than map when you don't need associated values.

Counter in Python

- `collections.Counter` is a specialized dict for counting.
- `Counter(arr)` builds frequency map in one line.
- Supports `most_common()`, arithmetic operations, etc.

16. Related Algorithms/Data Structures

Structure	When to Use Instead
Array (fixed size)	Small known alphabet (26 letters, 256 chars) — faster and memory efficient.
Set	Only need existence, not counts.
Ordered Map	Need sorted keys, or guarantee $O(\log n)$ worst case.
Trie	Prefix-based string operations.
Bloom Filter	Probabilistic membership with space efficiency.
Sort + Two Pointers	Alternative to hash map for pair problems when memory is limited.

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Two Sum	LeetCode 1	Complement lookup	Easy
Valid Anagram	LeetCode 242	Character counting	Easy

Medium

Problem	Platform	Pattern	Difficulty
Group Anagrams	LeetCode 49	Grouping by sorted key	Medium
Top K Frequent Elements	LeetCode 347	Frequency + heap	Medium
Subarray Sum Equals K	LeetCode 560	Prefix + hash map	Medium

Hard

Problem	Platform	Pattern	Difficulty
Longest Consecutive Sequence	LeetCode 128	Set-based grouping	Medium/Hard
Minimum Window Substring	LeetCode 76	Character count + sliding window	Hard
Count of Range Sum	LeetCode 327	Prefix + merge sort / BIT	Hard

18. Interview Explanation

"Hash map counting is a technique where we use a hash table to store frequencies or mappings, enabling $O(1)$ average lookup. The core idea is trading space for time — we store information about elements we've seen so we can answer queries about them instantly. The most common application is frequency counting, where we increment counts as we iterate. Another key pattern is complement lookup, as in Two Sum, where we store elements and check if their complement exists. In C++, I use

unordered_map for average O(1), but I'm careful to use count() or find() to avoid inadvertent insertion with the bracket operator."

19. Revision Notes

- **Key idea:** Store info about past elements for O(1) lookup.
- **Two Sum:** `complement = target - arr[i]`, check map, then store.
- **Frequency:** `freq[x]++` (default-initializes to 0).
- **Lookup without insert:** Use `mp.count(key)` or `mp.find(key)`.
- **Complexity:** O(n) time, O(n) space (average).
- **Common trap:** `mp[key]` in condition creates key if missing.
- **Array alternative:** For 26 lowercase letters, use `int cnt[26]`.
- **Python:** `Counter(arr)` or `defaultdict(int)`.

20. Final Cheat Sheet

HASH MAP COUNTING – CHEAT SHEET	
WHEN TO USE:	Counting, existence, pair lookup
MAIN OP:	insert O(1), lookup O(1)
SPACE:	O(n)
KEY CODE:	<code>freq[x]++</code>
TWO SUM:	<code>if (mp.count(target - x)) return</code> <code>mp[x] = i</code>
AVOID:	<code>mp[key]</code> in condition (inserts!)
EDGE CASES:	empty, duplicates, no match
ALTERNATIVE:	array if alphabet is small
RELATED:	Set, Ordered Map, Counter (Python)

3. TWO POINTERS

1. Overview

Two pointers is a technique where we use two index variables (pointers) to traverse a data structure, typically an array or string. The pointers move toward each other, in the same direction, or at different speeds, to solve problems efficiently — often reducing O(n²) to O(n).

2. Intuition

Core idea: Instead of nested loops checking all pairs, use two pointers that move intelligently based on conditions, skipping unnecessary comparisons.

Analogy: Two people looking for a specific sum in a sorted list of prices. One starts at the cheapest item (left), one at the most expensive (right). If their sum is too small, the cheap person moves to a slightly more

expensive item. If too large, the expensive person moves to a cheaper one. They meet in the middle, finding the answer in one pass.

Step-by-step reasoning:

1. **Sorted array, pair sum:** Left at 0, right at $n-1$. If $\text{sum} < \text{target}$, $\text{left}++$. If $\text{sum} > \text{target}$, $\text{right}--$. If equal, found.
2. **In-place removal:** One pointer reads elements, one writes the result.
3. **Fast and slow pointers:** One moves 2 steps, one moves 1 step. Used for cycle detection, middle of linked list.

Why it works: The pointers move monotonically (always forward or always inward), and each step eliminates a range of possibilities. The total number of moves is $O(n)$.

3. When to Use It

- **Sorted array** and need to find pairs with a property.
- **In-place modification** of array (remove duplicates, partition).
- **Palindrome checking** in strings.
- **Linked list** cycle detection, middle element.
- **Merging** two sorted arrays.
- **Container with most water** / trapping rain water type problems.
- **Three sum** / four sum (combined with sorting).

Common trigger phrases:

- "sorted array"
- "pair sum"
- "in-place"
- "remove duplicates"
- "palindrome"
- "three sum"
- "cycle detection"
- "move zeros"
- "container with most water"

4. When Not to Use It

- **Unsorted array** and you need to find pairs (sorting first is an option, but if $O(n)$ space is needed, use hash map).
- **Three or more pointers needed** (k -sum with $k > 2$): Two pointers can still be used after fixing outer elements, but complexity increases.
- **Non-monotonic property:** The condition must allow deterministic pointer movement. If knowing whether sum is too small doesn't tell you which pointer to move, two pointers won't work.
- **Problems requiring random access patterns:** Two pointers are linear scan techniques.

5. Core Concepts

5.1 Opposite Direction (Inward/Outward)

Two pointers start at opposite ends and move toward each other.

Used for: Pair sum in sorted array, palindrome checking, two-sum in sorted array.

```
int left = 0, right = n - 1;
while (left < right) {
    // process arr[left] and arr[right]
    if (condition) left++;
    else right--;
}
```

5.2 Same Direction (Fast and Slow / Sliding Window)

Both pointers start at the same end and move forward, possibly at different speeds.

Used for: Removing duplicates, in-place partitioning, finding subarrays.

```
int slow = 0;
for (int fast = 0; fast < n; fast++) {
    if (condition(arr[fast])) {
        arr[slow++] = arr[fast];
    }
}
```

5.3 Fast and Slow (Tortoise and Hare)

One pointer moves 2 steps, the other moves 1 step. Used for cycle detection in linked lists.

```
ListNode* slow = head, *fast = head;
while (fast && fast->next) {
    slow = slow->next;
    fast = fast->next->next;
    if (slow == fast) { /* cycle found */ }
}
```

5.4 Fixed Window / Sliding Window

Two pointers maintain a window. Right expands, left contracts.

Used for: Subarray problems with constraints.

6. Step-by-Step Algorithm

Two Sum in Sorted Array (Opposite Direction):

1. Sort array if not sorted ($O(n \log n)$).
2. Initialize `left = 0, right = n-1`.

3. While `left < right`:
 - `sum = arr[left] + arr[right]`.
 - If `sum == target`: return `{left, right}`.
 - If `sum < target`: `left++` (need larger sum).
 - If `sum > target`: `right--` (need smaller sum).
4. If loop ends, no pair found.

Remove Duplicates from Sorted Array (Same Direction):

1. If `n == 0`: return 0.
2. `writeIdx = 1` (first element is always kept).
3. For `i = 1` to `n-1`:
 - If `arr[i] != arr[i-1]`: `arr[writeIdx++] = arr[i]`.
4. Return `writeIdx` (new length).

7. Dry Run

Problem: Two Sum in Sorted Array

Input: `arr = [-3, -1, 0, 2, 4, 7]`, `target = 6`

Step	left	right	arr[left]	arr[right]	sum	Action
1	0	5	-3	7	4	sum < 6, left++
2	1	5	-1	7	6	FOUND! Return {1, 5}

Result: `{1, 5} → -1 + 7 = 6`

Problem: Remove Duplicates from Sorted Array

Input: `arr = [0, 0, 1, 1, 1, 2, 2, 3, 3, 4]`

i	arr[i]	arr[i-1]	arr[i] != arr[i-1]?	writeIdx	Array after write
—	—	—	—	1	<code>[0, ...]</code>
1	0	0	No	1	—
2	1	0	Yes	2	<code>[0, 1, ...]</code>
3	1	1	No	2	—
4	1	1	No	2	—
5	2	1	Yes	3	<code>[0, 1, 2, ...]</code>
6	2	2	No	3	—
7	3	2	Yes	4	<code>[0, 1, 2, 3, ...]</code>
8	3	3	No	4	—
9	4	3	Yes	5	<code>[0, 1, 2, 3, 4]</code>

Result: New length = 5, array = [0, 1, 2, 3, 4]

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Two Sum in Sorted Array (opposite direction)
pair<int, int> twoSumSorted(const vector<int>& arr, int target) {
    int left = 0, right = arr.size() - 1;
    while (left < right) {
        int sum = arr[left] + arr[right];
        if (sum == target) return {left, right};
        if (sum < target) left++;
        else right--;
    }
    return {-1, -1}; // not found
}

// Remove Duplicates from Sorted Array (same direction)
int removeDuplicates(vector<int>& arr) {
    int n = arr.size();
    if (n == 0) return 0;

    int writeIdx = 1;
    for (int i = 1; i < n; i++) {
        if (arr[i] != arr[i - 1]) {
            arr[writeIdx++] = arr[i];
        }
    }
    return writeIdx;
}

// Three Sum: find all triplets that sum to 0
vector<vector<int>> threeSum(vector<int>& nums) {
    sort(nums.begin(), nums.end());
    vector<vector<int>> result;
    int n = nums.size();

    for (int i = 0; i < n - 2; i++) {
        // Skip duplicates for i
        if (i > 0 && nums[i] == nums[i - 1]) continue;

        int left = i + 1, right = n - 1;
        int target = -nums[i];

        while (left < right) {
            int sum = nums[left] + nums[right];
            if (sum == target) {
                result.push_back({nums[i], nums[left], nums[right]});
                // Skip duplicates
                while (left < right && nums[left] == nums[left + 1]) left++;
            }
        }
    }
}
```

```

        while (left < right && nums[right] == nums[right - 1]) right--;
        left++;
        right--;
    } else if (sum < target) {
        left++;
    } else {
        right--;
    }
}
}
return result;
}

// Container With Most Water
int maxArea(const vector<int>& height) {
    int left = 0, right = height.size() - 1;
    int maxWater = 0;

    while (left < right) {
        int h = min(height[left], height[right]);
        int w = right - left;
        maxWater = max(maxWater, h * w);

        // Move the shorter line inward
        if (height[left] < height[right]) left++;
        else right--;
    }
    return maxWater;
}

// Example usage
int main() {
    // Two Sum Sorted
    vector<int> arr1 = {-3, -1, 0, 2, 4, 7};
    auto p = twoSumSorted(arr1, 6);
    cout << "Two Sum: " << p.first << ", " << p.second << "\n"; // 1, 5

    // Remove Duplicates
    vector<int> arr2 = {0, 0, 1, 1, 1, 2, 2, 3, 3, 4};
    int newLen = removeDuplicates(arr2);
    cout << "After remove duplicates: ";
    for (int i = 0; i < newLen; i++) cout << arr2[i] << " ";
    cout << "\n"; // 0 1 2 3 4

    // Three Sum
    vector<int> arr3 = {-1, 0, 1, 2, -1, -4};
    auto triplets = threeSum(arr3);
    cout << "Three Sum:\n";
    for (auto& t : triplets) {
        cout << " " << t[0] << " " << t[1] << " " << t[2] << "\n";
    }

    // Container With Most Water
    vector<int> heights = {1, 8, 6, 2, 5, 4, 8, 3, 7};

```

```
    cout << "Max Water: " << maxArea(heights) << "\n"; // 49

    return 0;
}
```

9. Python Implementation

```
from typing import List, Tuple

def two_sum_sorted(arr: List[int], target: int) -> Tuple[int, int]:
    left, right = 0, len(arr) - 1
    while left < right:
        s = arr[left] + arr[right]
        if s == target:
            return (left, right)
        if s < target:
            left += 1
        else:
            right -= 1
    return (-1, -1)

def remove_duplicates(arr: List[int]) -> int:
    n = len(arr)
    if n == 0:
        return 0
    write_idx = 1
    for i in range(1, n):
        if arr[i] != arr[i - 1]:
            arr[write_idx] = arr[i]
            write_idx += 1
    return write_idx

def three_sum(nums: List[int]) -> List[List[int]]:
    nums.sort()
    result = []
    n = len(nums)

    for i in range(n - 2):
        if i > 0 and nums[i] == nums[i - 1]:
            continue

        left, right = i + 1, n - 1
        target = -nums[i]

        while left < right:
            s = nums[left] + nums[right]
            if s == target:
                result.append([nums[i], nums[left], nums[right]])
                while left < right and nums[left] == nums[left + 1]:
                    left += 1
                while left < right and nums[right] == nums[right - 1]:
```

```

        right -= 1
        left += 1
        right -= 1
    elif s < target:
        left += 1
    else:
        right -= 1
    return result

def max_area(height: List[int]) -> int:
    left, right = 0, len(height) - 1
    max_water = 0
    while left < right:
        h = min(height[left], height[right])
        w = right - left
        max_water = max(max_water, h * w)
        if height[left] < height[right]:
            left += 1
        else:
            right -= 1
    return max_water

# Example usage
if __name__ == "__main__":
    print(two_sum_sorted([-3, -1, 0, 2, 4, 7], 6))          # (1, 5)

    arr = [0, 0, 1, 1, 1, 2, 2, 3, 3, 4]
    new_len = remove_duplicates(arr)
    print(arr[:new_len])                                    # [0, 1, 2, 3, 4]

    print(three_sum([-1, 0, 1, 2, -1, -4]))                 # [[-1, -1, 2], [-1,
0, 1]]

    print(max_area([1, 8, 6, 2, 5, 4, 8, 3, 7]))           # 49

```

10. Code Explanation

Two Sum Sorted:

- `left` and `right` start at opposite ends.
- If sum is too small, we need a larger value → move left forward.
- If sum is too large, we need a smaller value → move right backward.
- The monotonic movement ensures we don't miss any pair.
- **Why no pair is missed:** At each step, we eliminate either the current left (all pairs with this left and any right > current are too small) or the current right (all pairs with this right and any left < current are too large).

Remove Duplicates:

- `writeIdx` is the position where the next unique element goes.
- Since the array is sorted, duplicates are adjacent.

- `arr[i] != arr[i-1]` detects the first occurrence of a new value.
- We write it to `arr[writeIdx]` and increment.

Three Sum:

- Fix one element (`nums[i]`), then use two pointers on the rest.
- Sort first to enable two-pointer technique.
- Skip duplicates for the fixed element and for the two-pointer pair.
- Target is `-nums[i]` because we want `nums[i] + nums[left] + nums[right] = 0`.

Container With Most Water:

- Area = `min(height[left], height[right]) * (right - left)`.
- The shorter line limits the water. Moving it inward might find a taller line.
- If we move the taller line, the width decreases and height is at most the same → area can only decrease.
- So we always move the shorter line.

11. Complexity Analysis

Algorithm	Time	Space	Notes
Two Sum (sorted)	$O(n)$	$O(1)$	Already sorted
Two Sum (unsorted + sort)	$O(n \log n)$	$O(1)$ or $O(n)$	Sorting dominates
Remove Duplicates	$O(n)$	$O(1)$	In-place
Three Sum	$O(n^2)$	$O(1)$ (excluding output)	Sorting $O(n \log n)$ + nested loop $O(n^2)$
Container With Most Water	$O(n)$	$O(1)$	Single pass
Palindrome Check	$O(n)$	$O(1)$	Two pointers from ends

12. Common Patterns

Pattern 1: Pair Sum in Sorted Array

Identify: Sorted array, find pair with given sum. **Approach:** Left at 0, right at $n-1$. Move based on sum comparison. **Examples:** LeetCode 167 — Two Sum II, LeetCode 1 — Two Sum (with sort).

Pattern 2: In-Place Array Modification

Identify: "Remove duplicates", "move zeros", "partition array". **Approach:** Slow/fast pointer. Fast scans, slow writes the result. **Examples:** LeetCode 26 — Remove Duplicates, LeetCode 283 — Move Zeros.

Pattern 3: K-Sum Problems

Identify: "Find k elements that sum to target". **Approach:** Sort, fix $k-2$ elements, use two pointers on remaining. **Examples:** LeetCode 15 — 3Sum, LeetCode 18 — 4Sum.

Pattern 4: Container / Trapping Rain Water

Identify: "Max area", "trapping water", "container". **Approach:** Two pointers from ends, move the shorter one.

Examples: LeetCode 11 — Container With Most Water, LeetCode 42 — Trapping Rain Water.

Pattern 5: Palindrome / String Validation

Identify: "Palindrome", "valid palindrome". **Approach:** Two pointers from ends, skip non-alphanumeric characters. **Examples:** LeetCode 125 — Valid Palindrome, LeetCode 680 — Valid Palindrome II.

Pattern 6: Linked List — Cycle Detection

Identify: "Cycle in linked list", "find middle". **Approach:** Fast and slow pointers. **Examples:** LeetCode 141 — Linked List Cycle, LeetCode 876 — Middle of Linked List.

13. Common Mistakes

- **Forgetting to sort first:** Two pointers on unsorted array doesn't work for pair sum.
- **Infinite loop:** Not moving pointers in one of the branches.
- **Skipping duplicates incorrectly:** Wrong condition for duplicate skipping can miss valid pairs.
- **Off-by-one in while condition:** `left < right` vs `left <= right`. For pairs, use `left < right`.
- **Not handling pointer movement after found:** After finding a pair, move both pointers to avoid infinite loop.
- **Slow/fast pointer on linked list:** Forgetting to check `fast != nullptr && fast->next != nullptr` in while condition.
- **Modifying original array unintentionally:** When using in-place techniques, be clear about what you're overwriting.

14. Edge Cases

Case	Expected Behavior
Empty array	Return empty / -1 for pair; return 0 for remove duplicates
Single element	No pair possible; remove duplicates returns 1
Two elements	Pair sum works if they sum to target
All same values	Pair sum: only works if <code>2 * val == target</code> ; remove duplicates: returns 1
No pair found	Return {-1, -1} or empty
Target is 0 with negative numbers	Works fine
Very large or small values	Use <code>long long</code> for sum
Unsorted input for remove duplicates	Algorithm assumes sorted input
Negative numbers in sorted array	Works fine, two pointers handle negative+positive

15. Variations

Three Pointers

Used for specific problems like sorting an array of 0s, 1s, and 2s (Dutch National Flag problem). Three pointers: low, mid, high.

Two Pointers on Two Arrays

Merge two sorted arrays: one pointer on each array, advancing the smaller one.

Two Pointers with Sliding Window

Right pointer expands window, left pointer contracts. Often combined with hash map for constraints.

Two Pointers from Center (Expand Around Center)

Used for palindrome substring problems. Start from each center and expand outward. **Example:** LeetCode 5 — Longest Palindromic Substring.

16. Related Algorithms/Data Structures

Technique	Connection
Sliding Window	Same-direction two pointers, but window size typically varies.
Binary Search	Another way to find elements in sorted array. Two pointers is often better for pair problems.
Hash Map	Alternative for pair problems (Two Sum with unsorted array).
Sorting	Prerequisite for many two-pointer problems.
Fast & Slow (Floyd's)	Specialized for linked list cycles.

Decision guide:

- Sorted array, find pair → Two pointers ($O(n)$, $O(1)$ space).
- Unsorted array, find pair → Hash map ($O(n)$, $O(n)$ space) or sort + two pointers ($O(n \log n)$, $O(1)$ space).
- In-place deletion → Two pointers (slow/fast).
- K elements sum → Sort + two pointers (after fixing $k-2$ elements).

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Two Sum II — Input Array Is Sorted	LeetCode 167	Opposite direction	Easy
Remove Duplicates from Sorted Array	LeetCode 26	Same direction	Easy

Medium

Problem	Platform	Pattern	Difficulty
3Sum	LeetCode 15	K-sum with two pointers	Medium
Container With Most Water	LeetCode 11	Opposite direction optimization	Medium
Sort Colors (Dutch National Flag)	LeetCode 75	Three pointers	Medium

Hard

Problem	Platform	Pattern	Difficulty
Trapping Rain Water	LeetCode 42	Two pointers with max tracking	Hard
4Sum	LeetCode 18	K-sum with nested two pointers	Medium/Hard
Minimum Window Substring	LeetCode 76	Two pointers + hash map	Hard

18. Interview Explanation

"Two pointers is a technique where we use two index variables to traverse an array, typically from opposite ends or in the same direction. For sorted arrays, we can find pairs in $O(n)$ time by moving pointers based on whether the current sum is too small or too large. For in-place modifications like removing duplicates, we use a slow pointer that writes and a fast pointer that scans. The key insight is that each pointer moves monotonically, so we make at most $O(n)$ total moves. I use this for problems like Two Sum in sorted arrays, removing duplicates, three sum, and container with most water."

19. Revision Notes

- **Opposite direction:** `left = 0, right = n-1; while (left < right)`
- **Same direction:** `slow = 0; for (fast: 0..n-1) if (cond) arr[slow++] = arr[fast]`
- **Pair sum:** `sum < target → left++, sum > target → right--`
- **Three Sum:** Sort, fix i , then two pointers on $i+1..n-1$
- **Container:** Move the shorter line inward
- **Complexity:** $O(n)$ for most two-pointer, $O(n^2)$ for 3Sum
- **Common trap:** Forgetting to sort first for pair sum

20. Final Cheat Sheet

TWO POINTERS – CHEAT SHEET	
WHEN TO USE:	Sorted array, pair problems, in-place modification, palindrome
TYPES:	Opposite direction, same direction, fast & slow, sliding window
KEY PATTERN:	<code>left=0, right=n-1, while(l<r)</code>
2-SUM:	<code>if sum<target → l++ else r--</code>
REMOVE DUP:	<code>slow=1, for i=1..n if diff → write</code>
COMPLEXITY:	$O(n)$ time, $O(1)$ space
EDGE CASES:	empty, single, all same, no pair

RELATED: Sliding window, binary search, map

4. SLIDING WINDOW

1. Overview

Sliding window is a technique where we maintain a contiguous subarray (window) of the original array and slide it across the data. The window can be fixed-size or variable-size. It converts $O(n^2)$ nested loop problems into $O(n)$ single-pass solutions by efficiently updating the window state as we move.

2. Intuition

Core idea: Instead of recomputing the result for every subarray from scratch, we reuse computation from the previous window. When the window slides, we remove the left element and add the right element — only $O(1)$ work per slide.

Analogy: You're looking through a train window at a landscape. As the train moves, you don't need to re-describe the entire view from scratch — you just note what comes into view on the right and what leaves on the left.

Step-by-step reasoning:

1. Start with a window covering the first k elements (or starting empty for variable size).
2. Process the window (compute sum, check condition, etc.).
3. Slide the window: remove the element at the left, add the element at the right.
4. Update the result based on the new window.
5. Repeat until the window reaches the end.

Why it works: The window state is updated incrementally. We only do $O(1)$ work per slide instead of $O(k)$ work per subarray.

3. When to Use It

- **Fixed-size window:** Find max/min/average of all subarrays of size k .
- **Variable-size window:** Find longest/shortest subarray satisfying a condition (sum $\leq k$, at most k distinct characters, etc.).
- **Substring problems** with character constraints (at most k distinct, no repeating chars).
- **String anagram / permutation** matching.
- **Problems phrased as:** "subarray", "contiguous", "substring", "window", "consecutive".

Common trigger phrases:

- "subarray of size k "
- "maximum sum of subarray"
- "longest substring without repeating"
- "at most k distinct"

- "subarray with sum $\leq k$ "
- "minimum window"
- "contains all characters"

4. When Not to Use It

- **Non-contiguous subsequences:** Sliding window only works for contiguous subarrays/substrings.
- **Negative numbers in sum-constrained variable window:** If array has negative numbers, a window that exceeds the sum might become valid again by adding more elements. Sliding window doesn't handle this — use prefix sum instead.
- **Condition is not monotonic:** The condition must be such that expanding the window makes it harder to satisfy (or easier). If the relationship is not monotonic, sliding window may fail.
- **Small n:** $O(n^2)$ might be fine for $n \leq 1000$.
- **Need sorted order:** Sliding window works on the original order, not sorted.

5. Core Concepts

5.1 Fixed-Size Window

Window size is constant (k). We slide it one step at a time.

```
int windowSum = 0;
for (int i = 0; i < k; i++) windowSum += arr[i];
int maxSum = windowSum;
for (int i = k; i < n; i++) {
    windowSum += arr[i] - arr[i - k];
    maxSum = max(maxSum, windowSum);
}
```

5.2 Variable-Size (Expanding/Shrinking) Window

Window size changes based on a condition. Right pointer expands, left pointer shrinks when condition is violated.

```

int left = 0;
for (int right = 0; right < n; right++) {
    // Add arr[right] to window state
    updateState(arr[right], +1);

    // Shrink window while condition is violated
    while (conditionViolated()) {
        updateState(arr[left], -1);
        left++;
    }

    // Update result – window [left, right] is valid
    result = max(result, right - left + 1);
}

```

5.3 Window State

The data structure that tracks information about the current window. Common examples:

- **Sum:** Just a running sum variable.
- **Character frequencies:** Array of size 26 or 256, or a hash map.
- **Distinct count:** Integer counter + hash map of frequencies.
- **Set of elements:** Hash set.

5.4 Monotonicity Requirement

For variable-size window:

- If the window is valid, any sub-window (smaller) is also valid.
- If the window is invalid, any super-window (larger) is also invalid.

This is why shrinking fixes invalidity, and expanding can only break validity.

6. Step-by-Step Algorithm

Fixed-Size Window (Maximum Sum of Subarray of Size K):

1. Compute sum of first k elements: `windowSum = sum(arr[0..k-1])`.
2. Set `maxSum = windowSum`.
3. For `i = k` to `n-1`:
 - `windowSum = windowSum + arr[i] - arr[i-k]`.
 - `maxSum = max(maxSum, windowSum)`.
4. Return `maxSum`.

Variable-Size Window (Longest Substring Without Repeating Characters):

1. Initialize `left = 0`, `maxLen = 0`, empty map for character positions.
2. For `right = 0` to `n-1`:
 - If `s[right]` is already in the map and its index $\geq$ `left`:
 - Move `left` to `map[s[right]] + 1` (skip past the duplicate).

- Update `map[s[right]] = right`.
- `maxLen = max(maxLen, right - left + 1)`.

3. Return `maxLen`.

7. Dry Run

Problem: Maximum Sum of Subarray of Size K = 3

Input: `arr = [2, 1, 5, 1, 3, 2]`, `k = 3`

Window	Elements	Sum	Max
[0,2]	2, 1, 5	8	8
[1,3]	1, 5, 1	$8 - 2 + 1 = 7$	8
[2,4]	5, 1, 3	$7 - 1 + 3 = 9$	9
[3,5]	1, 3, 2	$9 - 5 + 2 = 6$	9

Result: Maximum sum = 9 (subarray [5, 1, 3])

Problem: Longest Substring Without Repeating Characters

Input: `s = "abcabcbb"`

right	char	Map	left	Window	maxLen
0	'a'	{a:0}	0	"a"	1
1	'b'	{a:0, b:1}	0	"ab"	2
2	'c'	{a:0, b:1, c:2}	0	"abc"	3
3	'a'	found at 0 $\geq$ left	1	"bca"	3
4	'b'	found at 1 $\geq$ left	2	"cab"	3
5	'c'	found at 2 $\geq$ left	3	"abc"	3
6	'b'	found at 4 $\geq$ left	5	"b"	3
7	'b'	found at 6 $\geq$ left	7	""	3

Result: 3 ("abc")

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// 1. Fixed-size: Maximum sum of subarray of size k
int maxSumSubarrayOfSizeK(const vector<int>& arr, int k) {
    int n = arr.size();
```

```

    if (n < k) return -1;

    int windowSum = 0;
    for (int i = 0; i < k; i++) windowSum += arr[i];

    int maxSum = windowSum;
    for (int i = k; i < n; i++) {
        windowSum += arr[i] - arr[i - k];
        maxSum = max(maxSum, windowSum);
    }
    return maxSum;
}

// 2. Variable-size: Longest substring without repeating characters
int longestSubstringWithoutRepeating(const string& s) {
    int n = s.length();
    vector<int> lastIndex(256, -1); // for all ASCII chars
    int left = 0, maxLen = 0;

    for (int right = 0; right < n; right++) {
        char c = s[right];
        // If char was seen and is within current window, move left
        if (lastIndex[c] >= left) {
            left = lastIndex[c] + 1;
        }
        lastIndex[c] = right;
        maxLen = max(maxLen, right - left + 1);
    }
    return maxLen;
}

// 3. Variable-size: Longest subarray with sum ≤ k
int longestSubarraySumAtMostK(const vector<int>& arr, int k) {
    int n = arr.size();
    int left = 0, sum = 0, maxLen = 0;

    for (int right = 0; right < n; right++) {
        sum += arr[right];

        while (sum > k) {
            sum -= arr[left];
            left++;
        }

        maxLen = max(maxLen, right - left + 1);
    }
    return maxLen;
}

// 4. Variable-size: Minimum window substring (LeetCode 76)
string minWindowSubstring(const string& s, const string& t) {
    if (s.empty() || t.empty()) return "";

    vector<int> freq(128, 0);

```

```

    for (char c : t) freq[c]++;

    int left = 0, right = 0;
    int required = t.size();
    int minLen = INT_MAX, startIdx = 0;

    while (right < (int)s.size()) {
        // Expand window
        if (freq[s[right]] > 0) required--;
        freq[s[right]]--;
        right++;

        // Contract window
        while (required == 0) {
            if (right - left < minLen) {
                minLen = right - left;
                startIdx = left;
            }

            freq[s[left]]++;
            if (freq[s[left]] > 0) required++;
            left++;
        }
    }

    return minLen == INT_MAX ? "" : s.substr(startIdx, minLen);
}

// Example usage
int main() {
    // Fixed size
    vector<int> arr1 = {2, 1, 5, 1, 3, 2};
    cout << "Max sum of size 3: " << maxSumSubarraySizeK(arr1, 3) << "\n"; // 9

    // Longest without repeating
    cout << "Longest w/o repeat: " << longestSubstringWithoutRepeating("abcabcbb")
    << "\n"; // 3

    // Longest subarray sum ≤ k
    vector<int> arr2 = {3, 1, 2, 1, 1, 1, 5, 2};
    cout << "Longest subarray sum ≤ 5: " << longestSubarraySumAtMostK(arr2, 5) <<
    "\n"; // 4

    // Minimum window substring
    cout << "Min window substring: " << minWindowSubstring("ADOBECODEBANC", "ABC")
    << "\n"; // "BANC"

    return 0;
}

```

9. Python Implementation

```
from typing import List
from collections import defaultdict

def max_sum_subarray_size_k(arr: List[int], k: int) -> int:
    n = len(arr)
    if n < k:
        return -1

    window_sum = sum(arr[:k])
    max_sum = window_sum

    for i in range(k, n):
        window_sum += arr[i] - arr[i - k]
        max_sum = max(max_sum, window_sum)
    return max_sum

def longest_substring_without_repeating(s: str) -> int:
    n = len(s)
    last_index = {}
    left = 0
    max_len = 0

    for right, c in enumerate(s):
        if c in last_index and last_index[c] >= left:
            left = last_index[c] + 1
        last_index[c] = right
        max_len = max(max_len, right - left + 1)
    return max_len

def longest_subarray_sum_at_most_k(arr: List[int], k: int) -> int:
    n = len(arr)
    left = 0
    total = 0
    max_len = 0

    for right in range(n):
        total += arr[right]

        while total > k:
            total -= arr[left]
            left += 1

        max_len = max(max_len, right - left + 1)
    return max_len

def min_window_substring(s: str, t: str) -> str:
    if not s or not t:
        return ""

    freq = defaultdict(int)
    for c in t:
        freq[c] += 1
```

```

left = 0
required = len(t)
min_len = float('inf')
start_idx = 0

for right, c in enumerate(s):
    # Expand window
    if freq[c] > 0:
        required -= 1
    freq[c] += 1

    # Contract window
    while required == 0:
        if right - left + 1 < min_len:
            min_len = right - left + 1
            start_idx = left

        freq[s[left]] -= 1
        if freq[s[left]] > 0:
            required += 1
        left += 1

    return "" if min_len == float('inf') else s[start_idx:start_idx + min_len]

# Example usage
if __name__ == "__main__":
    print(max_sum_subarray_size_k([2, 1, 5, 1, 3, 2], 3))          # 9
    print(longest_substring_without_repeating("abcabcbb"))        # 3
    print(longest_subarray_sum_at_most_k([3, 1, 2, 1, 1, 1, 5, 2], 5)) # 4
    print(min_window_substring("ADOBECODEBANC", "ABC"))          # "BANC"

```

10. Code Explanation

Fixed-Size Window (maxSumSubarraySizeK):

- Compute initial window sum for first k elements.
- Slide: `windowSum += arr[i] - arr[i - k]`. Remove the element that left the window, add the new element.
- Track max across all windows.

Variable-Size Window (longestSubstringWithoutRepeating):

- `lastIndex` array stores the most recent index of each character (256 for ASCII).
- When we see a character that's already in the current window (`lastIndex[c] >= left`), we move `left` past its previous occurrence.
- Window `[left, right]` always has no repeating characters.
- `maxLen` tracks the longest valid window.

Variable-Size with Constraint (longestSubarraySumAtMostK):

- Expand **right** unconditionally, adding to sum.
- While **sum > k**, shrink from left by moving **left** forward and subtracting.
- After shrinking, window [**left**, **right**] is valid ($\text{sum} \leq k$).
- Track max length.

Minimum Window Substring:

- **freq** array tracks character deficits. Positive means we still need that character.
- **required** counts how many more characters we need total.
- Expand right until **required == 0** (all characters found).
- Then shrink left while **required == 0**, tracking the minimum window.
- When a character leaves the window, if its frequency becomes positive, **required** increases.

11. Complexity Analysis

Algorithm	Time	Space	Notes
Fixed-size window	$O(n)$	$O(1)$	Single pass
Variable-size (no repeat)	$O(n)$	$O(1)$	256-size array (or $O(k)$ for map)
Variable-size ($\text{sum} \leq k$)	$O(n)$	$O(1)$	Each element added/removed at most once
Min window substring	$O(n)$	$O(1)$	128/256-size array
Longest substring with $\leq k$ distinct	$O(n)$	$O(k)$	Map of k distinct chars

Key insight: Each element is added once and removed at most once, giving $O(n)$ amortized time.

12. Common Patterns

Pattern 1: Fixed-Size Window

Identify: "Subarray of size k ", " k consecutive elements". **Approach:** Slide window, update sum/count incrementally. **Examples:** LeetCode 643 — Maximum Average Subarray I, LeetCode 1343 — Number of Subarrays of Size K .

Pattern 2: Longest Substring with No Repeating Characters

Identify: "Longest substring without repeating characters." **Approach:** Track last index of each character. Move left past duplicate. **Example:** LeetCode 3 — Longest Substring Without Repeating Characters.

Pattern 3: Longest Substring with At Most K Distinct Characters

Identify: "At most k distinct", " k unique characters". **Approach:** Expand right, maintain map of frequencies. Shrink when $\text{distinct} > k$. **Example:** LeetCode 340 — Longest Substring with At Most K Distinct Characters.

Pattern 4: Substring with All Characters (Minimum Window)

Identify: "Contains all characters of string t ", "minimum window". **Approach:** Expand until all characters found, then shrink while still valid. **Example:** LeetCode 76 — Minimum Window Substring.

Pattern 5: Count Subarrays with Condition

Identify: "Count subarrays with $\text{sum} \leq k$ ", "count subarrays with at most k distinct". **Approach:** For each right, count subarrays ending at right that satisfy condition. $\text{Number} = \text{right} - \text{left} + 1$. **Example:** LeetCode 713 — Subarray Product Less Than K, LeetCode 1248 — Count Number of Nice Subarrays.

Pattern 6: String Permutation / Anagram

Identify: "Permutation of string", "anagram substring". **Approach:** Fixed-size window of length $|t|$, compare character frequencies. **Example:** LeetCode 567 — Permutation in String, LeetCode 438 — Find All Anagrams in a String.

13. Common Mistakes

- **Using sliding window with negative numbers for sum constraint:** If sum can decrease by adding elements (negative values), the window condition is not monotonic. Use prefix sum instead.
- **Not updating window state correctly on both add and remove:** For variable window, both adding `arr[right]` and removing `arr[left]` must update the state correctly.
- **Wrong while vs if for shrinking:** If the condition breaks by adding one element, shrinking by one might not be enough. Use `while` not `if`.
- **Off-by-one in window size calculation:** $\text{right} - \text{left} + 1$ for inclusive window.
- **Forgetting to handle empty input.**
- **Not resetting state between test cases.**
- **Using incorrect data structure for tracking:** Array of 26 for lowercase, 256 for ASCII, hash map for unicode.

14. Edge Cases

Case	Expected Behavior
Empty array/string	Return 0, -1, or empty string based on problem
Single element	Window of size 1 works
$k > n$ for fixed window	Return -1 or handle specially
All same characters	Longest without repeating = 1
All distinct characters	Longest without repeating = n
$\text{Sum} \leq k$ with all positive	Works fine
$\text{Sum} \leq k$ with negative numbers	Does NOT work — use prefix sum
String longer than 256 unique chars	Use hash map, not array

15. Variations

Two-Pointer (Grow and Shrink) vs Sliding Window

These terms are often used interchangeably. Strictly:

- **Sliding window:** Usually fixed-size or variable-size window that slides.
- **Two pointers:** More general — can be opposite directions, not just same direction.

Sliding Window with Deque (Monotonic Queue)

Used for sliding window maximum/minimum problems. Maintain a deque of indices with decreasing/increasing values. **Example:** LeetCode 239 — Sliding Window Maximum.

Sliding Window with Frequency Map

Used for substring problems with character constraints. Maintain a map of character frequencies in the current window.

Sliding Window for Count of Subarrays

For each right, count all valid subarrays ending at right. If `[left, right]` is valid, then all `[i, right]` for `i = left..right` are valid. Add `right - left + 1` to count.

16. Related Algorithms/Data Structures

Technique	Connection
Two Pointers	Same-direction two pointers is essentially sliding window.
Prefix Sum	Alternative for subarray sum problems (handles negatives).
Deque (Monotonic Queue)	Used for sliding window max/min.
Hash Map	Used to track window state for character problems.
Segment Tree	Alternative when window operations are complex.

Decision guide:

- Subarray sum, all positive → Sliding window.
- Subarray sum, has negatives → Prefix sum.
- Sliding window max/min → Deque.
- Substring with char constraints → Sliding window + freq map.
- Need to answer many queries on different windows → Segment tree / BIT.

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Maximum Average Subarray I	LeetCode 643	Fixed-size window	Easy
Contains Duplicate II	LeetCode 219	Fixed-size window with set	Easy

Medium

Problem	Platform	Pattern	Difficulty
Longest Substring Without Repeating Characters	LeetCode 3	Variable-size window	Medium
Permutation in String	LeetCode 567	Fixed-size window + freq	Medium
Subarray Product Less Than K	LeetCode 713	Variable-size window, count	Medium

Hard

Problem	Platform	Pattern	Difficulty
Minimum Window Substring	LeetCode 76	Variable-size, all chars	Hard
Sliding Window Maximum	LeetCode 239	Deque-based sliding window	Hard
Longest Substring with At Most K Distinct Characters	LeetCode 340	Variable-size, distinct count	Medium/Hard

18. Interview Explanation

"Sliding window is a technique where we maintain a contiguous subarray and update it incrementally as we traverse. For fixed-size windows, we slide by removing the left element and adding the right one, updating the result in $O(1)$ per slide. For variable-size windows, we expand the right pointer and shrink the left pointer when a condition is violated. The key requirement is that the condition must be monotonic — expanding the window makes it harder to satisfy. The total time is $O(n)$ because each element is added and removed at most once. Common use cases are maximum sum subarray of size k , longest substring without repeating characters, and minimum window substring."

19. Revision Notes

- **Fixed-size:** `sum += arr[i] - arr[i - k]`
- **Variable-size:** `right` expands, `left` shrinks when invalid
- **Condition must be monotonic** (expanding = harder to satisfy)
- **State management:** sum variable, freq map, distinct count, etc.
- **Complexity:** $O(n)$ time, $O(1)$ or $O(k)$ space
- **Count subarrays:** For each valid window, add `right - left + 1` to count
- **Common trap:** Negatives in sum problems → use prefix sum

20. Final Cheat Sheet

SLIDING WINDOW – CHEAT SHEET
WHEN TO USE: Contiguous subarray/substring with constraint, fixed/variable

FIXED-SIZE:	sum += arr[i] - arr[i-k]
VARIABLE:	expand right, while(!valid) shrink
MONOTONIC:	Condition must be monotonic
KEY CODE:	for(r=0;r<n;r++) { add; while→shrink }
COMPLEXITY:	O(n) time, O(1)/O(k) space
COMMON USE:	Max sum, no repeat, min window, count subarrays with condition
TRAP:	Negatives break monotonicity
RELATED:	Two pointers, prefix sum, deque

5. BINARY SEARCH

1. Overview

Binary search is a divide-and-conquer algorithm that finds a target value in a **sorted** array by repeatedly dividing the search space in half. It reduces a linear $O(n)$ search to $O(\log n)$.

2. Intuition

Core idea: If the array is sorted, you can eliminate half the elements with each comparison. Check the middle element. If it's the target, done. If the target is smaller, search the left half. If larger, search the right half.

Analogy: Finding a name in a phone book. You don't start from page 1. You open the book in the middle, check if the name is before or after, and tear the problem in half. Each step eliminates half the remaining pages.

Step-by-step reasoning:

1. Look at the middle element of the sorted array.
2. If it's what you want, return it.
3. If it's greater than what you want, the target must be in the left half (since array is sorted).
4. If it's less than what you want, the target must be in the right half.
5. Repeat on the chosen half until the target is found or the search space is empty.

Why it works: Each comparison eliminates half the remaining elements. After k steps, the search space is reduced from n to $n/2^k$. When $n/2^k = 1$, $k = \log_2(n)$. So $O(\log n)$ steps.

3. When to Use It

- **Sorted array** and need to find an element.
- **Find first/last occurrence** of a value (lower_bound, upper_bound).
- **Find insertion point** for a value.
- **Search in rotated sorted array.**
- **Search in 2D matrix** (sorted row-wise and column-wise).
- **Finding a peak element** in an array.
- **As a building block** for binary search on answer.

Common trigger phrases:

- "sorted array"
- "search"
- "find element"
- "first occurrence"
- "last occurrence"
- "lower bound"
- "upper bound"
- "logarithmic"

4. When Not to Use It

- **Unsorted array:** Binary search requires sorted data. Sort first if needed ($O(n \log n)$).
- **Single element or small n:** Linear search is simpler and fast enough for small n.
- **Only one query:** If n is small, linear search is fine.
- **Linked list:** Random access is $O(n)$, so binary search is $O(n \log n)$ — not worth it.
- **Dynamic data with frequent insertions/deletions:** Maintaining sorted order is expensive. Use BST or hash set.
- **Non-comparable elements:** Binary search requires a comparison operator ($<$, $>$, $==$).

5. Core Concepts

5.1 Search Space

The range of indices where the target could be. Initially $[0, n-1]$. Halved at each step.

5.2 Midpoint Calculation

```
int mid = left + (right - left) / 2; // Avoids overflow
```

Never use $(left + right) / 2$ — it can overflow for large `left` and `right`.

5.3 Three Standard Templates

Template 1: Basic search (find exact value)

```
while (left <= right) {
    mid = left + (right - left) / 2;
    if (arr[mid] == target) return mid;
    if (arr[mid] < target) left = mid + 1;
    else right = mid - 1;
}
```

Template 2: Lower bound (first element $\geq$ target)

```
while (left < right) {
    mid = left + (right - left) / 2;
    if (arr[mid] >= target) right = mid;
    else left = mid + 1;
}
return left;
```

Template 3: Upper bound (first element > target)

```
while (left < right) {
    mid = left + (right - left) / 2;
    if (arr[mid] > target) right = mid;
    else left = mid + 1;
}
return left;
```

5.4 Lower Bound vs Upper Bound

- **lower_bound:** First index where `arr[i] >= target`.
- **upper_bound:** First index where `arr[i] > target`.
- Count of target = `upper_bound - lower_bound`.

5.5 Invariant Maintenance

The key to writing correct binary search is maintaining a clear invariant:

- `arr[left-1] < target` (everything left of left is definitely < target)
- `arr[right+1] > target` (everything right of right is definitely > target)

When the loop ends, `left` is the insertion point.

6. Step-by-Step Algorithm

Standard Binary Search (find exact value):

1. Initialize `left = 0, right = n - 1`.
2. While `left <= right`:
 - `mid = left + (right - left) / 2`.
 - If `arr[mid] == target`: return `mid`.
 - If `arr[mid] < target`: `left = mid + 1` (search right half).
 - Else: `right = mid - 1` (search left half).
3. Return -1 (not found).

Lower Bound (first element $\geq$ target):

1. Initialize `left = 0, right = n` (note: right is n, not n-1).
2. While `left < right`:
 - `mid = left + (right - left) / 2`.

- If `arr[mid] >= target`: `right = mid` (could be the answer).
 - Else: `left = mid + 1` (must be to the right).
3. Return `left`.

7. Dry Run

Problem: Find 7 in sorted array `[1, 3, 5, 7, 9, 11, 13]`

Step	left	right	mid	arr[mid]	arr[mid] vs target	Action
1	0	6	3	7	==	Found!

Result: Index 3

Problem: Find 4 in `[1, 3, 5, 7, 9, 11, 13]` (not present)

Step	left	right	mid	arr[mid]	Comparison	Action
1	0	6	3	7	$7 > 4$	<code>right = 2</code>
2	0	2	1	3	$3 < 4$	<code>left = 2</code>
3	2	2	2	5	$5 > 4$	<code>right = 1</code>
4	2	1	—	—	<code>left > right</code>	Stop

Result: -1 (not found)

Problem: Lower bound of 4 in `[1, 3, 5, 7, 9]`

Step	left	right	mid	arr[mid]	Condition	Action
1	0	5	2	5	$5 \geq 4$	<code>right = 2</code>
2	0	2	1	3	$3 < 4$	<code>left = 2</code>
3	2	2	—	—	<code>left == right</code>	Stop

Result: 2 (`arr[2] = 5` is first ≥ 4)

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Standard binary search (find exact value)
int binarySearch(const vector<int>& arr, int target) {
    int left = 0, right = arr.size() - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;
```

```

        if (arr[mid] == target) return mid;
        if (arr[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}

// Lower bound: first index where arr[i] >= target
int lowerBound(const vector<int>& arr, int target) {
    int left = 0, right = arr.size(); // right is size, not size-1

    while (left < right) {
        int mid = left + (right - left) / 2;
        if (arr[mid] >= target) right = mid;
        else left = mid + 1;
    }
    return left;
}

// Upper bound: first index where arr[i] > target
int upperBound(const vector<int>& arr, int target) {
    int left = 0, right = arr.size();

    while (left < right) {
        int mid = left + (right - left) / 2;
        if (arr[mid] > target) right = mid;
        else left = mid + 1;
    }
    return left;
}

// Count occurrences of target
int countOccurrences(const vector<int>& arr, int target) {
    int lb = lowerBound(arr, target);
    int ub = upperBound(arr, target);
    return ub - lb;
}

// Binary search in rotated sorted array (no duplicates)
int searchRotated(const vector<int>& arr, int target) {
    int left = 0, right = arr.size() - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (arr[mid] == target) return mid;

        // Left half is sorted
        if (arr[left] <= arr[mid]) {
            if (arr[left] <= target && target < arr[mid]) right = mid - 1;
            else left = mid + 1;
        }
        // Right half is sorted

```

```

        else {
            if (arr[mid] < target && target <= arr[right]) left = mid + 1;
            else right = mid - 1;
        }
    }
    return -1;
}

// Example usage
int main() {
    vector<int> arr = {1, 3, 5, 7, 9, 11, 13};

    cout << "Search 7: " << binarySearch(arr, 7) << "\n";          // 3
    cout << "Search 4: " << binarySearch(arr, 4) << "\n";          // -1

    cout << "Lower bound of 4: " << lowerBound(arr, 4) << "\n";    // 2
    cout << "Upper bound of 7: " << upperBound(arr, 7) << "\n";    // 4
    cout << "Count of 7: " << countOccurrences(arr, 7) << "\n";    // 1

    // Rotated array
    vector<int> rotated = {4, 5, 6, 7, 0, 1, 2};
    cout << "Search in rotated (0): " << searchRotated(rotated, 0) << "\n"; // 4
    cout << "Search in rotated (3): " << searchRotated(rotated, 3) << "\n"; // -1

    return 0;
}

```

9. Python Implementation

```

from typing import List

def binary_search(arr: List[int], target: int) -> int:
    left, right = 0, len(arr) - 1

    while left <= right:
        mid = left + (right - left) // 2

        if arr[mid] == target:
            return mid
        if arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1

def lower_bound(arr: List[int], target: int) -> int:
    left, right = 0, len(arr)

    while left < right:
        mid = left + (right - left) // 2
        if arr[mid] >= target:

```

```

        right = mid
    else:
        left = mid + 1
    return left

def upper_bound(arr: List[int], target: int) -> int:
    left, right = 0, len(arr)

    while left < right:
        mid = left + (right - left) // 2
        if arr[mid] > target:
            right = mid
        else:
            left = mid + 1
    return left

def count_occurrences(arr: List[int], target: int) -> int:
    return upper_bound(arr, target) - lower_bound(arr, target)

def search_rotated(arr: List[int], target: int) -> int:
    left, right = 0, len(arr) - 1

    while left <= right:
        mid = left + (right - left) // 2

        if arr[mid] == target:
            return mid

        # Left half is sorted
        if arr[left] <= arr[mid]:
            if arr[left] <= target < arr[mid]:
                right = mid - 1
            else:
                left = mid + 1
        # Right half is sorted
        else:
            if arr[mid] < target <= arr[right]:
                left = mid + 1
            else:
                right = mid - 1
    return -1

# Example usage
if __name__ == "__main__":
    arr = [1, 3, 5, 7, 9, 11, 13]
    print(f"Search 7: {binary_search(arr, 7)}")          # 3
    print(f"Search 4: {binary_search(arr, 4)}")          # -1
    print(f"Lower bound of 4: {lower_bound(arr, 4)}")    # 2
    print(f"Upper bound of 7: {upper_bound(arr, 7)}")    # 4
    print(f"Count of 7: {count_occurrences(arr, 7)}")    # 1

    rotated = [4, 5, 6, 7, 0, 1, 2]

```

```
print(f"Rotated search 0: {search_rotated(rotated, 0)}") # 4
print(f"Rotated search 3: {search_rotated(rotated, 3)}") # -1
```

10. Code Explanation

Standard binarySearch:

- `while (left <= right)`: Loop continues while search space is non-empty.
- `mid = left + (right - left) / 2`: Safe mid calculation, avoids overflow.
- Three-way comparison: equal, less than, greater than.
- On mismatch, we narrow to `[left, mid-1]` or `[mid+1, right]`.
- Returns -1 if not found.

Lower bound:

- `right = arr.size()` (not `size()-1`): This handles the case where target is greater than all elements. The answer would be `n` (insert at end).
- `while (left < right)`: Loop until `left == right`.
- `arr[mid] >= target`: If true, mid is a candidate. Narrow right to mid (include mid).
- Else: mid is too small, narrow left to `mid+1`.
- Returns `left` (insertion point).

Upper bound:

- Same as lower bound but condition is `arr[mid] > target`.
- Returns first index where value > target.

Rotated array search:

- Determine which half is sorted by comparing `arr[left]` with `arr[mid]`.
- If left half is sorted (`arr[left] <= arr[mid]`):
 - Check if target is in `[arr[left], arr[mid]]`. If yes, search left. Else search right.
- If right half is sorted:
 - Check if target is in `(arr[mid], arr[right]]`. If yes, search right. Else search left.

11. Complexity Analysis

Operation	Time	Space
Standard binary search	$O(\log n)$	$O(1)$
Lower bound	$O(\log n)$	$O(1)$
Upper bound	$O(\log n)$	$O(1)$
Count occurrences	$O(\log n)$	$O(1)$
Rotated array search	$O(\log n)$	$O(1)$

Best case: $O(1)$ — target is at the middle element on first check. **Worst case:** $O(\log n)$ — target is at a leaf of the search tree. **Space:** $O(1)$ iterative implementation.

12. Common Patterns

Pattern 1: Exact Value Search

Identify: Find if element exists in sorted array. **Approach:** Standard binary search with `left <= right`.

Example: LeetCode 704 — Binary Search.

Pattern 2: First/Last Occurrence

Identify: "First occurrence", "last occurrence", "frequency count". **Approach:** Lower bound for first, upper bound - 1 for last. **Example:** LeetCode 34 — Find First and Last Position of Element in Sorted Array.

Pattern 3: Search in Rotated Sorted Array

Identify: "Rotated sorted array", "circularly shifted". **Approach:** Determine which half is sorted, check if target is in that half. **Example:** LeetCode 33 — Search in Rotated Sorted Array.

Pattern 4: Search in 2D Matrix

Identify: "Matrix with sorted rows and columns". **Approach:** Treat as 1D array: `midVal = matrix[mid / cols][mid % cols]`. **Example:** LeetCode 74 — Search a 2D Matrix.

Pattern 5: Find Peak Element

Identify: "Peak element", "local maximum". **Approach:** Compare mid with neighbors. Move toward the larger neighbor. **Example:** LeetCode 162 — Find Peak Element.

Pattern 6: Find Minimum in Rotated Sorted Array

Identify: "Minimum in rotated array". **Approach:** Compare mid with right. If `mid > right`, min is in right half. Else in left half. **Example:** LeetCode 153 — Find Minimum in Rotated Sorted Array.

13. Common Mistakes

- **Off-by-one in while condition:** `left <= right` vs `left < right`. Use `<=` for exact search, `<` for lower/upper bound.
- **Mid calculation overflow:** `(left + right) / 2` can overflow. Use `left + (right - left) / 2`.
- **Infinite loop:** In lower bound template, `right = mid` (not `mid - 1`). If you use `mid - 1`, you might miss the answer.
- **Forgetting to handle duplicates:** Lower/upper bound handles duplicates correctly.
- **Wrong initial right value:** For lower bound, `right = n` (not `n-1`) to handle the case where target > all elements.
- **Not checking bounds:** If `arr` is empty, `arr.size() - 1` wraps to a huge number.
- **Assuming sorted order:** Binary search requires sorted input. Always verify or sort first.

14. Edge Cases

Case	Input	Expected Behavior
Empty array	<code>[]</code>	Return -1; lower bound returns 0

Case	Input	Expected Behavior
Single element	[5], target=5	Return 0
Single element	[5], target=3	Return -1; lower bound returns 0
Target not present	[1,3,5], target=4	Return -1; lower bound returns 2
Target at ends	[1,3,5], target=1	Return 0
All same values	[5,5,5,5], target=5	lower bound = 0, upper bound = 4, count = 4
Target greater than all	[1,3,5], target=7	lower bound returns 3 (n)
Rotated, no rotation	[1,2,3,4,5]	Works as standard binary search
Rotated, n=1	[5]	Works fine

15. Variations

Binary Search on Real Numbers

Used when searching for a value in a continuous range (e.g., square root).

```
double binarySearchReal(double left, double right, double target) {
    for (int i = 0; i < 100; i++) { // fixed iterations for precision
        double mid = (left + right) / 2;
        if (f(mid) < target) left = mid;
        else right = mid;
    }
    return left;
}
```

Ternary Search

Used for finding maximum/minimum of a unimodal function. Divides into three parts instead of two. Less commonly used; binary search is often sufficient.

Binary Search on Answer

See next section. The most important variation for CP.

Exponential Search

Used when the array size is unknown or infinite. Start with size 1, double until target is within range, then binary search.

16. Related Algorithms/Data Structures

Algorithm	Connection
Binary Search on Answer	Extension of binary search to non-array contexts.

Algorithm	Connection
Ternary Search	For unimodal functions (max/min).
Exponential Search	For unbounded/infinite arrays.
Interpolation Search	Uses value distribution for better average case (but $O(n)$ worst case).
STL Functions	<code>lower_bound</code> , <code>upper_bound</code> , <code>binary_search</code> in <code><algorithm></code> .

C++ STL equivalents:

- `binary_search(arr.begin(), arr.end(), target)` → returns bool.
- `lower_bound(arr.begin(), arr.end(), target)` → iterator.
- `upper_bound(arr.begin(), arr.end(), target)` → iterator.

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Binary Search	LeetCode 704	Standard binary search	Easy
First Bad Version	LeetCode 278	Lower bound pattern	Easy

Medium

Problem	Platform	Pattern	Difficulty
Find First and Last Position of Element in Sorted Array	LeetCode 34	Lower/upper bound	Medium
Search in Rotated Sorted Array	LeetCode 33	Rotated array	Medium
Find Peak Element	LeetCode 162	Peak finding	Medium

Hard

Problem	Platform	Pattern	Difficulty
Search in Rotated Sorted Array II	LeetCode 81	Rotated with duplicates	Hard
Median of Two Sorted Arrays	LeetCode 4	Partition-based binary search	Hard
Find K-th Smallest Pair Distance	LeetCode 719	Binary search on answer	Hard

18. Interview Explanation

"Binary search finds a target in a sorted array in $O(\log n)$ time by repeatedly dividing the search space in half. I maintain two pointers — left and right — and a $mid = left + (right-left)/2$ to avoid overflow. At each step, I compare the middle element with the target and eliminate the half that cannot contain the target. I use the lower bound template when I need the first occurrence of a value or the insertion

point. The key to writing correct binary search is maintaining a clear invariant about what the left and right pointers represent."

19. Revision Notes

- **Key idea:** Divide search space in half each step $\rightarrow O(\log n)$
- **Mid formula:** `mid = left + (right - left) / 2` (avoid overflow)
- **Exact search:** `while (left <= right), mid == target  $\rightarrow$  return`
- **Lower bound:** `while (left < right), arr[mid] >= target  $\rightarrow$  right = mid`
- **Upper bound:** `while (left < right), arr[mid] > target  $\rightarrow$  right = mid`
- **Count = upper_bound - lower_bound**
- **Rotated array:** Check which half is sorted, then search accordingly
- **Common trap:** Off-by-one on `right = mid` vs `right = mid - 1`, overflow in mid

20. Final Cheat Sheet

BINARY SEARCH – CHEAT SHEET	
WHEN TO USE:	Sorted array, find element/position
COMPLEXITY:	$O(\log n)$ time, $O(1)$ space
KEY CODE:	<code>while(l<=r) { mid=l+(r-l)/2; ... }</code>
LOWER BOUND:	<code>while(l<r) { if(v>=t) r=m; else l=m+1 }</code>
UPPER BOUND:	<code>while(l<r) { if(v>t) r=m; else l=m+1 }</code>
COUNT:	<code>ub - lb</code>
ROTATED:	<code>if arr[l]<=arr[m] (left sorted)</code>
EDGE CASES:	empty, single, not found, all same
TRAP:	overflow in mid, wrong right init
STL:	<code>lower_bound, upper_bound, binary_search</code>

6. BINARY SEARCH ON ANSWER

1. Overview

Binary search on answer (also called parametric search) is a technique where we don't search for an element in an array, but instead search for the **answer value** itself within a range of possible answers. We use a decision function `isFeasible(mid)` that checks if a candidate answer `mid` is valid. This converts optimization problems ("find the maximum/minimum such that...") into decision problems ("is x feasible?").

2. Intuition

Core idea: If the answer space is monotonic (if x is feasible, then all values $\geq x$ or $\leq x$ are also feasible), we can binary search on the answer value instead of computing it directly.

Analogy: You're guessing a friend's age. You know it's between 1 and 100. You ask "Are you at least 50?" If yes, you know the answer is in [50, 100]. If no, it's in [1, 49]. Each question halves the range. The "decision function" is the friend's honest answer.

Step-by-step reasoning:

1. Define the range of possible answers [low, high].
2. Define a function `isFeasible(mid)` that returns true if `mid` is a valid answer.
3. The function must be **monotonic**: either `TTTT...TTFFFF...FF` or `FFFF...FFT...TT`.
4. Binary search on the answer range to find the boundary between feasible and infeasible.

Why it works: Many optimization problems have a monotonic property. If a certain value works, larger values also work (or smaller values also work). This allows binary search to find the optimal value in $O(\log R)$ decision checks, where R is the answer range.

3. When to Use It

- **Minimization problems:** "Find the minimum X such that condition holds."
- **Maximization problems:** "Find the maximum X such that condition holds."
- Problems where the answer is a **numeric value** (length, time, speed, distance, count).
- Problems where you can check if a candidate answer is feasible in polynomial time.
- The decision function is easier to write than computing the answer directly.

Common trigger phrases:

- "maximize the minimum"
- "minimize the maximum"
- "largest minimum"
- "smallest maximum"
- "allocate"
- "divide into k parts"
- "minimum time to complete"
- "maximum capacity"
- "koko eating bananas"
- "split array"

4. When Not to Use It

- **Answer space is not monotonic:** If feasibility doesn't follow a pattern, binary search fails.
- **Decision function is hard to write:** If checking feasibility is as hard as solving the problem, binary search on answer doesn't help.
- **Answer space is small:** If the answer is in a small range, linear search is simpler.
- **Answer is not numeric:** Binary search requires ordered answer space.
- **Decision function is not deterministic:** Must return the same result for the same input.

5. Core Concepts

5.1 Answer Space (Search Range)

The range of possible values the answer could take. Must be bounded.

Examples:

- Speed: `[1, max(arr)]`
- Time: `[0, 1e18]` or `[0, INT_MAX]`
- Capacity: `[max(arr), sum(arr)]`

5.2 Decision Function (Feasibility Check)

A function `bool isFeasible(mid)` that returns true if `mid` is a valid answer.

Must be:

- Monotonic (if `mid` works, all larger/smaller also work).
- Efficient to compute (ideally $O(n)$ or $O(n \log n)$).
- Deterministic.

5.3 Monotonicity Pattern

Two common patterns:

Pattern 1: FFFF...TTTT (Find first true):

- `isFeasible(x)` is false for small `x`, true for large `x`.
- We want the minimum `x` that is feasible.
- Example: Minimum capacity to ship packages within `D` days.

Pattern 2: TTTT...FFFF (Find last true):

- `isFeasible(x)` is true for small `x`, false for large `x`.
- We want the maximum `x` that is feasible.
- Example: Maximum number of flowers you can plant.

5.4 Lower Bound vs Upper Bound on Answer

- **Find minimum feasible:** Lower bound on answer (first true in FFFTTT).
- **Find maximum feasible:** Upper bound on answer (last true in TTFFFF).

6. Step-by-Step Algorithm**General Binary Search on Answer (Minimum Feasible):**

1. Define answer range: `low = min_possible, high = max_possible`.
2. While `low < high`:
 - `mid = low + (high - low) / 2`.
 - If `isFeasible(mid)`: `high = mid` (mid is feasible, try smaller).
 - Else: `low = mid + 1` (mid is not feasible, need larger).
3. Return `low` (minimum feasible answer).

General Binary Search on Answer (Maximum Feasible):

1. Define answer range: `low = min_possible, high = max_possible`.
2. While `low < high`:

- $mid = low + (high - low + 1) / 2$ (upper mid to avoid infinite loop).
- If `isFeasible(mid)`: $low = mid$ (mid is feasible, try larger).
- Else: $high = mid - 1$ (mid is not feasible, need smaller).

3. Return `low` (maximum feasible answer).

7. Dry Run

Problem: Koko Eating Bananas — Find minimum eating speed K such that Koko can eat all bananas in H hours.

Input: `piles = [3, 6, 7, 11]`, $H = 8$

Decision function: `isFeasible(K)` = hours needed at speed $K \leq H$

Range: $low = 1$, $high = 11$ (max pile)

Step	low	high	mid	isFeasible(mid)	Hours at mid	Action
1	1	11	6	True	$1+1+2+2=6 \leq 8$	high = 6
2	1	6	3	False	$1+2+3+4=10 > 8$	low = 4
3	4	6	5	True	$1+2+2+3=8 \leq 8$	high = 5
4	4	5	4	True	$1+2+2+3=8 \leq 8$	high = 4
5	4	4	—	—	—	low == high, stop

Result: Minimum speed = 4

Problem: Split Array Largest Sum — Minimum possible largest sum when splitting into m subarrays.

Input: `nums = [7, 2, 5, 10, 8]`, $m = 2$

Range: $low = 10$ (max element), $high = 32$ (total sum)

Step	low	high	mid	isFeasible(mid)	Subarrays needed	Action
1	10	32	21	True	$2 \leq 2$	high = 21
2	10	21	15	False	$3 > 2$	low = 16
3	16	21	18	True	$2 \leq 2$	high = 18
4	16	18	17	True	$2 \leq 2$	high = 17
5	16	17	16	False	$3 > 2$	low = 17
6	17	17	—	—	—	Stop

Result: Minimum largest sum = 17

8. C++ Implementation

```

#include <bits/stdc++.h>
using namespace std;

// =====
// Problem 1: Koko Eating Bananas
// Find minimum eating speed to finish in H hours
// =====
class KokoBananas {
private:
    vector<int> piles;
    int H;

    // Decision function: can Koko eat all bananas at speed k in H hours?
    bool canEatAll(int speed) {
        int hours = 0;
        for (int pile : piles) {
            hours += (pile + speed - 1) / speed; // ceil(pile / speed)
            if (hours > H) return false; // early exit
        }
        return true;
    }

public:
    int minEatingSpeed(vector<int>& piles_, int H_) {
        piles = piles_;
        H = H_;

        int low = 1;
        int high = *max_element(piles.begin(), piles.end());

        while (low < high) {
            int mid = low + (high - low) / 2;
            if (canEatAll(mid)) {
                high = mid; // mid is feasible, try smaller
            } else {
                low = mid + 1; // mid is too slow
            }
        }
        return low;
    }
};

// =====
// Problem 2: Split Array Largest Sum
// Minimize the largest sum when splitting into m subarrays
// =====
class SplitArray {
private:
    vector<int> nums;
    int m;

    // Can we split such that each subarray sum ≤ maxSum?

```

```

    bool canSplit(int maxSum) {
        int count = 1; // start with first subarray
        int currentSum = 0;

        for (int num : nums) {
            if (currentSum + num > maxSum) {
                count++; // start new subarray
                currentSum = num;
                if (count > m) return false;
            } else {
                currentSum += num;
            }
        }
        return true;
    }

public:
    int splitArray(vector<int>& nums_, int m_) {
        nums = nums_;
        m = m_;

        int low = *max_element(nums.begin(), nums.end());
        int high = accumulate(nums.begin(), nums.end(), 0);

        while (low < high) {
            int mid = low + (high - low) / 2;
            if (canSplit(mid)) {
                high = mid; // try smaller max sum
            } else {
                low = mid + 1; // need larger max sum
            }
        }
        return low;
    }
};

// =====
// Problem 3: Aggressive Cows (Maximize Minimum Distance)
// Place k cows in stalls, maximize minimum distance between them
// =====
class AggressiveCows {
private:
    vector<int> stalls;
    int k;

    bool canPlace(int minDist) {
        int count = 1; // place first cow at first stall
        int lastPos = stalls[0];

        for (int i = 1; i < (int)stalls.size(); i++) {
            if (stalls[i] - lastPos >= minDist) {
                count++;
                lastPos = stalls[i];
                if (count >= k) return true;
            }
        }
    }
};

```

```

    }
    }
    return false;
}

public:
    int maxMinDistance(vector<int>& stalls_, int k_) {
        stalls = stalls_;
        k = k_;
        sort(stalls.begin(), stalls.end());

        int low = 1;
        int high = stalls.back() - stalls.front();

        // Maximize min distance → TTTTFFFF pattern
        while (low < high) {
            int mid = low + (high - low + 1) / 2; // upper mid
            if (canPlace(mid)) {
                low = mid; // try larger distance
            } else {
                high = mid - 1; // too large, need smaller
            }
        }
        return low;
    }
};

// Example usage
int main() {
    // Koko Bananas
    KokoBananas kb;
    vector<int> piles = {3, 6, 7, 11};
    cout << "Koko min speed: " << kb.minEatingSpeed(piles, 8) << "\n"; // 4

    // Split Array
    SplitArray sa;
    vector<int> nums = {7, 2, 5, 10, 8};
    cout << "Min largest sum: " << sa.splitArray(nums, 2) << "\n"; // 18

    // Aggressive Cows
    AggressiveCows ac;
    vector<int> stalls = {1, 2, 8, 4, 9};
    cout << "Max min distance: " << ac.maxMinDistance(stalls, 3) << "\n"; // 3

    return 0;
}

```

9. Python Implementation

```

from typing import List

```

```

class KokoBananas:
    def min_eating_speed(self, piles: List[int], h: int) -> int:
        def can_eat_all(speed: int) -> bool:
            hours = 0
            for pile in piles:
                hours += (pile + speed - 1) // speed # ceil division
            if hours > h:
                return False
            return True

        low, high = 1, max(piles)
        while low < high:
            mid = (low + high) // 2
            if can_eat_all(mid):
                high = mid
            else:
                low = mid + 1
        return low

```

```

class SplitArray:
    def split_array(self, nums: List[int], m: int) -> int:
        def can_split(max_sum: int) -> bool:
            count = 1
            current = 0
            for num in nums:
                if current + num > max_sum:
                    count += 1
                    current = num
                    if count > m:
                        return False
                else:
                    current += num
            return True

        low, high = max(nums), sum(nums)
        while low < high:
            mid = (low + high) // 2
            if can_split(mid):
                high = mid
            else:
                low = mid + 1
        return low

```

```

class AggressiveCows:
    def max_min_distance(self, stalls: List[int], k: int) -> int:
        def can_place(min_dist: int) -> bool:
            count = 1
            last = stalls[0]
            for i in range(1, len(stalls)):
                if stalls[i] - last >= min_dist:
                    count += 1
                    last = stalls[i]

```

```

        if count >= k:
            return True
        return False

    stalls.sort()
    low, high = 1, stalls[-1] - stalls[0]

    while low < high:
        mid = (low + high + 1) // 2 # upper mid
        if can_place(mid):
            low = mid
        else:
            high = mid - 1
    return low

# Example usage
if __name__ == "__main__":
    print(KokoBananas().min_eating_speed([3, 6, 7, 11], 8)) # 4
    print(SplitArray().split_array([7, 2, 5, 10, 8], 2)) # 18
    print(AggressiveCows().max_min_distance([1, 2, 8, 4, 9], 3)) # 3

```

10. Code Explanation

Koko Bananas (Minimize speed, FFFTTT pattern):

- `canEatAll(speed)`: Computes total hours needed at speed `speed`. Uses $(pile + speed - 1) / speed$ for ceiling division. Returns true if total $\leq H$.
- Binary search: `low` = 1 (minimum possible speed), `high` = `max(piles)` (`speed` = `max pile` means each pile takes 1 hour).
- Pattern: FFFTTT (too slow $\rightarrow$ not feasible, fast enough $\rightarrow$ feasible). Find first true.

Split Array (Minimize max sum, FFFTTT pattern):

- `canSplit(maxSum)`: Greedy — iterate through `nums`, accumulate current sum. If adding next element exceeds `maxSum`, start a new subarray. Count subarrays needed. Return true if count $\leq m$.
- Binary search: `low` = `max element` (each subarray must contain at least one element), `high` = `total sum` (one subarray).
- Pattern: FFFTTT (max sum too small $\rightarrow$ infeasible, large enough $\rightarrow$ feasible). Find first true.

Aggressive Cows (Maximize min distance, TTFFFF pattern):

- `canPlace(minDist)`: Greedy — place first cow at first stall. For each subsequent stall, if distance from last placed cow $\geq minDist$, place a cow. Return true if we placed all `k` cows.
- Binary search: `low` = 1, `high` = `max - min stall position`.
- Pattern: TTFFFF (small distances are always feasible, large distances may not be). Find last true.
- **Key difference:** Uses `mid = low + (high - low + 1) / 2` (upper mid) to avoid infinite loop when `low` and `high` are close.

11. Complexity Analysis

Problem	Decision Function	Binary Search	Total
Koko Eating Bananas	$O(n)$	$O(\log \text{maxPile})$	$O(n \log \text{maxPile})$
Split Array	$O(n)$	$O(\log \text{sum})$	$O(n \log \text{sum})$
Aggressive Cows	$O(n)$	$O(\log \text{range})$	$O(n \log \text{range})$

General formula:

- Time: $O(f(n) \times \log R)$ where $f(n)$ is complexity of decision function, R is answer range size.
- Space: $O(1)$ extra (excluding input).

12. Common Patterns

Pattern 1: Minimum Feasible (First True in FFFTTT)

Approach: `while (low < high) { mid = (low+high)/2; if (feasible(mid)) high = mid; else low = mid+1; }` **Examples:** Koko Bananas, Split Array, Capacity to Ship Packages.

Pattern 2: Maximum Feasible (Last True in TTFFFF)

Approach: `while (low < high) { mid = (low+high+1)/2; if (feasible(mid)) low = mid; else high = mid-1; }` **Examples:** Aggressive Cows, Maximum Number of Groups.

Pattern 3: Search in Unsorted Array with Property

Key insight: Even though the array is not sorted, the answer space (integer/real numbers) is ordered.

Examples: Find the sqrt of a number, find the kth smallest element.

Pattern 4: Minimize the Maximum / Maximize the Minimum

How to identify: Classic min-max or max-min problems. **Approach:** Almost always binary search on answer.

Examples: Split Array (minimize max sum), Aggressive Cows (maximize min distance).

13. Common Mistakes

- **Wrong mid calculation for maximization:** Use `mid = (low + high + 1) / 2` (upper mid) for maximization to avoid infinite loop.
- **Not defining answer range correctly:** Low bound too low (wastes iterations) or too high (misses answer). High bound too low (misses answer).
- **Decision function not monotonic:** If the function is not monotonic, binary search gives wrong answer.
- **Off-by-one in feasibility check:** `hours > H` vs `hours >= H`. Be precise about the condition.
- **Integer overflow in mid calculation:** `(low + high) / 2` can overflow. Use `low + (high - low) / 2`.
- **Not handling edge cases:** Empty input, $k = 0$, $m = 0$, etc.
- **Inefficient decision function:** If decision function is $O(n^2)$, the total becomes $O(n^2 \log R)$, which might be too slow.

14. Edge Cases

Case	Expected Behavior
Empty input	Return 0 or handle specially
m = 1 (split into 1)	Sum of all elements
m = n (split individually)	Max element
k = 1 (one cow)	No constraint, return 0
k = 2 (two cows)	Max distance = max - min
All same values	Any split works, answer is the value itself
Very large answer range	Use long long, ensure binary search terminates
Decision function always true	Minimum feasible = low, maximum feasible = high
Decision function always false	Opposite bound

15. Variations

Binary Search on Real Numbers (Floating Point)

Used when answer is a real number (e.g., square root, minimum average).

```
double binarySearchReal(double low, double high, double target) {
    for (int i = 0; i < 100; i++) { // fixed iterations for precision
        double mid = (low + high) / 2;
        if (f(mid) < target) low = mid;
        else high = mid;
    }
    return low;
}
```

Binary Search on Integer Range with Custom Comparator

Used for problems like "kth smallest element in a matrix" where the array is not sorted but the answer space is.

Parallel Binary Search

Used when you have multiple queries and need to binary search on each. Instead of binary searching each query independently, you process all queries simultaneously.

16. Related Algorithms/Data Structures

Algorithm	Connection
Binary Search	Foundation. Same logic, different search space.
Greedy Algorithms	Decision function is often greedy.

Algorithm	Connection
Dynamic Programming	Sometimes decision function uses DP.
Parametric Search	Another name for binary search on answer.

Distinction: Regular binary search searches for an element in an array. Binary search on answer searches for the best value in the answer space.

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Square Root (Integer)	LeetCode 69	Binary search on real	Easy
Koko Eating Bananas	LeetCode 875	FFFTTT, minimize	Medium

Medium

Problem	Platform	Pattern	Difficulty
Capacity To Ship Packages Within D Days	LeetCode 1011	FFFTTT, minimize	Medium
Split Array Largest Sum	LeetCode 410	FFFTTT, minimize	Hard
Aggressive Cows	SPOJ / GFG	TTTFFF, maximize	Medium

Hard

Problem	Platform	Pattern	Difficulty
Find K-th Smallest Pair Distance	LeetCode 719	Count-based decision	Hard
Median of Two Sorted Arrays	LeetCode 4	Partition-based	Hard
Minimum Time to Complete Trips	LeetCode 2187	FFFTTT, minimize	Medium/Hard

18. Interview Explanation

"Binary search on answer is used when we need to find an optimal numeric value — like minimum speed, maximum distance, or minimum capacity — and we can check if a candidate value is feasible. The key insight is that the answer space is ordered and the feasibility function is monotonic. I define low and high bounds for the answer, then binary search, using a decision function that checks feasibility. For minimization, I use the lower bound template (first true). For maximization, I use the upper bound template with $(low + high + 1) / 2$ to avoid infinite loops. The decision function is typically greedy or DP. The overall time is $O(f(n) \times \log R)$, where $f(n)$ is the decision function complexity."

19. Revision Notes

- **Key idea:** Search for answer value, not array index.

- **Monotonicity required:** Feasibility must be monotonic (TTTTFFFF or FFFFTTTT).
- **Minimize (first true):** `mid = (low+high)/2; if(feasible) high=mid; else low=mid+1`
- **Maximize (last true):** `mid = (low+high+1)/2; if(feasible) low=mid; else high=mid-1`
- **Decision function:** Usually greedy, $O(n)$ or $O(n \log n)$.
- **Answer range:** Low = min possible, High = max possible.
- **Complexity:** $O(f(n) \times \log R)$
- **Common trap:** Wrong mid for maximization (infinite loop). Not monotonic function.

20. Final Cheat Sheet

BINARY SEARCH ON ANSWER – CHEAT SHEET	
WHEN TO USE:	Optimization → "min/max such that"
KEY IDEA:	Binary search on answer value
REQUIREMENT:	Monotonic feasible function
MINIMIZE:	<pre>while(l<r) { m=(l+r)/2; if(f(m)) r=m; else l=m+1 }</pre>
MAXIMIZE:	<pre>while(l<r) { m=(l+r+1)/2; if(f(m)) l=m; else r=m-1 }</pre>
DECISION FN:	Usually greedy, $O(n)$
COMPLEXITY:	$O(f(n) \times \log R)$
COMMON USE:	Koko, ship packages, agg cows, split array, kth smallest
TRAP:	Wrong mid for max, non-monotonic
REAL ANSWER:	<code>for(int i=0;i<100;i++) { ... }</code>

7. SORTING + GREEDY

1. Overview

Sorting + Greedy is a powerful combination where we first sort the data (often by a specific key) and then apply a greedy algorithm — making the locally optimal choice at each step to reach a globally optimal solution. Sorting enables the greedy choice by organizing the data so that the optimal next step is obvious.

2. Intuition

Core idea: Many optimization problems become trivial if the data is sorted. By sorting first, we can make decisions in order (smallest first, largest first, earliest deadline first, etc.) and the greedy choice at each step leads to the optimal solution.

Analogy: Packing a suitcase for a trip with limited space. You want to maximize the value of items you bring. If you sort items by value per weight (density), then greedily take the most valuable items first, you get a good (and sometimes optimal) packing.

Step-by-step reasoning:

1. Sort the data by some criterion (value, deadline, start time, weight, etc.).
2. Process elements in sorted order.
3. At each step, make the locally optimal choice (take it, schedule it, assign it).
4. The sorting ensures that the locally optimal choice doesn't prevent future optimal choices.

Why it works: For problems with optimal substructure and the greedy choice property, sorting ensures we can make the correct greedy decisions. The greedy choice property means that a globally optimal solution can be assembled by making locally optimal choices.

3. When to Use It

- **Activity selection / Interval scheduling:** Maximum number of non-overlapping intervals.
- **Fractional knapsack:** Sort by value/weight ratio.
- **Minimum number of platforms / rooms:** Sort start and end times.
- **Job sequencing with deadlines:** Sort by profit, schedule at latest possible slot.
- **Huffman coding:** Sort by frequency.
- **Minimum spanning tree (Kruskal's):** Sort edges by weight.
- **Coin change (greedy works):** Sort coins by denomination.
- **Assign cookies / distribute resources:** Sort both arrays, match greedily.

Common trigger phrases:

- "maximum number of"
- "minimum number of"
- "schedule"
- "assign"
- "sort by"
- "greedy"
- "optimize"
- "interval"

4. When Not to Use It

- **Greedy doesn't guarantee optimality:** Many problems need DP (e.g., 0/1 Knapsack, coin change where greedy fails).
- **Sorting is too expensive:** If $O(n \log n)$ is too slow for the constraints (rare, but happens with $n = 10^7$).
- **Multiple conflicting criteria:** If sorting by one criterion breaks another, you might need more advanced techniques.
- **Problem has no greedy choice property:** If the locally optimal choice can lead to a globally suboptimal solution, greedy fails.
- **Already sorted data:** Skip the sorting step.

5. Core Concepts

5.1 Greedy Choice Property

A problem has the greedy choice property if a globally optimal solution can be reached by making a series of locally optimal (greedy) choices.

Example: In activity selection, choosing the activity that ends earliest always leaves maximum room for remaining activities.

5.2 Optimal Substructure

A problem has optimal substructure if an optimal solution to the problem contains optimal solutions to its subproblems.

Example: After choosing the first activity, the optimal solution for the remaining activities is independent of the first choice.

5.3 Sorting Criteria

Different problems require different sorting keys:

- **By end time:** Activity selection.
- **By start time:** Merge intervals.
- **By value/weight:** Fractional knapsack.
- **By profit descending:** Job sequencing.
- **By absolute value:** Closest pair.
- **By index:** Keep original order if needed.

5.4 Custom Comparator

In C++, we can define custom sorting using lambda functions or comparator functions.

```
sort(intervals.begin(), intervals.end(), [](const auto& a, const auto& b) {
    return a[1] < b[1]; // sort by end time
});
```

6. Step-by-Step Algorithm

Activity Selection (Maximum number of non-overlapping intervals):

1. Sort activities by end time (ascending).
2. Initialize `end = -inf`, `count = 0`.
3. For each activity `[start, endTime]`:
 - If `start >= end`: select this activity, `end = endTime`, `count++`.
4. Return `count`.

Fractional Knapsack (Maximum value with weight constraint):

1. Sort items by value/weight ratio (descending).
2. Initialize `totalValue = 0`, `remainingWeight = W`.
3. For each item with `(value, weight, ratio)`:
 - If `remainingWeight >= weight`: take whole item. `totalValue += value`, `remainingWeight -= weight`.
 - Else: take fraction of item. `totalValue += ratio * remainingWeight`, break.
4. Return `totalValue`.

7. Dry Run

Problem: Activity Selection — Maximum number of non-overlapping meetings.

Input: `intervals = [[1,2], [3,4], [0,6], [5,7], [8,9], [5,9]]`

Step 1: Sort by end time

| After sorting | [1,2] | [3,4] | [0,6] | [5,7] | [8,9] | [5,9] | | End time | 2 | 4 | 6 | 7 | 9 | 9 |

Step 2: Greedy selection

Activity	start	end	start $\geq$ lastEnd?	Action	lastEnd	count
—	—	—	—	—	$-\infty$	0
[1,2]	1	2	$1 \geq -\infty \checkmark$	Take	2	1
[3,4]	3	4	$3 \geq 2 \checkmark$	Take	4	2
[0,6]	0	6	$0 \geq 4 \times$	Skip	4	2
[5,7]	5	7	$5 \geq 4 \checkmark$	Take	7	3
[8,9]	8	9	$8 \geq 7 \checkmark$	Take	9	4
[5,9]	5	9	$5 \geq 9 \times$	Skip	9	4

Result: Maximum 4 activities (sorted by end time: [1,2], [3,4], [5,7], [8,9])

Problem: Fractional Knapsack — $W = 50$

Input: `items = [(value=60, weight=10), (100, 20), (120, 30)]`

Step 1: Sort by value/weight ratio

Item	Value	Weight	Ratio (v/w)
1	60	10	6
2	100	20	5
3	120	30	4

Sorted by ratio descending: Item 1 (6) > Item 2 (5) > Item 3 (4)

Step 2: Greedy

Item	Value	Weight	Take?	Remaining	Total Value
1	60	10	Yes (full)	40	60
2	100	20	Yes (full)	20	160
3	120	30	20/30 fraction	0	$160 + 80 = 240$

Result: Maximum value = 240

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// =====
// 1. Activity Selection (Maximum non-overlapping intervals)
// =====
int activitySelection(vector<vector<int>>& intervals) {
    if (intervals.empty()) return 0;

    // Sort by end time
    sort(intervals.begin(), intervals.end(),
        [](const vector<int>& a, const vector<int>& b) {
            return a[1] < b[1];
        });

    int count = 1;
    int lastEnd = intervals[0][1];

    for (int i = 1; i < intervals.size(); i++) {
        if (intervals[i][0] >= lastEnd) {
            count++;
            lastEnd = intervals[i][1];
        }
    }
    return count;
}

// =====
// 2. Fractional Knapsack
// =====
struct Item {
    int value, weight;
    double ratio() const { return (double)value / weight; }
};

double fractionalKnapsack(vector<Item>& items, int W) {
    // Sort by value/weight ratio descending
    sort(items.begin(), items.end(), [](const Item& a, const Item& b) {
        return a.ratio() > b.ratio();
    });

    double totalValue = 0.0;
    int remaining = W;

    for (const auto& item : items) {
        if (remaining >= item.weight) {
            totalValue += item.value;
            remaining -= item.weight;
        }
    }
}
```

```

        } else {
            totalValue += item.ratio() * remaining;
            break;
        }
    }
    return totalValue;
}

// =====
// 3. Job Sequencing with Deadlines (Maximize profit)
// =====
struct Job {
    int id, deadline, profit;
};

int jobSequencing(vector<Job>& jobs) {
    // Sort by profit descending
    sort(jobs.begin(), jobs.end(), [](const Job& a, const Job& b) {
        return a.profit > b.profit;
    });

    int maxDeadline = 0;
    for (const auto& job : jobs)
        maxDeadline = max(maxDeadline, job.deadline);

    vector<int> slot(maxDeadline + 1, -1); // -1 = empty
    int totalProfit = 0;

    for (const auto& job : jobs) {
        // Find nearest empty slot <= deadline
        for (int j = job.deadline; j >= 1; j--) {
            if (slot[j] == -1) {
                slot[j] = job.id;
                totalProfit += job.profit;
                break;
            }
        }
    }
    return totalProfit;
}

// =====
// 4. Assign Cookies (Greedy matching)
// =====
int assignCookies(vector<int>& children, vector<int>& cookies) {
    sort(children.begin(), children.end());
    sort(cookies.begin(), cookies.end());

    int i = 0, j = 0;
    while (i < children.size() && j < cookies.size()) {
        if (cookies[j] >= children[i]) {
            i++; // satisfied this child
            j++;
        } else {

```

```

        j++; // try next cookie
    }
}
return i; // number of satisfied children
}

// Example usage
int main() {
    // Activity Selection
    vector<vector<int>> intervals = {{1,2}, {3,4}, {0,6}, {5,7}, {8,9}, {5,9}};
    cout << "Max activities: " << activitySelection(intervals) << "\n"; // 4

    // Fractional Knapsack
    vector<Item> items = {{60, 10}, {100, 20}, {120, 30}};
    cout << "Max value: " << fractionalKnapsack(items, 50) << "\n"; // 240

    // Job Sequencing
    vector<Job> jobs = {{1, 2, 100}, {2, 1, 19}, {3, 2, 27}, {4, 1, 25}, {5, 3,
15}};
    cout << "Max profit: " << jobSequencing(jobs) << "\n"; // 142

    // Assign Cookies
    vector<int> children = {1, 2, 3};
    vector<int> cookies = {1, 1};
    cout << "Satisfied children: " << assignCookies(children, cookies) << "\n"; //
1

    return 0;
}

```

9. Python Implementation

```

from typing import List

def activity_selection(intervals: List[List[int]]) -> int:
    if not intervals:
        return 0
    intervals.sort(key=lambda x: x[1]) # sort by end time
    count = 1
    last_end = intervals[0][1]
    for i in range(1, len(intervals)):
        if intervals[i][0] >= last_end:
            count += 1
            last_end = intervals[i][1]
    return count

def fractional_knapsack(items: List[tuple], W: int) -> float:
    # items: list of (value, weight)
    items.sort(key=lambda x: x[0] / x[1], reverse=True) # sort by ratio
    total_value = 0.0
    remaining = W

```

```

    for value, weight in items:
        if remaining >= weight:
            total_value += value
            remaining -= weight
        else:
            total_value += (value / weight) * remaining
            break
    return total_value

def job_sequencing(jobs: List[tuple]) -> int:
    # jobs: list of (id, deadline, profit)
    jobs.sort(key=lambda x: x[2], reverse=True) # sort by profit descending
    max_deadline = max(job[1] for job in jobs)
    slot = [-1] * (max_deadline + 1)
    total_profit = 0
    for job_id, deadline, profit in jobs:
        for j in range(deadline, 0, -1):
            if slot[j] == -1:
                slot[j] = job_id
                total_profit += profit
                break
    return total_profit

def assign_cookies(children: List[int], cookies: List[int]) -> int:
    children.sort()
    cookies.sort()
    i = j = 0
    while i < len(children) and j < len(cookies):
        if cookies[j] >= children[i]:
            i += 1
        j += 1
    return i

# Example usage
if __name__ == "__main__":
    print(activity_selection([[1,2], [3,4], [0,6], [5,7], [8,9], [5,9]])) # 4
    print(fractional_knapsack([(60, 10), (100, 20), (120, 30)], 50)) # 240.0
    print(job_sequencing([(1, 2, 100), (2, 1, 19), (3, 2, 27), (4, 1, 25), (5, 3, 15)])) # 142
    print(assign_cookies([1, 2, 3], [1, 1])) # 1

```

10. Code Explanation

Activity Selection:

- Sort by end time (ascending). This ensures we always pick the activity that finishes earliest, leaving maximum room for others.
- Greedy: pick activity if its start $\geq$ last selected end time.
- **Why it's optimal:** The earliest-finishing activity is always part of some optimal solution. Picking it doesn't hurt.

Fractional Knapsack:

- Sort by value/weight ratio (descending). This ensures we take the most "valuable per weight" items first.
- Greedy: take whole items until we can't, then take a fraction.
- **Why greedy works:** We can take fractions, so we always want the best ratio first. This is optimal.

Job Sequencing:

- Sort by profit descending. Process most profitable jobs first.
- For each job, try to schedule it at the latest possible slot before its deadline. This leaves earlier slots for other jobs.
- **Why greedy works:** Processing by profit ensures we maximize profit. Scheduling at the latest slot leaves room for jobs with earlier deadlines.

Assign Cookies:

- Sort both arrays. Smallest greed factor first, smallest cookie that satisfies it.
- **Why greedy works:** Matching the smallest possible cookie to each child (in order) ensures we don't waste big cookies on small children.

11. Complexity Analysis

Algorithm	Time	Space	Notes
Activity Selection	$O(n \log n)$	$O(1)$	Sorting dominates
Fractional Knapsack	$O(n \log n)$	$O(1)$	Sorting + linear scan
Job Sequencing	$O(n \log n + n \times d)$	$O(d)$	$d = \text{max deadline}$
Assign Cookies	$O(n \log n + m \log m)$	$O(1)$	Two sorts
Minimum Platforms	$O(n \log n)$	$O(1)$	Sort arrivals + departures
Huffman Coding	$O(n \log n)$	$O(n)$	Min-heap based

12. Common Patterns

Pattern 1: Interval Scheduling

Identify: "Maximum number of non-overlapping intervals/meetings." **Approach:** Sort by end time, greedy pick. **Example:** LeetCode 435 — Non-overlapping Intervals.

Pattern 2: Minimum Platforms / Rooms

Identify: "Minimum number of platforms required for all trains." **Approach:** Sort arrival and departure times separately. Two pointers counting active trains. **Example:** GFG — Minimum Number of Platforms.

Pattern 3: Scheduling with Deadlines

Identify: "Jobs with deadlines, maximize profit." **Approach:** Sort by profit descending, schedule at latest slot. **Example:** GFG — Job Sequencing Problem.

Pattern 4: Greedy Matching

Identify: "Assign resources to requests." **Approach:** Sort both, greedily match. **Examples:** LeetCode 455 — Assign Cookies, LeetCode 826 — Most Profit Assigning Work.

Pattern 5: Fractional Selection

Identify: "Take fractions of items, maximize value." **Approach:** Sort by value/weight ratio. **Example:** GFG — Fractional Knapsack.

Pattern 6: Huffman Coding

Identify: "Minimum length encoding, prefix-free codes." **Approach:** Min-heap, merge two smallest frequencies. **Example:** GFG — Huffman Coding.

13. Common Mistakes

- **Wrong sorting order:** Ascending vs descending makes a huge difference. Always verify.
- **Greedy when DP is needed:** Mistaking 0/1 knapsack (DP) for fractional knapsack (greedy).
- **Not considering ties in sorting:** When two items have the same key, the secondary criterion might matter.
- **Modifying original order when not needed:** If original order is needed, save a copy of indices.
- **Forgetting to handle empty input.**
- **Using wrong comparator:** $a < b$ vs $a > b$ in comparator. The comparator should return true if a should come before b .
- **Not realizing greedy fails for some problems:** Test with small examples to verify greedy optimality.

14. Edge Cases

Case	Expected Behavior
Empty intervals	Return 0
Single interval	Return 1
All overlapping	Non-overlapping = 1
No overlapping	All intervals selected
$W = 0$ in knapsack	Return 0
All items fit in knapsack	Take all
No children / no cookies	Return 0
Negative values (unusual)	Handle carefully, ratio may be negative
Deadlines = 0	Job can't be scheduled

15. Variations

Sort by Start Time (Merge Intervals)

Instead of sorting by end, sort by start. Used for merging overlapping intervals. **Example:** LeetCode 56 — Merge Intervals.

Sort by Start + End (Meeting Rooms II)

Sort by start time, use min-heap of end times. **Example:** LeetCode 253 — Meeting Rooms II.

Sort by Absolute Value

Used for closest pair, nearest point problems.

Sort by Multiple Keys

Use tuple as key: `sort by (end, start)` or `sort by (-profit, deadline)`.

16. Related Algorithms/Data Structures

Algorithm	Connection
Dynamic Programming	Greedy works when DP is not needed. If greedy fails, use DP.
Heap	Often used with sorting for dynamic greedy (e.g., Huffman, Dijkstra).
Two Pointers	After sorting, two pointers can process efficiently.
Binary Search on Answer	Sometimes combined with sorting + greedy as decision function.
Minimum Spanning Tree	Kruskal's = sort edges by weight + DSU (greedy).
Dijkstra's Algorithm	Greedy algorithm for shortest path (uses priority queue).

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Assign Cookies	LeetCode 455	Greedy matching	Easy
Non-overlapping Intervals	LeetCode 435	Interval scheduling	Medium

Medium

Problem	Platform	Pattern	Difficulty
Minimum Number of Arrows to Burst Balloons	LeetCode 452	Interval scheduling	Medium
Job Sequencing Problem	GFG	Scheduling with deadlines	Medium
Partition Labels	LeetCode 763	Greedy with last indices	Medium

Hard

Problem	Platform	Pattern	Difficulty
Minimum Time to Make Rope Colorful	LeetCode 1578	Greedy with DP	Medium
Maximum Profit in Job Scheduling	LeetCode 1235	Sorting + DP	Hard
Candy	LeetCode 135	Two-pass greedy	Hard

18. Interview Explanation

"Sorting + greedy is a technique where we first sort the data by a specific criterion, then make locally optimal choices to reach a globally optimal solution. The key is identifying the right sorting criterion and proving the greedy choice property. For example, in activity selection, we sort by end time and greedily pick the earliest-finishing activity. In fractional knapsack, we sort by value-to-weight ratio. The sorting is typically $O(n \log n)$, and the greedy pass is $O(n)$. I make sure the problem has both optimal substructure and the greedy choice property before using this approach."

19. Revision Notes

- **Key idea:** Sort + greedy pass = optimal solution for many problems.
- **Sorting criteria depends on problem:** End time, ratio, profit, deadline, etc.
- **Activity selection:** Sort by end time, pick non-overlapping.
- **Fractional knapsack:** Sort by value/weight, take whole then fraction.
- **Job sequencing:** Sort by profit descending, schedule at latest slot.
- **Assign cookies:** Sort both, match smallest.
- **Complexity:** $O(n \log n)$ for sorting, $O(n)$ for greedy.
- **Common trap:** Using greedy when DP is needed (0/1 knapsack, coin change with non-canonical coins).

20. Final Cheat Sheet

SORTING + GREEDY – CHEAT SHEET	
WHEN TO USE:	Optimization with greedy choice
KEY IDEA:	Sort → greedy pass
COMMON SORTS:	By end time (intervals), by ratio (knapsack), by profit (jobs), by value (matching)
COMPLEXITY:	$O(n \log n) + O(n)$
ACTIVITY SEL:	sort by end, pick if $\text{start} \geq \text{lastEnd}$
FRAC KNAPSACK:	sort by v/w , take whole → fraction
JOB SEQ:	sort by profit, schedule at latest
COOKIES:	sort both, match smallest possible
TRAP:	Greedy fails for 0/1 knapsack, coin change (non-canonical)
RELATED:	DP, Heap, Two pointers, MST

8. MONOTONIC STACK

1. Overview

Monotonic stack is a stack that maintains elements in a specific order (increasing or decreasing) as we iterate through an array. It is used to find the **next greater element**, **previous smaller element**, or similar relationships in $O(n)$ time — problems that would otherwise require $O(n^2)$.

2. Intuition

Core idea: As we traverse the array, we maintain a stack that is always increasing (or decreasing). When we encounter a new element, we pop elements from the stack that violate the monotonic property. The act of popping reveals relationships between the current element and the popped elements.

Analogy: You're organizing a line of people by height. You want to know, for each person, who is the next taller person in line. As you scan from left to right, you keep a stack of people waiting to find their next taller person. When a taller person arrives, all shorter people on the stack can finally see their next taller person — they pop off satisfied.

Step-by-step reasoning:

1. We want to find the next greater element (NGE) for each element.
2. Maintain a stack of indices with decreasing values (bottom to top: largest to smallest).
3. For each element `arr[i]`:
 - While stack is not empty and `arr[stack.top()] < arr[i]`:
 - `arr[stack.top()]` has found its NGE = `arr[i]`.
 - Pop the stack.
 - Push `i` onto the stack.
4. Elements remaining in stack have no NGE.

Why it works: The stack maintains a decreasing sequence of values whose NGE hasn't been found yet. When we see a larger value, it becomes the NGE for all smaller values on the stack.

3. When to Use It

- **Next Greater Element** (NGE): Find the next larger element to the right.
- **Next Smaller Element** (NSE): Find the next smaller element to the right.
- **Previous Greater Element** (PGE): Find the previous larger element to the left.
- **Previous Smaller Element** (PSE): Find the previous smaller element to the left.
- **Largest rectangle in histogram** (classic problem).
- **Trapping rain water** (can be solved with monotonic stack).
- **Daily temperatures** (days until warmer temperature).
- **Stock span** (consecutive days with lower price).
- **Maximum of minimums of every window size.**

Common trigger phrases:

- "next greater"
- "next smaller"

- "previous greater"
- "previous smaller"
- "nearest greater"
- "nearest smaller"
- "distance to next greater"
- "largest rectangle in histogram"
- "stock span"

4. When Not to Use It

- **Small n ($n \leq 1000$):** $O(n^2)$ is fine, monotonic stack might be overkill.
- **Need arbitrary order queries:** If you need to query arbitrary pairs (not just next/previous), use segment tree or sparse table.
- **Non-comparable elements:** Stack requires ordering.
- **Problem doesn't require nearest relationship:** If you just need max/min of subarrays, sliding window with deque might be simpler.

5. Core Concepts

5.1 Monotonic Increasing Stack

Elements in stack increase from bottom to top. Used for finding next smaller element.

```
// Stack is increasing: stack.top() is the largest element
// When we see a smaller element, it's the NSE for elements on stack
for (int i = 0; i < n; i++) {
    while (!st.empty() && arr[st.top()] > arr[i]) {
        // arr[i] is NSE for arr[st.top()]
        st.pop();
    }
    st.push(i);
}
```

5.2 Monotonic Decreasing Stack

Elements in stack decrease from bottom to top. Used for finding next greater element.

```
// Stack is decreasing: stack.top() is the smallest element
// When we see a larger element, it's the NGE for elements on stack
for (int i = 0; i < n; i++) {
    while (!st.empty() && arr[st.top()] < arr[i]) {
        // arr[i] is NGE for arr[st.top()]
        st.pop();
    }
    st.push(i);
}
```

5.3 Next vs Previous

- **Next:** Scan left to right. Stack stores elements waiting for their next greater/smaller.
- **Previous:** Scan left to right. The element at `stack.top()` is the previous greater/smaller for the current element.

5.4 Index vs Value in Stack

We usually store **indices** in the stack, not values. This allows us to compute distances (number of elements between) and access the original array.

6. Step-by-Step Algorithm

Next Greater Element (NGE) for each element:

1. Initialize empty stack, result array `nge[n]` filled with -1.
2. For `i = 0` to `n-1`:
 - While stack not empty and `arr[stack.top()] < arr[i]`:
 - `nge[stack.top()] = arr[i]` (or `i` depending on what's needed).
 - Pop stack.
 - Push `i` onto stack.
3. Elements still in stack have no NGE (remain -1).

Largest Rectangle in Histogram:

1. Initialize empty stack, `maxArea = 0`.
2. For `i = 0` to `n` (one extra iteration for cleanup):
 - While stack not empty and (`i == n` or `arr[i] < arr[stack.top()]`):
 - `height = arr[stack.pop()]`.
 - `width = stack.empty() ? i : i - stack.top() - 1`.
 - `maxArea = max(maxArea, height * width)`.
 - Push `i` onto stack.
3. Return `maxArea`.

7. Dry Run

Problem: Next Greater Element

Input: `arr = [2, 1, 5, 3, 4]`

Step	i	arr[i]	Stack (indices→values)	NGE updates
1	0	2	[0] → [2]	—
2	1	1	[0,1] → [2,1]	— (1 < 2, no pop)
3	2	5	Pop 1: 1 < 5 → NGE[1]=5	NGE[1]=5
			Pop 0: 2 < 5 → NGE[0]=5	NGE[0]=5
			Push 2 → [2] → [5]	

Step	i	arr[i]	Stack (indices→values)	NGE updates
4	3	3	[2,3] → [5,3]	— (3 < 5, no pop)
5	4	4	Pop 3: 3 < 4 → NGE[3]=4	NGE[3]=4
Push 4 → [2,4] → [5,4]				

Result: NGE = [5, 5, -1, 4, -1]

Problem: Largest Rectangle in Histogram

Input: heights = [2, 1, 5, 6, 2, 3]

i	heights[i]	Stack Before	Action	Height	Width	Area	Stack After
0	2	[]	Push 0	—	—	—	[0]
1	1	[0]	h[0]=2 > 1 → pop 0	2	1	2	[]
			Push 1	—	—	—	[1]
2	5	[1]	h[1]=1 ≤ 5 → push 2	—	—	—	[1,2]
3	6	[1,2]	h[2]=5 ≤ 6 → push 3	—	—	—	[1,2,3]
4	2	[1,2,3]	h[3]=6 > 2 → pop 3	6	1	6	[1,2]
			h[2]=5 > 2 → pop 2	5	2	10	[1]
			h[1]=1 ≤ 2 → push 4	—	—	—	[1,4]
5	3	[1,4]	h[4]=2 ≤ 3 → push 5	—	—	—	[1,4,5]
6	—	[1,4,5]	Cleanup: pop 5	3	1	3	[1,4]
			pop 4	2	4	8	[1]
			pop 1	1	6	6	[]

Result: Max area = 10 (rectangle of height 5, width 2)

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// =====
// 1. Next Greater Element (NGE)
// =====
vector<int> nextGreaterElement(const vector<int>& arr) {
    int n = arr.size();
    vector<int> nge(n, -1);
    stack<int> st; // stores indices
```

```

    for (int i = 0; i < n; i++) {
        while (!st.empty() && arr[st.top()] < arr[i]) {
            nge[st.top()] = arr[i]; // or store i for index
            st.pop();
        }
        st.push(i);
    }
    return nge;
}

// =====
// 2. Next Smaller Element (NSE)
// =====
vector<int> nextSmallerElement(const vector<int>& arr) {
    int n = arr.size();
    vector<int> nse(n, -1);
    stack<int> st;

    for (int i = 0; i < n; i++) {
        while (!st.empty() && arr[st.top()] > arr[i]) {
            nse[st.top()] = arr[i];
            st.pop();
        }
        st.push(i);
    }
    return nse;
}

// =====
// 3. Previous Greater Element (PGE)
// =====
vector<int> previousGreaterElement(const vector<int>& arr) {
    int n = arr.size();
    vector<int> pge(n, -1);
    stack<int> st;

    for (int i = 0; i < n; i++) {
        while (!st.empty() && arr[st.top()] <= arr[i]) {
            st.pop(); // pop smaller or equal elements
        }
        if (!st.empty()) pge[i] = arr[st.top()];
        st.push(i);
    }
    return pge;
}

// =====
// 4. Largest Rectangle in Histogram
// =====
int largestRectangleArea(const vector<int>& heights) {
    int n = heights.size();
    stack<int> st;
    int maxArea = 0;

```

```

    for (int i = 0; i <= n; i++) {
        int currHeight = (i == n) ? 0 : heights[i];

        while (!st.empty() && heights[st.top()] > currHeight) {
            int height = heights[st.top()];
            st.pop();
            int width = st.empty() ? i : i - st.top() - 1;
            maxArea = max(maxArea, height * width);
        }
        st.push(i);
    }
    return maxArea;
}

// =====
// 5. Daily Temperatures (days until warmer)
// =====
vector<int> dailyTemperatures(const vector<int>& temperatures) {
    int n = temperatures.size();
    vector<int> ans(n, 0);
    stack<int> st; // indices of temperatures

    for (int i = 0; i < n; i++) {
        while (!st.empty() && temperatures[st.top()] < temperatures[i]) {
            int prevIdx = st.top();
            st.pop();
            ans[prevIdx] = i - prevIdx;
        }
        st.push(i);
    }
    return ans;
}

// Example usage
int main() {
    vector<int> arr = {2, 1, 5, 3, 4};

    auto nge = nextGreaterElement(arr);
    cout << "NGE: ";
    for (int x : nge) cout << x << " ";
    cout << "\n"; // 5 5 -1 4 -1

    auto nse = nextSmallerElement(arr);
    cout << "NSE: ";
    for (int x : nse) cout << x << " ";
    cout << "\n"; // 1 -1 3 -1 -1

    // Largest Rectangle
    vector<int> heights = {2, 1, 5, 6, 2, 3};
    cout << "Largest Rectangle: " << largestRectangleArea(heights) << "\n"; // 10

    // Daily Temperatures
    vector<int> temps = {73, 74, 75, 71, 69, 72, 76, 73};
    auto days = dailyTemperatures(temps);

```

```
    cout << "Daily Temps: ";
    for (int d : days) cout << d << " ";
    cout << "\n"; // 1 1 4 2 1 1 0 0

    return 0;
}
```

9. Python Implementation

```
from typing import List

def next_greater_element(arr: List[int]) -> List[int]:
    n = len(arr)
    nge = [-1] * n
    stack = []
    for i in range(n):
        while stack and arr[stack[-1]] < arr[i]:
            nge[stack.pop()] = arr[i]
        stack.append(i)
    return nge

def next_smaller_element(arr: List[int]) -> List[int]:
    n = len(arr)
    nse = [-1] * n
    stack = []
    for i in range(n):
        while stack and arr[stack[-1]] > arr[i]:
            nse[stack.pop()] = arr[i]
        stack.append(i)
    return nse

def previous_greater_element(arr: List[int]) -> List[int]:
    n = len(arr)
    pge = [-1] * n
    stack = []
    for i in range(n):
        while stack and arr[stack[-1]] <= arr[i]:
            stack.pop()
        if stack:
            pge[i] = arr[stack[-1]]
        stack.append(i)
    return pge

def largest_rectangle_area(heights: List[int]) -> int:
    n = len(heights)
    stack = []
    max_area = 0

    for i in range(n + 1):
        curr_height = 0 if i == n else heights[i]
```

```

        while stack and heights[stack[-1]] > curr_height:
            height = heights[stack.pop()]
            width = i if not stack else i - stack[-1] - 1
            max_area = max(max_area, height * width)
        stack.append(i)
    return max_area

def daily_temperatures(temperatures: List[int]) -> List[int]:
    n = len(temperatures)
    ans = [0] * n
    stack = []
    for i in range(n):
        while stack and temperatures[stack[-1]] < temperatures[i]:
            prev = stack.pop()
            ans[prev] = i - prev
        stack.append(i)
    return ans

# Example usage
if __name__ == "__main__":
    print(next_greater_element([2, 1, 5, 3, 4]))          # [5, 5, -1, 4, -1]
    print(next_smaller_element([2, 1, 5, 3, 4]))          # [1, -1, 3, -1, -1]
    print(largest_rectangle_area([2, 1, 5, 6, 2, 3]))      # 10
    print(daily_temperatures([73, 74, 75, 71, 69, 72, 76, 73])) #
[1,1,4,2,1,1,0,0]

```

10. Code Explanation

Next Greater Element:

- Stack stores indices of elements whose NGE hasn't been found yet.
- Stack is maintained in decreasing order (bottom to top: largest to smallest).
- When `arr[i] > arr[stack.top()]`, `arr[i]` is the NGE for the top element.
- Pop and record, then push current index.
- Elements remaining in stack have no NGE.

Next Smaller Element:

- Same logic but with `>` comparison (stack is increasing).
- When `arr[i] < arr[stack.top()]`, `arr[i]` is the NSE.

Previous Greater Element:

- Stack stores indices of elements seen so far.
- Stack is maintained in decreasing order.
- For current element, pop elements $\leq$ current (they can't be PGE for any future element).
- Top of stack is the PGE for current element (if stack is not empty).

Largest Rectangle in Histogram:

- The stack maintains indices of increasing heights.

- When we see a smaller height, we pop from stack and calculate area for each popped height.
- For a popped height h , the rectangle extends from the new top of stack (or 0) to the current index.
- The extra iteration $i = n$ (with height 0) forces all remaining elements to be processed.

11. Complexity Analysis

Algorithm	Time	Space	Notes
NGE / NSE	$O(n)$	$O(n)$	Each element pushed/popped at most once
PGE / PSE	$O(n)$	$O(n)$	Same
Largest Rectangle	$O(n)$	$O(n)$	Same
Daily Temperatures	$O(n)$	$O(n)$	Same
Stock Span	$O(n)$	$O(n)$	Same

Key insight: Each element is pushed onto the stack exactly once and popped at most once. Total operations = $O(n)$.

12. Common Patterns

Pattern 1: Next Greater/Smaller Element

Identify: "Find the next greater element to the right." **Approach:** Monotonic decreasing stack for NGE, increasing for NSE. **Examples:** LeetCode 496 — Next Greater Element I, LeetCode 503 — Next Greater Element II.

Pattern 2: Previous Greater/Smaller Element

Identify: "Find the previous greater element to the left." **Approach:** Scan left to right, maintain decreasing stack, pop while $\text{top} \leq \text{current}$. **Example:** GFG — Previous Greater Element.

Pattern 3: Largest Rectangle in Histogram

Identify: "Largest rectangle in histogram." **Approach:** Monotonic stack, calculate area when a smaller height is encountered. **Example:** LeetCode 84 — Largest Rectangle in Histogram.

Pattern 4: Maximal Rectangle in Binary Matrix

Identify: "Largest rectangle containing only 1s in binary matrix." **Approach:** Row by row, compute heights, apply largest rectangle in histogram. **Example:** LeetCode 85 — Maximal Rectangle.

Pattern 5: Stock Span / Consecutive Smaller Elements

Identify: "Consecutive days with smaller/equal prices." **Approach:** Monotonic stack counting consecutive smaller elements. **Example:** GFG — Stock Span Problem.

Pattern 6: Trapping Rain Water

Identify: "Amount of water that can be trapped." **Approach:** Monotonic (decreasing) stack. When a higher bar appears, calculate trapped water. **Example:** LeetCode 42 — Trapping Rain Water.

13. Common Mistakes

- **Wrong comparison operator:** For NGE, use `arr[st.top()] < arr[i]`. For NSE, use `arr[st.top()] > arr[i]`. Getting this reversed is the most common bug.
- **Storing values instead of indices:** Always store indices in the stack. You need them for distance calculations and to access the original array.
- **Forgetting to handle remaining elements:** After the loop, elements in stack have no NGE/NSE. Initialize result array with -1 or 0.
- **Incorrect width calculation in histogram:** `width = (st.empty() ? i : i - st.top() - 1)`. The -1 accounts for the current element being the boundary.
- **Not handling duplicates:** Decide whether to use `<` or `<=` based on whether you want strictly greater/smaller.
- **Empty stack check:** Always check `!st.empty()` before accessing `st.top()`.

14. Edge Cases

Case	Expected Behavior
Empty array	Return empty
Single element	NGE = -1, PGE = -1
All increasing	NGE = -1 for all (last has no NGE), PGE = previous element
All decreasing	NGE = next element, PGE = -1 for all
All equal	NGE = -1 (if strict), PGE = -1 (if strict)
Duplicates	Depends on strict vs non-strict comparison
Histogram all same	Max area = $h \times n$
Histogram single bar	Height of that bar

15. Variations

Circular Array (Next Greater Element II)

For circular arrays, iterate twice: `for (int i = 0; i < 2 * n; i++)` and use `i % n`. **Example:** LeetCode 503 — Next Greater Element II.

Strict vs Non-Strict

- Strict: Use `<` or `>` (no equal).
- Non-strict: Use `<=` or `>=` (includes equal).
- Choose based on problem requirements.

Monotonic Stack with Pair

Sometimes you need to store both value and index (or other metadata) in the stack.

Stack of Indices vs Values

Always prefer indices. Values can be ambiguous when duplicates exist.

16. Related Algorithms/Data Structures

Data Structure	Connection
Stack	Foundation data structure.
Deque (Monotonic Queue)	Used for sliding window maximum/minimum. Similar concept but deque.
Segment Tree / Sparse Table	Can answer "next greater/smaller" type queries in $O(\log n)$ but with preprocessing.
Two Pointers	Can sometimes be used instead of monotonic stack (e.g., trapping rain water).

Decision guide:

- Need next/previous greater/smaller for all elements → Monotonic stack.
- Need slide window max/min → Deque.
- Need arbitrary range queries → Segment tree / Sparse table.
- Need top k elements → Heap.

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Next Greater Element I	LeetCode 496	NGE for subset	Easy
Base: Stock Span Problem	GFG	Consecutive smaller	Medium

Medium

Problem	Platform	Pattern	Difficulty
Daily Temperatures	LeetCode 739	NGE with distance	Medium
Largest Rectangle in Histogram	LeetCode 84	Largest rectangle	Hard
Online Stock Span	LeetCode 901	Stock span	Medium

Hard

Problem	Platform	Pattern	Difficulty
Maximal Rectangle	LeetCode 85	Binary matrix + histogram	Hard

Problem	Platform	Pattern	Difficulty
Trapping Rain Water	LeetCode 42	Water trapping	Hard
Sum of Subarray Minimums	LeetCode 907	Contribution using NSE/PSE	Medium/Hard

18. Interview Explanation

"Monotonic stack is a technique where we maintain a stack that is always increasing or decreasing. As we traverse the array, we pop elements that violate the monotonic property, and these pops reveal relationships between elements. For example, in finding the next greater element, we maintain a decreasing stack. When we encounter a larger element, it becomes the next greater element for all smaller elements on the stack. The key insight is that each element is pushed and popped at most once, giving $O(n)$ time. I use it for problems like next greater element, largest rectangle in histogram, and daily temperatures. The hardest part is getting the comparison direction right — I always double-check whether I need an increasing or decreasing stack."

19. Revision Notes

- **Core idea:** Maintain monotonic order in stack, pop to reveal relationships.
- **NGE:** Decreasing stack, pop when `arr[top] < arr[i]`.
- **NSE:** Increasing stack, pop when `arr[top] > arr[i]`.
- **Always store indices** in stack, not values.
- **Largest rectangle:** Pop when height decreases, `width = i - st.top() - 1`.
- **Complexity:** $O(n)$ time, $O(n)$ space.
- **Common trap:** Wrong comparison operator, not checking empty stack.

20. Final Cheat Sheet

MONOTONIC STACK – CHEAT SHEET	
WHEN TO USE:	Next/prev greater/smaller, histogram, stock span, rain water
NGE:	decreasing stack, pop if <code>top < cur</code>
NSE:	increasing stack, pop if <code>top > cur</code>
PGE:	decreasing, pop if <code>top ≤ cur</code>
KEY CODE:	<pre>while(!st.empty() && cond) { result[st.top()] = cur; pop; } push(i)</pre>
HISTOGRAM:	<code>while(top > cur) { pop, calc area }</code>
COMPLEXITY:	$O(n)$ time, $O(n)$ space
STORE:	indices, not values
CIRCULAR:	iterate $2*n$, use <code>i % n</code>
TRAP:	Wrong comparison (<code>>/<</code> vs <code></></code>), empty stack check

9. HEAP / TOP K

1. Overview

A heap is a complete binary tree where each parent node is either $\leq$ (min-heap) or $\geq$ (max-heap) its children. Heaps are used as priority queues — we can efficiently get the minimum or maximum element, insert new elements, and remove the top element, all in $O(\log n)$ time.

The "Top K" pattern uses heaps to find the K largest, K smallest, K most frequent, or K closest elements in a collection.

2. Intuition

Core idea: A heap maintains a partially ordered structure where the smallest (or largest) element is always at the top. By keeping only the K elements we care about in the heap, we can process large datasets while using only $O(K)$ memory.

Analogy: You're a judge in a talent show with 10,000 contestants, but only 3 finalists can go through. You don't need to rank all 10,000 — you just need to keep track of the top 3 at any point. Whenever a new contestant performs, if they're better than the current 3rd best, they replace that person. A min-heap of size 3 does exactly this: the top of the heap is the 3rd best (the weakest among the top 3), and any new contestant better than that replaces it.

Step-by-step reasoning:

1. To find top K largest elements:

- Create a min-heap of size K.
- For each element: if heap size $< K$, push. Else, if element $> \text{heap.top}()$, pop and push.
- At the end, heap contains K largest elements.

2. To find top K smallest elements:

- Create a max-heap of size K.
- For each element: if heap size $< K$, push. Else, if element $< \text{heap.top}()$, pop and push.

Why it works: The heap top represents the "least among the best" (for min-heap of top K largest). Any element that can't beat this threshold is irrelevant. Only elements that beat the threshold enter the heap.

3. When to Use It

- **Top K largest / smallest** elements.
- **K most frequent** elements.
- **K closest** points to origin.
- **Median from data stream** (two heaps).
- **Merge K sorted** lists/arrays.
- **Dijkstra's algorithm** (shortest path).
- **Prim's algorithm** (MST).
- **Huffman coding** (merge smallest frequencies).

- **Task scheduler** / CPU scheduling.
- **Sliding window median** / sliding window max.

Common trigger phrases:

- "top K"
- "K largest"
- "K smallest"
- "K most frequent"
- "K closest"
- "Kth largest"
- "Kth smallest"
- "median"
- "priority queue"
- "merge K sorted"

4. When Not to Use It

- **Need all elements sorted:** Use `sort()` — $O(n \log n)$ is simpler.
- **Small n:** Sorting or linear scan is simpler.
- **Need $O(1)$ access to min and max simultaneously:** Use a data structure that supports both (balanced BST, or two heaps).
- **K is close to n:** If $K = n$, sorting is simpler and $O(n \log n)$ for both.
- **Need to find Kth element in a sorted array:** Just index it directly.
- **Memory is a concern:** Heap uses $O(K)$ or $O(n)$ space.

5. Core Concepts

5.1 Min-Heap vs Max-Heap

- **Min-heap:** `top()` returns the smallest element. Parent $\leq$ children.
- **Max-heap:** `top()` returns the largest element. Parent $\geq$ children.

In C++:

- `priority_queue<int, vector<int>, greater<int>>` → min-heap.
- `priority_queue<int>` → max-heap (default).

5.2 Custom Comparator

For custom objects or custom ordering:

```
// Min-heap of pairs (by second element)
auto cmp = [](const pair<int,int>& a, const pair<int,int>& b) {
    return a.second > b.second; // > for min-heap
};
priority_queue<pair<int,int>, vector<pair<int,int>>, decltype(cmp)> pq(cmp);
```

5.3 Top K Pattern

Top K largest: Min-heap of size K. Keep the K largest elements. **Top K smallest:** Max-heap of size K. Keep the K smallest elements. **Kth largest:** Same as Top K largest, return `pq.top()`.

5.4 Two Heaps (Median Maintenance)

Maintain two heaps:

- Max-heap for the left half (smaller elements).
- Min-heap for the right half (larger elements).
- Balance: $\text{abs}(\text{left.size()} - \text{right.size()}) \leq 1$.
- Median = top of larger heap (or average of both tops).

5.5 Heapify

Building a heap from an array in $O(n)$ time (not $O(n \log n)$):

```
make_heap(arr.begin(), arr.end()); // max-heap
```

In C++, `priority_queue` constructor with container is $O(n)$ for heapify.

6. Step-by-Step Algorithm

Top K Largest Elements:

1. Create a min-heap.
2. For each element in array:
 - Push element into heap.
 - If heap size > K: pop the smallest element.
3. After loop, heap contains K largest elements.
4. (Optional) Extract elements from heap in descending order.

Kth Largest Element:

1. Same as above, but after processing all elements, return `pq.top()` (the smallest among the K largest = Kth largest).

Merge K Sorted Lists:

1. Create a min-heap of (value, list_index, element_index).
2. Push first element of each list.
3. While heap is not empty:
 - Pop smallest element, add to result.
 - If the list has more elements, push the next element from that list.

7. Dry Run

Problem: Find Kth Largest Element ($K = 3$)

Input: `arr = [3, 2, 1, 5, 6, 4], K = 3`

Min-heap of size 3:

Step	Element	Heap Before	Action	Heap After
1	3	[]	Push	[3]
2	2	[3]	Push	[2, 3]
3	1	[2, 3]	Push	[1, 2, 3]
4	5	[1, 2, 3]	5 > 1 → pop 1, push 5	[2, 3, 5]
5	6	[2, 3, 5]	6 > 2 → pop 2, push 6	[3, 5, 6]
6	4	[3, 5, 6]	4 > 3 → pop 3, push 4	[4, 5, 6]

Result: `pq.top()` = 4 (3rd largest)

Problem: Top K Frequent Elements

Input: `nums = [1,1,1,2,2,3], K = 2`

Step 1: Build frequency map

Element	Frequency
1	3
2	2
3	1

Step 2: Min-heap of size K = 2 (by frequency)

Element (freq)	Heap Before	Action	Heap After
1 (3)	[]	Push	[(1,3)]
2 (2)	[(1,3)]	Push	[(2,2), (1,3)]
3 (1)	[(2,2), (1,3)]	1 < 2 → skip	[(2,2), (1,3)]

Result: Elements in heap: [2, 1] (top 2 frequent)

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// =====
// 1. Kth Largest Element in Array
// =====
```

```

int findKthLargest(vector<int>& nums, int k) {
    priority_queue<int, vector<int>, greater<int>> minHeap;

    for (int num : nums) {
        minHeap.push(num);
        if (minHeap.size() > k) {
            minHeap.pop(); // remove smallest among k+1
        }
    }
    return minHeap.top(); // kth largest
}

// =====
// 2. Top K Frequent Elements
// =====
vector<int> topKFrequent(vector<int>& nums, int k) {
    // Count frequencies
    unordered_map<int, int> freq;
    for (int num : nums) freq[num]++;

    // Min-heap of (frequency, value)
    priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>>
minHeap;

    for (auto& [val, count] : freq) {
        minHeap.push({count, val});
        if (minHeap.size() > k) {
            minHeap.pop();
        }
    }

    vector<int> result;
    while (!minHeap.empty()) {
        result.push_back(minHeap.top().second);
        minHeap.pop();
    }
    // If you want descending order of frequency, reverse result
    reverse(result.begin(), result.end());
    return result;
}

// =====
// 3. Merge K Sorted Lists
// =====
struct ListNode {
    int val;
    ListNode* next;
    ListNode(int x) : val(x), next(nullptr) {}
};

struct Compare {
    bool operator()(ListNode* a, ListNode* b) {
        return a->val > b->val; // min-heap
    }
}

```

```

};

ListNode* mergeKLists(vector<ListNode*>& lists) {
    priority_queue<ListNode*, vector<ListNode*>, Compare> pq;

    // Push first node of each list
    for (ListNode* list : lists) {
        if (list) pq.push(list);
    }

    ListNode dummy(0);
    ListNode* tail = &dummy;

    while (!pq.empty()) {
        ListNode* node = pq.top();
        pq.pop();
        tail->next = node;
        tail = tail->next;

        if (node->next) pq.push(node->next);
    }
    return dummy.next;
}

// =====
// 4. Median from Data Stream
// =====
class MedianFinder {
private:
    priority_queue<int> maxHeap; // left half (smaller numbers)
    priority_queue<int, vector<int>, greater<int>> minHeap; // right half

public:
    void addNum(int num) {
        // Insert into appropriate heap
        if (maxHeap.empty() || num <= maxHeap.top()) {
            maxHeap.push(num);
        } else {
            minHeap.push(num);
        }

        // Balance: maxHeap can have at most 1 more element
        if (maxHeap.size() > minHeap.size() + 1) {
            minHeap.push(maxHeap.top());
            maxHeap.pop();
        } else if (minHeap.size() > maxHeap.size()) {
            maxHeap.push(minHeap.top());
            minHeap.pop();
        }
    }

    double findMedian() {
        if (maxHeap.size() == minHeap.size()) {
            return (maxHeap.top() + minHeap.top()) / 2.0;
        }
    }
}

```

```

    }
    return maxHeap.top(); // maxHeap has the extra element
}
};

// =====
// 5. K Closest Points to Origin
// =====
vector<vector<int>> kClosest(vector<vector<int>>& points, int k) {
    // Max-heap of (distance, point)
    auto cmp = [](const pair<int,int>& a, const pair<int,int>& b) {
        return a.first < b.first; // max-heap by distance
    };
    priority_queue<pair<int,int>, vector<pair<int,int>>, decltype(cmp)>
maxHeap(cmp);

    for (int i = 0; i < points.size(); i++) {
        int dist = points[i][0] * points[i][0] + points[i][1] * points[i][1];
        maxHeap.push({dist, i});
        if (maxHeap.size() > k) {
            maxHeap.pop(); // remove farthest
        }
    }

    vector<vector<int>> result;
    while (!maxHeap.empty()) {
        result.push_back(points[maxHeap.top().second]);
        maxHeap.pop();
    }
    return result;
}

// Example usage
int main() {
    // Kth Largest
    vector<int> nums1 = {3, 2, 1, 5, 6, 4};
    cout << "Kth Largest (k=3): " << findKthLargest(nums1, 3) << "\n"; // 4

    // Top K Frequent
    vector<int> nums2 = {1, 1, 1, 2, 2, 3};
    auto freq = topKFrequent(nums2, 2);
    cout << "Top K Frequent: ";
    for (int x : freq) cout << x << " ";
    cout << "\n"; // 1 2

    // Median Finder
    MedianFinder mf;
    mf.addNum(1); mf.addNum(2);
    cout << "Median after [1,2]: " << mf.findMedian() << "\n"; // 1.5
    mf.addNum(3);
    cout << "Median after [1,2,3]: " << mf.findMedian() << "\n"; // 2

    // K Closest Points
    vector<vector<int>> points = {{1,3}, {-2,2}, {5,8}, {0,1}};

```

```

    auto closest = kClosest(points, 2);
    cout << "K Closest points:\n";
    for (auto& p : closest) {
        cout << "    (" << p[0] << ", " << p[1] << ")\n";
    }

    return 0;
}

```

9. Python Implementation

```

from typing import List
import heapq
from collections import Counter

def find_kth_largest(nums: List[int], k: int) -> int:
    min_heap = []
    for num in nums:
        heapq.heappush(min_heap, num)
        if len(min_heap) > k:
            heapq.heappop(min_heap)
    return min_heap[0]

def top_k_frequent(nums: List[int], k: int) -> List[int]:
    freq = Counter(nums)
    # Min-heap of (frequency, value)
    heap = []
    for val, count in freq.items():
        heapq.heappush(heap, (count, val))
        if len(heap) > k:
            heapq.heappop(heap)
    return [val for _, val in heapq.nlargest(k, heap)]

def merge_k_sorted(lists: List[List[int]]) -> List[int]:
    import heapq
    heap = []
    for i, lst in enumerate(lists):
        if lst:
            heapq.heappush(heap, (lst[0], i, 0))

    result = []
    while heap:
        val, list_idx, elem_idx = heapq.heappop(heap)
        result.append(val)
        if elem_idx + 1 < len(lists[list_idx]):
            next_val = lists[list_idx][elem_idx + 1]
            heapq.heappush(heap, (next_val, list_idx, elem_idx + 1))
    return result

class MedianFinder:
    def __init__(self):

```

```

        self.max_heap = [] # left half (negative for max-heap in Python)
        self.min_heap = [] # right half

def add_num(self, num: int) -> None:
    if not self.max_heap or num <= -self.max_heap[0]:
        heapq.heappush(self.max_heap, -num)
    else:
        heapq.heappush(self.min_heap, num)

    # Balance
    if len(self.max_heap) > len(self.min_heap) + 1:
        val = -heapq.heappop(self.max_heap)
        heapq.heappush(self.min_heap, val)
    elif len(self.min_heap) > len(self.max_heap):
        val = heapq.heappop(self.min_heap)
        heapq.heappush(self.max_heap, -val)

def find_median(self) -> float:
    if len(self.max_heap) == len(self.min_heap):
        return (-self.max_heap[0] + self.min_heap[0]) / 2.0
    return -self.max_heap[0]

def k_closest(points: List[List[int]], k: int) -> List[List[int]]:
    # Max-heap by distance (use negative for max-heap)
    heap = []
    for i, (x, y) in enumerate(points):
        dist = x * x + y * y
        heapq.heappush(heap, (-dist, i))
        if len(heap) > k:
            heapq.heappop(heap)
    return [points[i] for _, i in heap]

# Example usage
if __name__ == "__main__":
    print(find_kth_largest([3, 2, 1, 5, 6, 4], 3)) # 4
    print(top_k_frequent([1, 1, 1, 2, 2, 3], 2)) # [1, 2]
    print(merge_k_sorted([[1, 4, 5], [1, 3, 4], [2, 6]])) # [1,1,2,3,4,4,5,6]

    mf = MedianFinder()
    mf.add_num(1); mf.add_num(2)
    print(mf.find_median()) # 1.5
    mf.add_num(3)
    print(mf.find_median()) # 2

    print(k_closest([[1,3], [-2,2], [5,8], [0,1]], 2)) # [[-2,2], [0,1]]

```

10. Code Explanation

Kth Largest Element:

- Min-heap of size K.

- For each element: push, then if size > K, pop the smallest.
- The heap always contains the K largest elements seen so far.
- `top()` is the smallest among them = Kth largest overall.

Top K Frequent Elements:

- First pass: count frequencies using hash map.
- Second pass: maintain min-heap of size K based on frequency.
- At the end, heap contains K most frequent elements.

Median Finder (Two Heaps):

- `maxHeap` stores the smaller half, `minHeap` stores the larger half.
- Invariant: `maxHeap.size()` is either equal to `minHeap.size()` or exactly 1 more.
- On insertion: decide which heap to put it in, then rebalance.
- Median: average of tops if equal size, top of `maxHeap` otherwise.

K Closest Points:

- Max-heap of size K (we want to keep the K smallest distances).
- For each point: compute squared distance, push, if size > K pop farthest.
- At the end, heap contains K closest points.

11. Complexity Analysis

Algorithm	Time	Space	Notes
Kth Largest (single pass)	$O(n \log K)$	$O(K)$	Heap of size K
Top K Frequent	$O(n + n \log K)$	$O(n + K)$	Freq map + heap
Merge K Sorted (total N)	$O(N \log K)$	$O(K)$	Heap of K elements
Median from Stream	$O(\log n)$ per add	$O(n)$	Two heaps
K Closest Points	$O(n \log K)$	$O(K)$	Heap of size K
Heapify	$O(n)$	$O(1)$	Build heap from array

Comparison with sorting:

- Sorting: $O(n \log n)$ always.
- Heap of size K: $O(n \log K)$. For $K \ll n$, this is much better.

12. Common Patterns

Pattern 1: Kth Largest/Smallest

Identify: "Kth largest element in array." **Approach:** Min-heap of size K (for largest) or max-heap of size K (for smallest). **Examples:** LeetCode 215 — Kth Largest Element, LeetCode 703 — Kth Largest in Stream.

Pattern 2: Top K Frequent

Identify: "K most frequent elements." **Approach:** Frequency map + min-heap of size K. **Example:** LeetCode 347 — Top K Frequent Elements.

Pattern 3: K Closest

Identify: "K closest points to origin." **Approach:** Max-heap of size K by distance. **Example:** LeetCode 973 — K Closest Points to Origin.

Pattern 4: Median Maintenance

Identify: "Median from data stream." **Approach:** Two heaps (max-heap for left, min-heap for right). **Example:** LeetCode 295 — Find Median from Data Stream.

Pattern 5: Merge K Sorted

Identify: "Merge K sorted lists/arrays." **Approach:** Min-heap of first elements, keep popping and pushing next. **Examples:** LeetCode 23 — Merge K Sorted Lists.

Pattern 6: Task Scheduler

Identify: "Schedule tasks with cooldown." **Approach:** Max-heap of task frequencies + queue for cooldown. **Example:** LeetCode 621 — Task Scheduler.

13. Common Mistakes

- **Using max-heap when min-heap is needed:** For Top K largest, use min-heap. Counter-intuitive but correct.
- **Wrong comparator:** In C++, `greater<T>` gives min-heap. `less<T>` (default) gives max-heap.
- **Not using `long long` for distances:** Squared coordinates can overflow.
- **Forgetting to handle empty input.**
- **Not checking heap size before accessing `top()`.**
- **Modifying heap elements after insertion:** Heaps don't rebalance automatically. Use `push/pop` only.
- **Using heap when sorting is simpler:** For small K, sorting is fine. For K close to n, sorting is $O(n \log n)$ which is better than heap $O(n \log K)$ when $K \approx n$.

14. Edge Cases

Case	Expected Behavior
Empty array	Return empty / 0 / handle specially
$K = 1$	Return min or max element
$K = n$	Return all elements (sorted or unsorted)
$K > n$	Return all elements
All same values	Kth largest = that value
Negative numbers	Works fine with comparison
Large K	Heap size may be large, $O(n \log n)$ effectively

Case	Expected Behavior
Single element	Kth largest = that element

15. Variations

Heap with Lazy Deletion

When you need to remove arbitrary elements from a heap, use a "lazy deletion" technique: mark elements as deleted, and skip them when they reach the top.

d-ary Heap

A heap where each node has d children (instead of 2). Can be faster for certain operations due to better cache behavior.

Fibonacci Heap

Theoretically better for decrease-key operations ($O(1)$ amortized). Used in advanced Dijkstra's implementations. Rarely used in CP due to complexity.

Bucket Sort Alternative for Top K

When frequency values are bounded (e.g., $\text{max frequency} \leq n$), we can use bucket sort instead of heap for $O(n)$ time.

16. Related Algorithms/Data Structures

Data Structure	Connection
Balanced BST	Can do everything heap does plus search/delete arbitrary elements. $O(\log n)$ for all operations.
Sorting	Sort + index = Kth element. Simpler but $O(n \log n)$.
Quickselect	$O(n)$ average, $O(n^2)$ worst case for Kth element. No extra space.
Bucket Sort	$O(n)$ for Top K when frequencies are bounded.
Monotonic Queue	For sliding window max/min, not top K.

Decision guide:

- Top K largest → Heap (size K) or Quickselect.
- Kth largest → Heap (size K) or Quickselect (better average).
- Median from stream → Two heaps.
- Merge K sorted → Heap.
- Need $O(1)$ access to max and min → Two heaps or balanced BST.

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Kth Largest Element in a Stream	LeetCode 703	Kth largest, stream	Easy
Last Stone Weight	LeetCode 1046	Max-heap simulation	Easy

Medium

Problem	Platform	Pattern	Difficulty
Kth Largest Element in an Array	LeetCode 215	Kth largest	Medium
Top K Frequent Elements	LeetCode 347	Top K frequent	Medium
K Closest Points to Origin	LeetCode 973	K closest	Medium

Hard

Problem	Platform	Pattern	Difficulty
Find Median from Data Stream	LeetCode 295	Two heaps	Hard
Merge K Sorted Lists	LeetCode 23	Merge K lists	Hard
Sliding Window Median	LeetCode 480	Two heaps + sliding window	Hard

18. Interview Explanation

"A heap is a tree-based data structure that gives us $O(1)$ access to the minimum or maximum element and $O(\log n)$ insertion and deletion. For Top K problems, I use a min-heap of size K to find the K largest elements — the heap always contains the K largest elements seen so far, and the top is the Kth largest. This is $O(n \log K)$ time and $O(K)$ space. For the median from a data stream, I use two heaps: a max-heap for the left half and a min-heap for the right half, maintaining balanced sizes. The key insight for heap-based Top K is that we don't need to sort everything — we only need to track the K elements that matter."

19. Revision Notes

- **Min-heap (C++):** `priority_queue<int, vector<int>, greater<int>>`
- **Max-heap (C++):** `priority_queue<int>` (default)
- **Top K largest:** Min-heap of size K. `top()` = Kth largest.
- **Top K smallest:** Max-heap of size K. `top()` = Kth smallest.
- **Two heaps:** Left = max-heap, Right = min-heap. Median from tops.
- **Complexity:** Push/pop = $O(\log K)$, Top = $O(1)$.
- **Common trap:** Wrong heap type for Top K. Kth largest needs min-heap.
- **Python:** `heapq` is min-heap only. Use negative for max-heap.

20. Final Cheat Sheet

HEAP / TOP K – CHEAT SHEET

WHEN TO USE:	Top K, Kth element, median, merge K sorted, priority queue
TOP K LARGEST:	Min-heap of size K
TOP K SMALLEST:	Max-heap of size K
KTH LARGEST:	min-heap, return top()
MEDIAN:	Two heaps (max + min)
COMPLEXITY:	Push/pop $O(\log K)$, Top $O(1)$ Overall $O(n \log K)$
C++ MIN-HEAP:	<code>priority_queue<..., greater<>></code>
C++ MAX-HEAP:	<code>priority_queue<T></code>
PYTHON MIN:	<code>heapq</code> (default)
PYTHON MAX:	<code>heapq</code> with negative values
TRAP:	Wrong heap type, Kth largest needs min-heap (counter-intuitive)
RELATED:	Quickselect $O(n)$, sort $O(n \log n)$

10. BFS (BREADTH-FIRST SEARCH)

1. Overview

Breadth-First Search (BFS) is a graph traversal algorithm that explores all vertices at the **current depth level** before moving to the next level. Starting from a source node, it visits all neighbors (distance 1), then all neighbors of neighbors (distance 2), and so on. BFS finds the **shortest path** in unweighted graphs.

2. Intuition

Core idea: BFS explores in layers, like ripples spreading from a stone dropped in water. The first time a node is discovered, it's via the shortest path from the source.

Analogy: You're trying to find the shortest route from your house to a friend's house in a city where all roads have equal length. You call all your neighbors (distance 1) and ask if they know your friend. If not, they call their neighbors (distance 2), and so on. The first person who knows your friend gives you the shortest path.

Step-by-step reasoning:

1. Start at the source node. It's at distance 0.
2. Visit all nodes at distance 1 (immediate neighbors).
3. Visit all nodes at distance 2 (neighbors of neighbors not yet visited).
4. Continue until all reachable nodes are visited.

Why BFS finds shortest paths: Since we explore level by level, the first time we encounter a node is necessarily via the shortest path (in terms of number of edges). Any later encounter would have to come from a longer path.

3. When to Use It

- **Shortest path in unweighted graph** (minimum number of edges).
- **Minimum number of moves** in a grid/maze (shortest path in grid).
- **Level-order traversal** of a tree.
- **Finding connected components** in an undirected graph.
- **Checking if a graph is bipartite** (2-coloring).
- **Word ladder** (minimum transformations).
- **Rotting oranges** (multi-source BFS) / spread of infection.
- **Topological sort** (Kahn's algorithm — BFS-based).
- **Serialize/deserialize a tree** (level order).

Common trigger phrases:

- "shortest path" (unweighted)
- "minimum number of steps"
- "minimum moves"
- "level order"
- "nearest"
- "distance from source"
- "rotten oranges"
- "word ladder"
- "bipartite"

4. When Not to Use It

- **Weighted graph with non-uniform edge weights:** Use Dijkstra's algorithm.
- **Need to find any path (not necessarily shortest):** DFS is simpler and uses less memory.
- **Very deep graph with large branching factor:** BFS can use $O(\text{branching}^{\text{depth}})$ memory. DFS uses $O(\text{depth})$.
- **Graph is a tree and you need to visit all nodes:** Both BFS and DFS work. BFS finds shortest path, DFS uses less memory.
- **Single source, all destinations:** BFS is fine, but if graph is a DAG, DP might be faster.

5. Core Concepts

5.1 Queue-Based Traversal

BFS uses a queue. The node at the front of the queue is processed first, and its unvisited neighbors are added to the back.

```
queue<int> q;
q.push(source);
visited[source] = true;

while (!q.empty()) {
    int u = q.front(); q.pop();
    // process u
    for (int v : adj[u]) {
```

```

        if (!visited[v]) {
            visited[v] = true;
            q.push(v);
        }
    }
}

```

5.2 Level Tracking

To track distances (levels), we can:

- Store distance in a separate array.
- Process level by level using a loop.

```

// Level-by-level processing
while (!q.empty()) {
    int levelSize = q.size();
    for (int i = 0; i < levelSize; i++) {
        int u = q.front(); q.pop();
        // process u
        for (int v : adj[u]) {
            if (!visited[v]) {
                visited[v] = true;
                q.push(v);
            }
        }
    }
    level++; // one level done
}

```

5.3 Visited Array

Essential to avoid revisiting nodes, which can cause infinite loops (in graphs with cycles). For trees, visited array is optional (no cycles).

5.4 BFS in Grid

Grids are implicit graphs. Each cell is a node, adjacent cells (up, down, left, right) are neighbors.

```

int dr[] = {-1, 1, 0, 0};
int dc[] = {0, 0, -1, 1};

queue<pair<int,int>> q;
q.push({sr, sc});
dist[sr][sc] = 0;

while (!q.empty()) {
    auto [r, c] = q.front(); q.pop();
    for (int d = 0; d < 4; d++) {

```

```

        int nr = r + dr[d], nc = c + dc[d];
        if (isValid(nr, nc) && dist[nr][nc] == -1) {
            dist[nr][nc] = dist[r][c] + 1;
            q.push({nr, nc});
        }
    }
}

```

5.5 Multi-Source BFS

Start BFS from multiple sources simultaneously. Used for problems like "rotting oranges" or "distance to nearest 0 in a matrix."

6. Step-by-Step Algorithm

BFS for Shortest Path (Unweighted Graph):

1. Create adjacency list `adj` of the graph.
2. Create `visited` array (bool), `dist` array (int, initialized to -1).
3. Create queue `q`.
4. Push source `s` to queue. Mark `visited[s] = true, dist[s] = 0`.
5. While queue is not empty:
 - `u = q.front(), q.pop()`.
 - For each neighbor `v` of `u`:
 - If `visited[v]` is false:
 - `visited[v] = true, dist[v] = dist[u] + 1`.
 - Push `v` to queue.
6. `dist` now contains shortest distances from source.

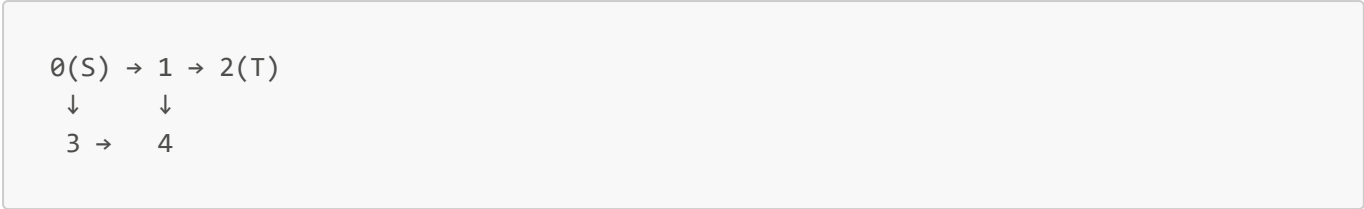
BFS for Shortest Path in Grid:

1. Create `dist` array initialized to -1.
2. Create queue of (row, col) pairs.
3. Push source cell. Set `dist[srcR][srcC] = 0`.
4. Define direction arrays: `dr = {-1, 1, 0, 0}, dc = {0, 0, -1, 1}`.
5. While queue is not empty:
 - Pop `(r, c)`.
 - For each direction `d`:
 - `nr = r + dr[d], nc = c + dc[d]`.
 - If `(nr, nc)` is within bounds and not blocked and `dist[nr][nc] == -1`:
 - `dist[nr][nc] = dist[r][c] + 1`.
 - Push `(nr, nc)`.
6. Return `dist[targetR][targetC]`.

7. Dry Run

Problem: Shortest path in unweighted graph from S to T.

Graph:



Adjacency list:

```
0: [1, 3]
1: [0, 2, 4]
2: [1]
3: [0, 4]
4: [1, 3]
```

BFS from S (node 0):

Step	Queue (front → back)	Process	Distances
Init	[0]	—	dist[0]=0, rest=-1
1	[]	Process 0, add 1, 3	dist[1]=1, dist[3]=1
	[1, 3]		
2	[3]	Process 1, add 2, 4	dist[2]=2, dist[4]=2
	[3, 2, 4]		
3	[2, 4]	Process 3, add none	—
4	[4]	Process 2, add none	—
5	[]	Process 4, add none	—

Result: dist[2] = 2 (shortest path S→1→2 or S→3→4→1→2? Actually S→1→2 = 2 edges)

Problem: Shortest path in grid (0 = open, 1 = blocked)

```
Grid:
S 0 0
1 1 0
0 0 T
```

BFS from S(0,0) to T(2,2):

Step	Queue	Process	Distances updated
Init	[(0,0)]	—	dist[0][0]=0
1	[]	(0,0) → add (0,1)	dist[0][1]=1

Step	Queue	Process	Distances updated
	[(0,1)]		
2	[]	(0,1) → add (0,2), (1,1) blocked	dist[0][2]=2
	[(0,2)]		
3	[]	(0,2) → add (1,2)	dist[1][2]=3
	[(1,2)]		
4	[]	(1,2) → add (2,2) T!	dist[2][2]=4

Result: Shortest path length = 4

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// =====
// 1. BFS on Graph (Shortest Path)
// =====
vector<int> bfsShortestPath(const vector<vector<int>>& adj, int source) {
    int n = adj.size();
    vector<int> dist(n, -1);
    queue<int> q;

    dist[source] = 0;
    q.push(source);

    while (!q.empty()) {
        int u = q.front();
        q.pop();

        for (int v : adj[u]) {
            if (dist[v] == -1) {
                dist[v] = dist[u] + 1;
                q.push(v);
            }
        }
    }

    return dist;
}

// =====
// 2. BFS on Grid (Shortest Path)
// =====
int bfsGrid(const vector<vector<int>>& grid, pair<int,int> src, pair<int,int>
dest) {
    int rows = grid.size(), cols = grid[0].size();
    vector<vector<int>> dist(rows, vector<int>(cols, -1));
```

```

queue<pair<int,int>> q;

int dr[] = {-1, 1, 0, 0};
int dc[] = {0, 0, -1, 1};

auto isValid = [&](int r, int c) {
    return r >= 0 && r < rows && c >= 0 && c < cols && grid[r][c] == 0;
};

dist[src.first][src.second] = 0;
q.push(src);

while (!q.empty()) {
    auto [r, c] = q.front();
    q.pop();

    if (r == dest.first && c == dest.second) {
        return dist[r][c];
    }

    for (int d = 0; d < 4; d++) {
        int nr = r + dr[d], nc = c + dc[d];
        if (isValid(nr, nc) && dist[nr][nc] == -1) {
            dist[nr][nc] = dist[r][c] + 1;
            q.push({nr, nc});
        }
    }
}
return -1; // unreachable
}

// =====
// 3. Multi-Source BFS (Rotten Oranges)
// =====
int orangesRotting(vector<vector<int>>& grid) {
    int rows = grid.size(), cols = grid[0].size();
    queue<pair<int,int>> q;
    int freshCount = 0;

    int dr[] = {-1, 1, 0, 0};
    int dc[] = {0, 0, -1, 1};

    // Add all rotten oranges to queue
    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            if (grid[r][c] == 2) q.push({r, c});
            else if (grid[r][c] == 1) freshCount++;
        }
    }

    if (freshCount == 0) return 0; // no fresh oranges

    int minutes = 0;

```

```

while (!q.empty()) {
    int levelSize = q.size();
    bool rotted = false;

    for (int i = 0; i < levelSize; i++) {
        auto [r, c] = q.front();
        q.pop();

        for (int d = 0; d < 4; d++) {
            int nr = r + dr[d], nc = c + dc[d];
            if (nr >= 0 && nr < rows && nc >= 0 && nc < cols && grid[nr][nc]
== 1) {
                grid[nr][nc] = 2;
                freshCount--;
                q.push({nr, nc});
                rotted = true;
            }
        }
    }
    if (rotted) minutes++;
}

return freshCount == 0 ? minutes : -1;
}

// =====
// 4. BFS for Bipartite Graph Check
// =====
bool isBipartite(const vector<vector<int>>& graph) {
    int n = graph.size();
    vector<int> color(n, -1); // -1 = uncolored, 0 and 1 are colors

    for (int start = 0; start < n; start++) {
        if (color[start] != -1) continue;

        queue<int> q;
        color[start] = 0;
        q.push(start);

        while (!q.empty()) {
            int u = q.front();
            q.pop();

            for (int v : graph[u]) {
                if (color[v] == -1) {
                    color[v] = 1 - color[u]; // opposite color
                    q.push(v);
                } else if (color[v] == color[u]) {
                    return false; // same color adjacent → not bipartite
                }
            }
        }
    }

    return true;
}

```

```

}

// Example usage
int main() {
    // Graph BFS
    vector<vector<int>> adj = {
        {1, 3},    // 0
        {0, 2, 4}, // 1
        {1},       // 2
        {0, 4},    // 3
        {1, 3}     // 4
    };
    auto dist = bfsShortestPath(adj, 0);
    cout << "Distances from 0: ";
    for (int d : dist) cout << d << " ";
    cout << "\n"; // 0 1 2 1 2

    // Grid BFS
    vector<vector<int>> grid = {
        {0, 0, 0},
        {1, 1, 0},
        {0, 0, 0}
    };
    cout << "Shortest path: " << bfsGrid(grid, {0,0}, {2,2}) << "\n"; // 4

    // Rotting Oranges
    vector<vector<int>> oranges = {
        {2, 1, 1},
        {1, 1, 0},
        {0, 1, 1}
    };
    cout << "Minutes to rot: " << orangesRotting(oranges) << "\n"; // 4

    // Bipartite Check
    vector<vector<int>> graph = {
        {1, 3}, {0, 2}, {1, 3}, {0, 2}
    };
    cout << "Is bipartite: " << isBipartite(graph) << "\n"; // 1 (true)

    return 0;
}

```

9. Python Implementation

```

from typing import List
from collections import deque

def bfs_shortest_path(adj: List[List[int]], source: int) -> List[int]:
    n = len(adj)
    dist = [-1] * n
    q = deque()

```

```

dist[source] = 0
q.append(source)

while q:
    u = q.popleft()
    for v in adj[u]:
        if dist[v] == -1:
            dist[v] = dist[u] + 1
            q.append(v)
return dist

def bfs_grid(grid: List[List[int]], src: tuple, dest: tuple) -> int:
    rows, cols = len(grid), len(grid[0])
    dist = [[-1] * cols for _ in range(rows)]
    q = deque()

    dr = [-1, 1, 0, 0]
    dc = [0, 0, -1, 1]

    def is_valid(r, c):
        return 0 <= r < rows and 0 <= c < cols and grid[r][c] == 0

    dist[src[0]][src[1]] = 0
    q.append(src)

    while q:
        r, c = q.popleft()
        if (r, c) == dest:
            return dist[r][c]

        for d in range(4):
            nr, nc = r + dr[d], c + dc[d]
            if is_valid(nr, nc) and dist[nr][nc] == -1:
                dist[nr][nc] = dist[r][c] + 1
                q.append((nr, nc))
    return -1

def oranges_rotting(grid: List[List[int]]) -> int:
    rows, cols = len(grid), len(grid[0])
    q = deque()
    fresh_count = 0

    dr = [-1, 1, 0, 0]
    dc = [0, 0, -1, 1]

    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 2:
                q.append((r, c))
            elif grid[r][c] == 1:
                fresh_count += 1

    if fresh_count == 0:

```

```

        return 0

minutes = 0

while q:
    level_size = len(q)
    rotted = False

    for _ in range(level_size):
        r, c = q.popleft()
        for d in range(4):
            nr, nc = r + dr[d], c + dc[d]
            if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == 1:
                grid[nr][nc] = 2
                fresh_count -= 1
                q.append((nr, nc))
                rotted = True

    if rotted:
        minutes += 1

return minutes if fresh_count == 0 else -1

def is_bipartite(graph: List[List[int]]) -> bool:
    n = len(graph)
    color = [-1] * n

    for start in range(n):
        if color[start] != -1:
            continue

        q = deque()
        color[start] = 0
        q.append(start)

        while q:
            u = q.popleft()
            for v in graph[u]:
                if color[v] == -1:
                    color[v] = 1 - color[u]
                    q.append(v)
                elif color[v] == color[u]:
                    return False

    return True

# Example usage
if __name__ == "__main__":
    adj = [[1, 3], [0, 2, 4], [1], [0, 4], [1, 3]]
    print(bfs_shortest_path(adj, 0)) # [0, 1, 2, 1, 2]

    grid = [[0, 0, 0], [1, 1, 0], [0, 0, 0]]
    print(bfs_grid(grid, (0, 0), (2, 2))) # 4

```

```

oranges = [[2, 1, 1], [1, 1, 0], [0, 1, 1]]
print(oranges_rotting(oranges)) # 4

graph = [[1, 3], [0, 2], [1, 3], [0, 2]]
print(is_bipartite(graph)) # True

```

10. Code Explanation

BFS on Graph:

- `dist` array initialized to -1 (unvisited).
- `queue` stores nodes to process.
- Start with source: `dist[source] = 0`, push to queue.
- For each node `u`, iterate neighbors `v`. If `dist[v] == -1` (unvisited), set `dist[v] = dist[u] + 1` and push.
- BFS ensures first visit = shortest path because we process level by level.

BFS on Grid:

- Same as graph BFS but with direction arrays for 4-directional movement.
- `isValid` lambda checks bounds and whether cell is traversable (0).
- Early exit when destination is reached.

Multi-Source BFS (Rotting Oranges):

- Initialize queue with all rotten oranges (multi-source).
- Count fresh oranges initially.
- Process level by level (each level = 1 minute).
- For each level, rot adjacent fresh oranges. If any orange rotted this level, increment minutes.
- If all fresh oranges are rotten, return minutes. Otherwise, return -1 (some oranges can't rot).

Bipartite Check:

- Color vertices with 0 and 1. Adjacent vertices must have different colors.
- BFS from each uncolored vertex (handles disconnected graphs).
- If any neighbor has same color as current, graph is not bipartite.

11. Complexity Analysis

Algorithm	Time	Space	Notes
BFS on Graph	$O(V + E)$	$O(V)$	Queue + dist array
BFS on Grid	$O(R \times C)$	$O(R \times C)$	Grid cells
Multi-Source BFS	$O(R \times C)$	$O(R \times C)$	Same as grid
Bipartite Check	$O(V + E)$	$O(V)$	Color array
Level Order Tree	$O(n)$	$O(n)$	Queue size up to max width

BFS vs DFS memory:

- BFS: $O(\text{width})$ — worst case is the widest level.
- DFS: $O(\text{depth})$ — worst case is the deepest path.

12. Common Patterns

Pattern 1: Shortest Path in Unweighted Graph

Identify: "Minimum number of edges from A to B." **Approach:** Standard BFS with distance array. **Example:** LeetCode 433 — Minimum Genetic Mutation.

Pattern 2: Shortest Path in Grid

Identify: "Minimum steps in a grid/maze." **Approach:** BFS with direction arrays. **Examples:** LeetCode 1091 — Shortest Path in Binary Matrix, LeetCode 1293 — Shortest Path with Obstacles.

Pattern 3: Multi-Source BFS

Identify: "Distance to nearest 0/1", "rotting oranges", "spread of infection." **Approach:** Start BFS from all sources simultaneously. **Examples:** LeetCode 994 — Rotting Oranges, LeetCode 542 — 01 Matrix.

Pattern 4: Level Order Traversal

Identify: "Binary tree level order." **Approach:** BFS, process level by level. **Example:** LeetCode 102 — Binary Tree Level Order Traversal.

Pattern 5: Bipartite Graph

Identify: "Can graph be colored with 2 colors?" **Approach:** BFS with alternating colors. **Example:** LeetCode 785 — Is Graph Bipartite?

Pattern 6: Word Ladder

Identify: "Transform word A to word B, changing one letter at a time." **Approach:** BFS on implicit graph (each word is a node, edge if one letter differs). **Example:** LeetCode 127 — Word Ladder.

13. Common Mistakes

- **Not marking visited on push:** Always mark visited when pushing to queue, not when popping. Otherwise, duplicates can enter the queue.
- **Infinite loop in graph with cycles:** Without visited array, BFS keeps looping.
- **Forgetting to handle disconnected graphs:** BFS from one source only covers the connected component. Loop over all nodes for full coverage.
- **Wrong direction arrays in grid:** Off-by-one or missing a direction.
- **Not checking bounds in grid:** Accessing out-of-bounds cells.
- **Using DFS for shortest path:** DFS finds a path, but not necessarily the shortest. Use BFS for unweighted shortest path.
- **Not resetting visited between test cases:** Always reset for each test case.

14. Edge Cases

Case	Expected Behavior
Empty graph ($V=0$)	Return empty
Single node, no edges	$\text{dist}[\text{source}] = 0$
Source = destination	$\text{dist} = 0$
Disconnected graph	$\text{dist} = -1$ for unreachable nodes
Graph with cycles	BFS handles correctly (visited prevents loops)
Grid with no path	Return -1
Grid with obstacles	Navigate around them
All oranges already rotten	Return 0
Some oranges unreachable	Return -1

15. Variations

0-1 BFS (Dijkstra with Edge Weights 0 or 1)

Use deque instead of queue. Push front for 0-weight edges, push back for 1-weight edges. **Example:** Shortest path in graph with 0/1 edges.

BFS with State (3D BFS)

When the state includes position + some additional info (keys, direction, health). **Example:** LeetCode 864 — Shortest Path to Get All Keys.

Bidirectional BFS

Search from both source and destination simultaneously. When the two frontiers meet, we have the shortest path. Cuts search space significantly. **Example:** LeetCode 127 — Word Ladder (bidirectional is faster).

BFS with Bitmask

Used for traveling salesman, shortest path with state (visited set encoded as bitmask).

16. Related Algorithms/Data Structures

Algorithm	Connection
DFS	Alternative traversal. BFS for shortest path, DFS for path existence, connectivity.
Dijkstra's	BFS for weighted graphs (non-negative weights).
0-1 BFS	BFS for graphs with edge weights 0 and 1.
Topological Sort (Kahn's)	BFS-based algorithm for DAG ordering.

Algorithm	Connection
Queue	Core data structure for BFS.

BFS vs DFS:

- BFS: Shortest path, level order, queue, $O(\text{width})$ memory.
- DFS: Path existence, topological sort, backtracking, recursion/stack, $O(\text{depth})$ memory.

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Binary Tree Level Order Traversal	LeetCode 102	BFS on tree	Easy
Symmetric Tree	LeetCode 101	BFS check	Easy

Medium

Problem	Platform	Pattern	Difficulty
Rotting Oranges	LeetCode 994	Multi-source BFS	Medium
Number of Islands	LeetCode 200	BFS/DFS for components	Medium
Word Ladder	LeetCode 127	BFS on implicit graph	Hard

Hard

Problem	Platform	Pattern	Difficulty
Shortest Path in a Grid with Obstacles Elimination	LeetCode 1293	BFS with state	Hard
Sliding Puzzle	LeetCode 773	BFS on state space	Hard
Bus Routes	LeetCode 815	BFS with route-level graph	Hard

18. Interview Explanation

"BFS is a graph traversal algorithm that explores nodes level by level using a queue. Starting from a source, I first visit all nodes at distance 1, then distance 2, and so on. The key property is that BFS finds the shortest path in unweighted graphs because the first time a node is discovered is via the shortest path. I use a queue, a visited array, and a distance array. The time complexity is $O(V + E)$. Important implementation details: I mark nodes as visited when they're pushed to the queue, not when they're popped, to avoid duplicates. I also use BFS for level-order tree traversal, bipartite graph checking, and multi-source problems like rotting oranges."

19. Revision Notes

- **Core idea:** Level-by-level traversal using a queue.
- **Shortest path:** BFS finds shortest path in unweighted graphs.
- **Queue + visited array:** Mark visited on push, not on pop.
- **Grid BFS:** Direction arrays `dr = [-1,1,0,0]`, `dc = [0,0,-1,1]`.
- **Multi-source BFS:** Push all sources initially, then BFS normally.
- **Level tracking:** `levelSize = q.size()`, loop `levelSize` times.
- **Complexity:** $O(V + E)$ time, $O(V)$ space.
- **Common trap:** Not marking visited on push, wrong direction arrays.

20. Final Cheat Sheet

BFS – CHEAT SHEET	
WHEN TO USE:	Shortest path (unweighted), level order, multi-source spread
DATA STRUCTURE:	Queue
KEY CODE:	<pre>q.push(src); dist[src]=0; while(!q.empty()) { u=q.pop(); for(v: adj[u]) if(dist[v]==-1) dist[v]=dist[u]+1; q.push(v); }</pre>
GRID BFS:	<code>dr=[-1,1,0,0]</code> , <code>dc=[0,0,-1,1]</code>
COMPLEXITY:	$O(V+E)$ time, $O(V)$ space
GRID:	$O(R \times C)$ time, $O(R \times C)$ space
MARK VISITED:	On push, not on pop
BIPARTITE:	Color 0/1, check neighbors differ
TRAP:	Not marking visited, no bounds check, DFS instead of BFS for SP
DFS vs BFS:	DFS for existence, BFS for SP
RELATED:	Dijkstra, 0-1 BFS, Topological

11. DFS (DEPTH-FIRST SEARCH)

1. Overview

Depth-First Search (DFS) is a graph traversal algorithm that explores as far as possible along each branch before backtracking. It uses a stack (or recursion) to remember which nodes to visit next. DFS is used for pathfinding, connectivity, topological sorting, and solving puzzles with constraints.

2. Intuition

Core idea: Go deep first. Explore one path completely before trying another. If you reach a dead end, backtrack to the last junction and try a different path.

Analogy: Exploring a maze. You pick a direction and walk until you hit a dead end or find the exit. At each junction, you remember the other paths you could have taken. When you reach a dead end, you backtrack to

the last junction and try the next path.

Step-by-step reasoning:

1. Start at the source node. Mark it as visited.
2. Pick an unvisited neighbor and go there (recursively or using a stack).
3. Repeat until you reach a node with no unvisited neighbors.
4. Backtrack to the previous node and try another unvisited neighbor.
5. Continue until all reachable nodes are visited.

Why it works: DFS systematically explores the entire graph by following each path to its end before backtracking. The recursion/stack ensures we always have a path back.

3. When to Use It

- **Path existence** between two nodes.
- **Connected components** count / detection.
- **Cycle detection** in directed/undirected graphs.
- **Topological sorting** (post-order DFS).
- **Tree traversals** (preorder, inorder, postorder).
- **Backtracking problems** (N-Queens, Sudoku, permutations).
- **Maze solving** with constraints.
- **Finding strongly connected components** (Kosaraju's/Tarjan's).
- **Articulation points / bridges** in a graph.
- **Flood fill** (connected cells with same color).

Common trigger phrases:

- "path exists"
- "connected components"
- "cycle detection"
- "topological order"
- "all paths"
- "flood fill"
- "maze"
- "backtracking"
- "island"
- "permutations"
- "combinations"

4. When Not to Use It

- **Shortest path in unweighted graph:** BFS is better. DFS finds a path, not necessarily the shortest.
- **Level-order traversal:** BFS is natural for level-order.
- **Very deep graph with risk of stack overflow:** Recursive DFS may overflow the call stack. Use iterative DFS with explicit stack.
- **Graph is too large for recursion depth:** Python has recursion limit (default 1000). Use iterative or increase limit.
- **Need minimum number of steps:** Use BFS (or Dijkstra for weighted).

5. Core Concepts

5.1 Recursive DFS (Most Common)

```
void dfs(int u, vector<vector<int>>& adj, vector<bool>& visited) {
    visited[u] = true;
    // process u
    for (int v : adj[u]) {
        if (!visited[v]) {
            dfs(v, adj, visited);
        }
    }
}
```

5.2 Iterative DFS (Using Explicit Stack)

```
void dfsIterative(int start, vector<vector<int>>& adj, vector<bool>& visited) {
    stack<int> st;
    st.push(start);

    while (!st.empty()) {
        int u = st.top();
        st.pop();

        if (visited[u]) continue;
        visited[u] = true;
        // process u

        for (int v : adj[u]) {
            if (!visited[v]) {
                st.push(v);
            }
        }
    }
}
```

5.3 Tree Traversal (Preorder, Inorder, Postorder)

Preorder: Process root, then left, then right. **Inorder:** Process left, then root, then right (BST gives sorted order). **Postorder:** Process left, then right, then root.

5.4 Backtracking

DFS with state modification and restoration. Used for constraint satisfaction.

```
void backtrack(vector<int>& path, vector<bool>& used, ...) {
    if (path.size() == n) {
```

```

        // process solution
        return;
    }
    for (int i = 0; i < n; i++) {
        if (!used[i]) {
            used[i] = true;
            path.push_back(nums[i]);
            backtrack(path, used, ...);
            path.pop_back();
            used[i] = false;
        }
    }
}

```

5.5 Visited States

- **Unvisited:** Not yet seen.
- **Visiting:** Currently in the recursion stack (for cycle detection).
- **Visited:** All descendants processed.

6. Step-by-Step Algorithm

DFS for Path Existence (from source to target):

1. Create adjacency list `adj` and `visited` array.
2. Call `dfs(source)`.
3. In `dfs(u)`:
 - If `u == target`: return true.
 - Mark `visited[u] = true`.
 - For each neighbor `v` of `u`:
 - If not `visited[v]`: recurse `dfs(v)`. If it returns true, propagate.
 - Return false.

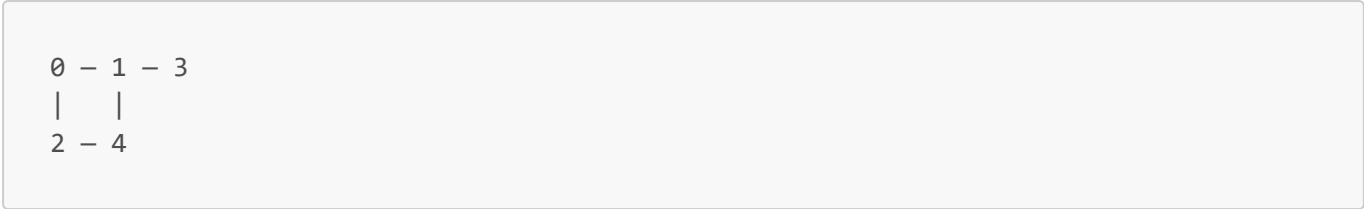
DFS for Count Connected Components:

1. Initialize `visited` array to false, `components = 0`.
2. For each node `u` from 0 to `n-1`:
 - If not `visited[u]`:
 - `components++`.
 - Run DFS from `u`, marking all reachable nodes as visited.
3. Return `components`.

7. Dry Run

Problem: DFS traversal from node 0 in graph.

Graph:



Adjacency list:

```
0: [1, 2]
1: [0, 3, 4]
2: [0, 4]
3: [1]
4: [1, 2]
```

Recursive DFS from 0 (assuming neighbors in order):

Call	Node	Visited	Action	Next
dfs(0)	0	{0}	Visit 0. Neighbors: 1, 2	Call dfs(1)
dfs(1)	1	{0,1}	Visit 1. Neighbors: 0(v), 3, 4	Call dfs(3)
dfs(3)	3	{0,1,3}	Visit 3. Neighbor: 1(v)	Return
(back)	1	—	Next neighbor: 4	Call dfs(4)
dfs(4)	4	{0,1,3,4}	Visit 4. Neighbors: 1(v), 2	Call dfs(2)
dfs(2)	2	{0,1,3,4,2}	Visit 2. Neighbors: 0(v), 4(v)	Return
(back)	4	—	All neighbors done	Return
(back)	1	—	All neighbors done	Return
(back)	0	—	Next neighbor: 2(v)	Done

DFS Traversal Order: 0 → 1 → 3 → 4 → 2

Problem: Cycle Detection in Directed Graph

Graph: 0 → 1 → 2 → 0 (cycle)

Step	Node	State	Action
1	0	Visiting (enter)	Mark 0 as visiting
2	1	Visiting (enter)	Called from 0
3	2	Visiting (enter)	Called from 1
4	0	Already Visiting!	Detected back edge → cycle!

8. C++ Implementation

```

#include <bits/stdc++.h>
using namespace std;

class DFS {
private:
    vector<vector<int>> adj;
    vector<bool> visited;
    int n;

public:
    DFS(int n) : n(n) {
        adj.resize(n);
        visited.resize(n, false);
    }

    void addEdge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u); // undirected
    }

    // =====
    // 1. Recursive DFS Traversal
    // =====
    void dfsRecursive(int u) {
        visited[u] = true;
        cout << u << " ";

        for (int v : adj[u]) {
            if (!visited[v]) {
                dfsRecursive(v);
            }
        }
    }

    // =====
    // 2. Iterative DFS Traversal
    // =====
    void dfsIterative(int start) {
        fill(visited.begin(), visited.end(), false);
        stack<int> st;
        st.push(start);

        while (!st.empty()) {
            int u = st.top();
            st.pop();

            if (visited[u]) continue;
            visited[u] = true;
            cout << u << " ";

            // Push in reverse order to maintain order
            for (auto it = adj[u].rbegin(); it != adj[u].rend(); ++it) {

```

```

        if (!visited[*it]) {
            st.push(*it);
        }
    }
}
cout << "\n";
}

// =====
// 3. Count Connected Components
// =====
int countComponents() {
    fill(visited.begin(), visited.end(), false);
    int components = 0;

    for (int u = 0; u < n; u++) {
        if (!visited[u]) {
            components++;
            dfsRecursive(u);
            cout << "\n";
        }
    }
    return components;
}

// =====
// 4. Cycle Detection in Directed Graph
// =====
bool hasCycleDirected() {
    vector<int> state(n, 0); // 0=unvisited, 1=visiting, 2=done

    function<bool(int)> dfsCycle = [&](int u) -> bool {
        state[u] = 1; // visiting

        for (int v : adj[u]) {
            if (state[v] == 1) return true; // back edge → cycle
            if (state[v] == 0 && dfsCycle(v)) return true;
        }

        state[u] = 2; // done
        return false;
    };

    for (int u = 0; u < n; u++) {
        if (state[u] == 0 && dfsCycle(u)) return true;
    }
    return false;
}

// =====
// 5. Topological Sort (Post-order DFS)
// =====
vector<int> topologicalSort() {
    fill(visited.begin(), visited.end(), false);

```

```

vector<int> order;

function<void(int)> dfsTopo = [&](int u) {
    visited[u] = true;
    for (int v : adj[u]) {
        if (!visited[v]) {
            dfsTopo(v);
        }
    }
    order.push_back(u); // post-order
};

for (int u = 0; u < n; u++) {
    if (!visited[u]) dfsTopo(u);
}

reverse(order.begin(), order.end());
return order;
}

// =====
// 6. Flood Fill (on grid)
// =====
static void floodFill(vector<vector<int>>& image, int sr, int sc, int
newColor) {
    int oldColor = image[sr][sc];
    if (oldColor == newColor) return;

    int rows = image.size(), cols = image[0].size();
    int dr[] = {-1, 1, 0, 0};
    int dc[] = {0, 0, -1, 1};

    function<void(int,int)> dfs = [&](int r, int c) {
        if (r < 0 || r >= rows || c < 0 || c >= cols) return;
        if (image[r][c] != oldColor) return;

        image[r][c] = newColor;
        for (int d = 0; d < 4; d++) {
            dfs(r + dr[d], c + dc[d]);
        }
    };

    dfs(sr, sc);
}

};

// Example usage
int main() {
    DFS g(5);
    g.addEdge(0, 1);
    g.addEdge(0, 2);
    g.addEdge(1, 3);
    g.addEdge(1, 4);
    g.addEdge(2, 4);

```

```

cout << "DFS Recursive: ";
g.dfsRecursive(0); // 0 1 3 4 2
cout << "\n";

cout << "Components: " << g.countComponents() << "\n"; // 1

// Cycle detection
DFS dg(3);
dg.addEdge(0, 1);
dg.addEdge(1, 2);
// dg.addEdge(2, 0); // uncomment for cycle
cout << "Has cycle: " << dg.hasCycleDirected() << "\n"; // 0 (false)

// Flood Fill
vector<vector<int>> image = {
    {1, 1, 1},
    {1, 1, 0},
    {1, 0, 1}
};
DFS::floodFill(image, 1, 1, 2);
cout << "Flood Fill:\n";
for (auto& row : image) {
    for (int x : row) cout << x << " ";
    cout << "\n";
}

return 0;
}

```

9. Python Implementation

```

from typing import List
from collections import defaultdict

class DFS:
    def __init__(self, n: int):
        self.n = n
        self.adj = [[] for _ in range(n)]

    def add_edge(self, u: int, v: int):
        self.adj[u].append(v)
        self.adj[v].append(u) # undirected

    def dfs_recursive(self, u: int, visited: List[bool]):
        visited[u] = True
        print(u, end=" ")
        for v in self.adj[u]:
            if not visited[v]:
                self.dfs_recursive(v, visited)

```

```

def count_components(self) -> int:
    visited = [False] * self.n
    components = 0
    for u in range(self.n):
        if not visited[u]:
            components += 1
            self.dfs_recursive(u, visited)
            print()
    return components

def has_cycle_directed(self) -> bool:
    state = [0] * self.n # 0=unvisited, 1=visiting, 2=done

    def dfs_cycle(u: int) -> bool:
        state[u] = 1
        for v in self.adj[u]:
            if state[v] == 1:
                return True
            if state[v] == 0 and dfs_cycle(v):
                return True
        state[u] = 2
        return False

    for u in range(self.n):
        if state[u] == 0:
            if dfs_cycle(u):
                return True
    return False

def topological_sort(self) -> List[int]:
    visited = [False] * self.n
    order = []

    def dfs_topo(u: int):
        visited[u] = True
        for v in self.adj[u]:
            if not visited[v]:
                dfs_topo(v)
        order.append(u) # post-order

    for u in range(self.n):
        if not visited[u]:
            dfs_topo(u)

    return order[::-1] # reverse

    @staticmethod
    def flood_fill(image: List[List[int]], sr: int, sc: int, new_color: int):
        old_color = image[sr][sc]
        if old_color == new_color:
            return

        rows, cols = len(image), len(image[0])
        dr = [-1, 1, 0, 0]

```

```

    dc = [0, 0, -1, 1]

    def dfs(r: int, c: int):
        if r < 0 or r >= rows or c < 0 or c >= cols:
            return
        if image[r][c] != old_color:
            return
        image[r][c] = new_color
        for d in range(4):
            dfs(r + dr[d], c + dc[d])

    dfs(sr, sc)

# Example usage
if __name__ == "__main__":
    g = DFS(5)
    g.add_edge(0, 1)
    g.add_edge(0, 2)
    g.add_edge(1, 3)
    g.add_edge(1, 4)
    g.add_edge(2, 4)

    print("DFS Recursive:", end=" ")
    g.dfs_recursive(0, [False] * 5)  # 0 1 3 4 2
    print()

    print("Components:", g.count_components())  # 1

    dg = DFS(3)
    dg.add_edge(0, 1)
    dg.add_edge(1, 2)
    print("Has cycle:", dg.has_cycle_directed())  # False

    image = [[1, 1, 1], [1, 1, 0], [1, 0, 1]]
    DFS.flood_fill(image, 1, 1, 2)
    print("Flood Fill:", image)  # [[2,2,2],[2,2,0],[2,0,1]]

```

10. Code Explanation

Recursive DFS:

- Base case: mark `u` as visited, process it.
- Recursive case: for each unvisited neighbor, call `dfs(v)`.
- The recursion stack implicitly handles backtracking.

Iterative DFS:

- Uses explicit stack. Push `start`.
- Pop node, if already visited skip. Otherwise mark visited and process.
- Push unvisited neighbors.

- **Important:** Iterative DFS may visit nodes in different order than recursive DFS due to stack LIFO behavior.

Count Connected Components:

- Loop over all nodes. If unvisited, start a new DFS and increment component count.
- The DFS marks all nodes in that component as visited.

Cycle Detection (Directed):

- Three states: 0 = unvisited, 1 = visiting (in recursion stack), 2 = done.
- If we encounter a node with state 1 (visiting), we found a back edge → cycle.
- This is Tarjan's approach for cycle detection.

Topological Sort:

- Post-order DFS: after processing all descendants, add current node to order.
- Reverse the order to get the topological ordering.
- For DAGs only. If graph has a cycle, topological sort is not defined.

Flood Fill:

- Standard DFS on grid. If current cell matches old color, change to new color and recurse.
- Check bounds and color match before recursing.

11. Complexity Analysis

Algorithm	Time	Space	Notes
DFS Traversal	$O(V + E)$	$O(V)$	Recursion stack (depth)
Count Components	$O(V + E)$	$O(V)$	
Cycle Detection	$O(V + E)$	$O(V)$	State array
Topological Sort	$O(V + E)$	$O(V)$	Order array + stack
Flood Fill	$O(R \times C)$	$O(R \times C)$	Worst case stack depth
All Paths	$O(V!)$	$O(V)$	Can be exponential

Space consideration: Recursive DFS uses $O(\text{depth})$ stack space. For deep graphs, this can overflow. Use iterative DFS in such cases.

12. Common Patterns

Pattern 1: Connected Components / Islands

Identify: "Count number of islands", "connected components". **Approach:** DFS/BFS from each unvisited node.

Examples: LeetCode 200 — Number of Islands, LeetCode 547 — Number of Provinces.

Pattern 2: Cycle Detection

Identify: "Detect cycle in graph." **Approach:** DFS with parent tracking (undirected) or state array (directed).

Examples: LeetCode 207 — Course Schedule, LeetCode 802 — Find Eventual Safe States.

Pattern 3: Topological Sort

Identify: "Order of courses/tasks with prerequisites." **Approach:** Post-order DFS or Kahn's algorithm (BFS).

Example: LeetCode 210 — Course Schedule II.

Pattern 4: Path Finding

Identify: "All paths from source to target." **Approach:** DFS with backtracking. **Example:** LeetCode 797 — All Paths From Source to Target.

Pattern 5: Tree Traversals

Identify: "Preorder/inorder/postorder traversal." **Approach:** Recursive DFS with different processing order.

Examples: LeetCode 144 (Preorder), 94 (Inorder), 145 (Postorder).

Pattern 6: Flood Fill / DFS on Grid

Identify: "Fill connected region", "capture surrounded region." **Approach:** DFS on grid with boundary checks.

Examples: LeetCode 733 — Flood Fill, LeetCode 130 — Surrounded Regions.

Pattern 7: Strongly Connected Components

Identify: "Find SCCs" (Kosaraju's or Tarjan's algorithm). **Approach:** Two-pass DFS or single-pass DFS with low-link values. **Example:** GFG — Strongly Connected Components (Kosaraju's).

13. Common Mistakes

- **Stack overflow:** Recursive DFS on very deep graph (e.g., 10^5 nodes in a line). Use iterative DFS or increase recursion limit.
- **Not marking visited:** Leads to infinite loops in graphs with cycles.
- **Marking visited on pop (iterative):** Mark on push to avoid pushing the same node multiple times.
- **Wrong order in iterative DFS:** Pushing neighbors in the wrong order changes traversal order.
- **Not resetting visited between test cases.**
- **Incorrect cycle detection:** For undirected graphs, checking `parent != v` is needed. For directed, need three-state (unvisited/visiting/done).
- **Using DFS for shortest path:** DFS finds a path, not necessarily the shortest. Use BFS for unweighted graphs.

14. Edge Cases

Case	Expected Behavior
Empty graph	No nodes, nothing to traverse
Single node	DFS from that node = just that node
Disconnected graph	DFS from one node only covers its component

Case	Expected Behavior
Graph with cycles	Handled correctly with visited array
Tree (no cycles)	DFS works naturally
Linear chain (deep)	Recursive DFS may overflow stack
Complete graph	DFS visits all nodes in $O(V+E)$
Grid with no valid path	Flood fill doesn't change anything
Self-loop	Cycle detection should detect it

15. Variations

Iterative DFS with Precomputed Order

For problems where you need to process nodes in a specific order (e.g., finding articulation points), iterative DFS is harder to implement. Use recursive with explicit stack if needed.

DFS with Color (Three-State)

Used for cycle detection and topological sort:

- 0 = white (unvisited)
- 1 = gray (in recursion stack)
- 2 = black (processed)

DFS with Timestamps (Discovery and Finish Time)

Used in Tarjan's algorithm for SCCs and articulation points.

DFS for Generating All Paths

Instead of just finding one path, DFS can generate all paths from source to target. This is exponential in worst case.

16. Related Algorithms/Data Structures

Algorithm	Connection
BFS	Alternative traversal. DFS for depth, BFS for breadth.
Backtracking	DFS with state modification/restoration.
Topological Sort	Post-order DFS or Kahn's algorithm.
Kosaraju's Algorithm	Two DFS passes for SCC.
Tarjan's Algorithm	Single DFS for SCC, articulation points, bridges.
Union-Find	Alternative for connectivity queries (dynamic).

BFS vs DFS recap:

- BFS: Shortest path, level order, queue, $O(\text{width})$ memory.
- DFS: Connectivity, cycle detection, topological sort, backtracking, recursion/stack, $O(\text{depth})$ memory.

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Flood Fill	LeetCode 733	Grid DFS	Easy
Number of Islands	LeetCode 200	Connected components	Medium

Medium

Problem	Platform	Pattern	Difficulty
Course Schedule	LeetCode 207	Cycle detection	Medium
All Paths From Source to Target	LeetCode 797	All paths DFS	Medium
Surrounded Regions	LeetCode 130	Boundary DFS	Medium

Hard

Problem	Platform	Pattern	Difficulty
Longest Increasing Path in a Matrix	LeetCode 329	DFS with memoization	Hard
Word Search II	LeetCode 212	Trie + DFS backtracking	Hard
Critical Connections in a Network	LeetCode 1192	Tarjan's bridges	Hard

18. Interview Explanation

"DFS is a graph traversal algorithm that explores as far as possible along each branch before backtracking. I usually implement it recursively. The key idea is: mark the current node as visited, process it, then recursively visit all unvisited neighbors. The recursion stack naturally handles backtracking. DFS is used for connectivity problems, cycle detection, topological sorting, and backtracking. For cycle detection in directed graphs, I use a three-state system: unvisited, visiting (in the current recursion stack), and done. The time complexity is $O(V + E)$. I'm careful about stack overflow in very deep graphs, where I'd use an iterative approach with an explicit stack."

19. Revision Notes

- **Core idea:** Go deep, then backtrack. Recursion or explicit stack.
- **Recursive pattern:** `dfs(u) { mark; for(v: adj[u]) if(!vis[v]) dfs(v); }`
- **Iterative:** Stack, push neighbors, mark visited on push.
- **Connected components:** Loop over all nodes, DFS from unvisited.
- **Cycle detection (directed):** Three states (0/1/2), back edge $\rightarrow$ cycle.
- **Cycle detection (undirected):** Parent tracking, `if(v != parent)`.

- **Topological sort:** Post-order DFS, add to result after processing children.
- **Complexity:** $O(V + E)$ time, $O(V)$ space.
- **Common trap:** Stack overflow, not marking visited, using DFS for shortest path.

20. Final Cheat Sheet

DFS – CHEAT SHEET	
WHEN TO USE:	Connectivity, cycle detection, topological sort, backtracking, all paths, flood fill
RECURSIVE:	<pre>void dfs(u) { vis[u]=1; for(v: adj[u]) if(!vis[v]) dfs(v); }</pre>
ITERATIVE:	<pre>stack; push; while(!st.empty()) { u=st.top(); pop; if(vis)continue; vis[u]=1; for(v) if(!vis) st.push(v); }</pre>
CYCLE (DIR):	state: 0=unvisited, 1=visiting, 2=done; if(state[v]==1) → cycle
TOPO SORT:	post-order, add to result, reverse
COMPLEXITY:	$O(V+E)$ time, $O(V)$ space
GRID DFS:	dr=[-1,1,0,0], dc=[0,0,-1,1]
BFS vs DFS:	BFS for SP, DFS for connectivity
TRAP:	Stack overflow on deep recursion
RELATED:	BFS, Backtracking, Union-Find

12. BACKTRACKING

1. Overview

Backtracking is a brute-force algorithmic technique that incrementally builds candidates for a solution and abandons ("backtracks") a candidate as soon as it determines that the candidate cannot lead to a valid solution. It is essentially DFS on the state space tree with pruning.

2. Intuition

Core idea: Try all possibilities, but stop exploring a branch as soon as you know it can't work. This is "DFS with pruning" — we explore the search space systematically and cut off dead ends early.

Analogy: Solving a Sudoku puzzle. You try placing a number in an empty cell. If it leads to a contradiction later, you erase it (backtrack) and try the next number. You don't keep exploring after placing a number that obviously violates the rules.

Step-by-step reasoning:

1. Start with an empty/partial solution.

2. If the current solution is complete and valid, save it.
3. Otherwise, choose the next decision to make (which element to add, which position to fill, etc.).
4. For each possible choice:
 - Make the choice (modify state).
 - Check if the choice is valid (pruning). If not, skip.
 - Recursively continue building the solution.
 - Undo the choice (restore state) — this is the backtracking step.
5. Return.

Why it works: Backtracking explores the entire search space but prunes invalid branches early. In the worst case (no pruning), it's $O(\text{branches}^{\text{depth}})$. But with good pruning, it can be much faster.

3. When to Use It

- **Generating all permutations** / combinations / subsets.
- **Constraint satisfaction** (N-Queens, Sudoku, crossword puzzles).
- **Combinatorial optimization** (traveling salesman, knapSack with small constraints).
- **Path finding with constraints** (rat in a maze, knight's tour).
- **Parsing ambiguous grammars** (recursive descent parsing).
- **Subset sum** / partition problems.
- **Generating valid parentheses** / bracket sequences.

Common trigger phrases:

- "all permutations"
- "all combinations"
- "all subsets"
- "all possible solutions"
- "N-Queens"
- "Sudoku"
- "generate all"
- "solve"
- "constraints"
- "pruning"

4. When Not to Use It

- **Large search space with no effective pruning:** Backtracking is exponential and may not finish in reasonable time.
- **Problem has overlapping subproblems:** Use DP with memoization instead (e.g., 0/1 knapsack with large n).
- **Only one solution needed, not all:** BFS/DFS or greedy may be faster.
- **Problem has a direct formula:** Use combinatorics / math instead of generating all possibilities.
- **Constraints are large ($n > 20-30$):** Branching factor explodes. Consider DP, greedy, or approximation.

5. Core Concepts

5.1 State Space Tree

The tree of all possible decisions. Each node represents a partial solution. Leaves are complete solutions.

5.2 Pruning

Cutting off branches that cannot lead to a valid solution. This is what makes backtracking feasible.

Types of pruning:

- **Validity check:** Current partial solution violates constraints.
- **Optimization bound:** Current partial solution cannot beat the best known solution (branch and bound).
- **Symmetry breaking:** Skip equivalent states.

5.3 The "Choose, Explore, Unchoose" Pattern

```
void backtrack(state, choices) {
    if (isSolution(state)) {
        process(state);
        return;
    }
    for (choice : choices) {
        if (isValid(choice, state)) {
            makeChoice(choice, state); // choose
            backtrack(state, choices); // explore
            undoChoice(choice, state); // unchoose (backtrack)
        }
    }
}
```

5.4 State Representation

The state must be easy to modify and restore. Common patterns:

- **Array/vector:** Push to add, pop to remove.
- **Bitmask:** Use bits to track used elements.
- **Boolean array:** Mark used/unused.

6. Step-by-Step Algorithm

Generate All Permutations:

1. Define `result` to store all permutations.
2. Define `current` (current permutation being built) and `used` (boolean array).
3. Define `backtrack()`:
 - If `current.size() == n`: add `current` to `result`, return.
 - For each `i` from 0 to `n-1`:
 - If `used[i]`: continue.
 - `used[i] = true`.
 - `current.push_back(nums[i])`.

- `backtrack()`.
 - `current.pop_back()`.
 - `used[i] = false`.
4. Call `backtrack()`.

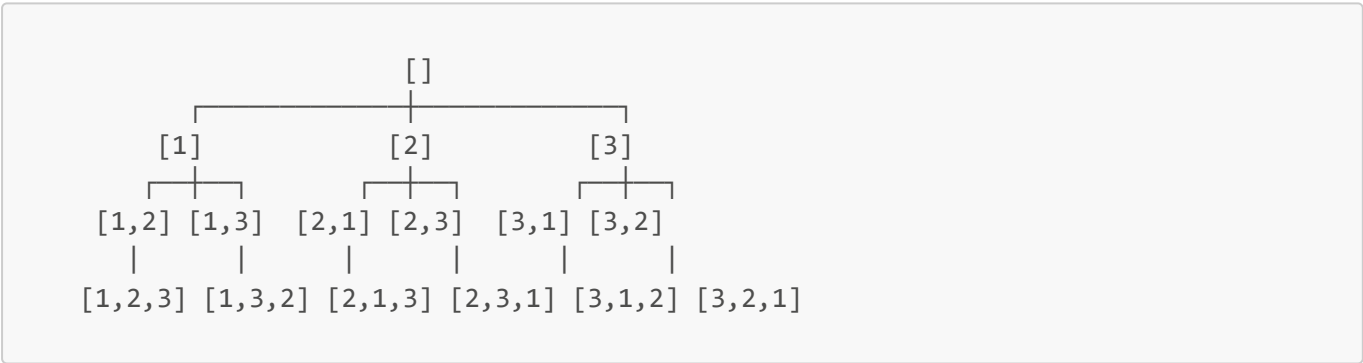
N-Queens:

1. Define `board` as vector of strings, `col` set, `diag1` set, `diag2` set.
2. Define `backtrack(row)`:
 - If `row == n`: add `board` to result, return.
 - For each `col` from 0 to n-1:
 - If `col` is in `col` set, or `row-col` in `diag1`, or `row+col` in `diag2`: continue.
 - Place queen: `board[row][col] = 'Q'`, add to sets.
 - `backtrack(row+1)`.
 - Remove queen: `board[row][col] = '.'`, remove from sets.

7. Dry Run

Problem: Generate all permutations of [1, 2, 3]

State space tree:



Backtracking steps:

Call	current	used	i	Action
bt()	[]	[F,F,F]	0	use 1 → [1]
bt()	[1]	[T,F,F]	0	skip (used)
bt()	[1]	[T,F,F]	1	use 2 → [1,2]
bt()	[1,2]	[T,T,F]	0,1 skip	2 → use 3 → [1,2,3]
bt()	[1,2,3]	[T,T,T]	size=3 → SAVE [1,2,3]	
(back)	[1,2]	[T,T,F]	2 done	pop 2, unuse 2
bt()	[1]	[T,F,F]	1 done	2 → use 3 → [1,3]
bt()	[1,3]	[T,F,T]	0 skip	1 use 2 → [1,3,2]
bt()	[1,3,2]	[T,T,T]	SAVE [1,3,2]	

Call	current	used	i	Action
...	continue	...	...	...

Result: `[[1,2,3], [1,3,2], [2,1,3], [2,3,1], [3,1,2], [3,2,1]]`

Problem: N-Queens (N=4) — Place 4 queens on 4×4 board so none attack each other.

Solution found:

```
. Q . .
. . . Q
Q . . .
. . Q .
```

Backtracking trace (row by row):

Row	Col Attempt	Valid?	Action
0	0	Yes	Place Q at (0,0)
1	0	No (col)	Skip
1	1	No (diag)	Skip
1	2	Yes	Place Q at (1,2)
2	0	No (diag)	Skip
2	1	No (col)	Skip
2	2	No (diag)	Skip
2	3	No (diag)	Skip
2	—	No valid col	Backtrack to row 1
1	3	Yes	Remove Q at (1,2), place at (1,3)
2	1	Yes	Place Q at (2,1)
3	0	No (diag)	Skip
3	2	Yes	Place Q at (3,2)

Solution found! `[[1,3,0,2]]` (col per row)

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;
```

```

class Backtracking {
public:
    // =====
    // 1. Generate All Permutations
    // =====
    vector<vector<int>> permute(vector<int>& nums) {
        vector<vector<int>> result;
        vector<int> current;
        vector<bool> used(nums.size(), false);

        function<void()> backtrack = [&]() {
            if (current.size() == nums.size()) {
                result.push_back(current);
                return;
            }

            for (int i = 0; i < nums.size(); i++) {
                if (used[i]) continue;

                used[i] = true;
                current.push_back(nums[i]);
                backtrack();
                current.pop_back();
                used[i] = false;
            }
        };

        backtrack();
        return result;
    }

    // =====
    // 2. Generate All Subsets
    // =====
    vector<vector<int>> subsets(vector<int>& nums) {
        vector<vector<int>> result;
        vector<int> current;

        function<void(int)> backtrack = [&](int start) {
            result.push_back(current); // add every subset

            for (int i = start; i < nums.size(); i++) {
                current.push_back(nums[i]);
                backtrack(i + 1);
                current.pop_back();
            }
        };

        backtrack(0);
        return result;
    }

    // =====
    // 3. N-Queens

```

```
// =====
vector<vector<string>> solveNQueens(int n) {
    vector<vector<string>> result;
    vector<string> board(n, string(n, '.'));
    vector<bool> col(n, false), diag1(2*n-1, false), diag2(2*n-1, false);

    function<void(int)> backtrack = [&](int row) {
        if (row == n) {
            result.push_back(board);
            return;
        }

        for (int c = 0; c < n; c++) {
            int d1 = row - c + n - 1; // diagonal index
            int d2 = row + c;          // anti-diagonal index

            if (col[c] || diag1[d1] || diag2[d2]) continue;

            // Place queen
            board[row][c] = 'Q';
            col[c] = diag1[d1] = diag2[d2] = true;
            backtrack(row + 1);
            // Remove queen
            board[row][c] = '.';
            col[c] = diag1[d1] = diag2[d2] = false;
        }
    };

    backtrack(0);
    return result;
}

// =====
// 4. Combination Sum (unique combinations that sum to target)
// =====
vector<vector<int>> combinationSum(vector<int>& candidates, int target) {
    vector<vector<int>> result;
    vector<int> current;

    function<void(int, int)> backtrack = [&](int start, int remaining) {
        if (remaining == 0) {
            result.push_back(current);
            return;
        }
        if (remaining < 0) return;

        for (int i = start; i < candidates.size(); i++) {
            current.push_back(candidates[i]);
            backtrack(i, remaining - candidates[i]); // can reuse same element
            current.pop_back();
        }
    };

    backtrack(0, target);
}
```

```
        return result;
    }
};

// Example usage
int main() {
    Backtracking bt;

    // Permutations
    vector<int> nums = {1, 2, 3};
    auto perms = bt.permute(nums);
    cout << "Permutations:\n";
    for (auto& p : perms) {
        cout << "  [";
        for (int x : p) cout << x << " ";
        cout << "]\n";
    }

    // Subsets
    auto subs = bt.subsets(nums);
    cout << "Subsets (count=" << subs.size() << "):\n";

    // N-Queens
    auto solutions = bt.solveNQueens(4);
    cout << "N-Queens solutions: " << solutions.size() << "\n";
    for (auto& board : solutions) {
        for (auto& row : board) cout << row << "\n";
        cout << "\n";
    }

    // Combination Sum
    vector<int> candidates = {2, 3, 6, 7};
    auto combos = bt.combinationSum(candidates, 7);
    cout << "Combination Sum:\n";
    for (auto& c : combos) {
        cout << "  [";
        for (int x : c) cout << x << " ";
        cout << "]\n";
    }

    return 0;
}
```